



CUSTOMER SERVICE PERFORMANCE EVALUATION USING MACHINE LEARNING

A Comparative Analysis of Machine Learning Models



Prasanth Murugan

Candidate Number: 925625

Written under the supervision of Prof. Marios Kremantzis

BDM200J - MSc Business Analytics Research Project

Master of Science (MSc) in Business Analytics

January 2025

This project is submitted in partial fulfillment of the requirements for the master's in Business Analytics at Aston University, dated January 27th, 2025

Declaration:

I declare that I have personally prepared this report and that it has not in whole or in part been submitted for any other degree or qualification. Nor has it appeared in whole or in part in any textbook, journal or any other document previously published or produced for any purpose. The work described here is my/our own, carried out personally unless otherwise stated. All sources of information, including quotations, are acknowledged by means of reference, both in the final reference section and at the point where they occur in the text.

Acknowledgement:

I would like to express my heartfelt gratitude to everyone who supported and contributed to the successful completion of my MSc program. I am deeply thankful to my family for their unwavering encouragement, unconditional love, and sacrifices throughout this academic journey. Their belief in me has been a constant source of inspiration and motivation.

I extend my sincere gratitude to my esteemed supervisor, Dr. Marios Kremantzis, for his invaluable guidance and support, which were instrumental in successfully completing my dissertation. I am also grateful to Aston University for providing the necessary resources and support that made this study possible. Additionally, I wish to acknowledge all my module teachers for their insightful knowledge, which greatly enhanced my analysis. Finally, my deepest thanks go to my colleagues and friends for their thoughtful discussions, collaboration, and encouragement throughout this journey.

Abstract:

In the fast-paced e-commerce industry, exceptional customer service is critical for maintaining customer satisfaction and driving business success. This project, titled *“Customer Service Performance Evaluation Using Machine Learning: A Comparative Analysis of Machine Learning Models,”* explores the use of machine learning to predict Customer Satisfaction Scores (CSAT) and optimize customer service strategies. Using a dataset of over 85,000 customer service interactions sourced from Kaggle, this study aims to address the challenge of class imbalance and improve predictive accuracy through advanced machine learning techniques.

A thorough methodology was employed, including data preprocessing, feature engineering, and addressing class imbalance using SMOTE (Synthetic Minority Oversampling Technique). Three machine learning models—Logistic Regression, Random Forest, and XGBoost—were implemented and evaluated. XGBoost emerged as the top-performing model, achieving 79% accuracy and a 75% F1 score on the training data and 71% accuracy with a 63% F1 score on the testing data after hyperparameter tuning and SMOTE application. While Random Forest performed reliably, Logistic Regression served as a baseline model, highlighting the incremental benefits of advanced ensemble and boosting methods.

The study concludes that machine learning, particularly XGBoost, can significantly enhance customer service performance by predicting satisfaction levels and identifying key drivers of satisfaction. However, challenges like class imbalance and overfitting persist, limiting the ability of models like Random Forest and Logistic Regression to generalize effectively. Recommendations include improving data quality, employing real-time sentiment analysis for personalized customer interactions, and integrating AI-driven tools like chatbots to streamline customer support. Future work could explore hybrid models and advanced frameworks like transformers to further improve scalability, predictive power, and real-time decision-making capabilities.

List of Figures:

Figure 1: E-Commerce Workflow Diagram	11
Figure 2: Machine Learning in E-Commerce	12
Figure 3: Distribution of CSAT Score	29
Figure 4: Distribution of Channel.....	30
Figure 5: Bar Chart Showing Category Distribution.....	31
Figure 6: Bar plot against Category and Sub-Category.....	32
Figure 7: Top Agents of the dataset.....	33
Figure 8: Details of Missing Values	34
Figure 9: Correlation and Cramer's Matrix to check relation with CSAT Score	35
Figure 10: Final Columns after handling Missing Values	36
Figure 11: Correlation of the new columns	37
Figure 12: Distribution of Sentiments	41
Figure 13: Distribution of Polarity and Subjectivity	42
Figure 14: Result of Confusion Matrix on Testing Data	51
Figure 15: Confusion Matrix on Testing Data after SMOTE.....	52
Figure 16: AUC-ROC Curve of Each Class.....	53
Figure 17: Precision-Recall Curve of each class.....	53
Figure 18: Details of Parameter Grid.....	55
Figure 19: Best Parameters of Random Forest	55
Figure 20: Result of Confusion Matrix on Testing Data	55
Figure 21: Result of Confusion matrix on Training Data after SMOTE	57
Figure 22: Result of confusion matrix on Testing data after SMOTE	58
Figure 23: AUC-ROC Curve of each class	59
Figure 24: Precision-Recall Curve of each Class	60
Figure 25: Top Features of Random Forest.....	61
Figure 26: Details of Parameter Grid.....	62
Figure 27: Best parameters of XGBoost	62
Figure 28: Result of Confusion Matrix on Testing Data	63
Figure 29: Result of confusion Matrix on training data after SMOTE	65
Figure 30: Result of confusion Matrix on Testing data after SMOTE.....	66
Figure 31: AUC-ROC Curve of each class	67
Figure 32: Precision-Recall Curve of each class.....	68
Figure 33: Top Features of XGBoost Classifier.....	69

List of Tables:

Table 1: Literature Review Summary	23
Table 2: DataSet Overview	27
Table 3: Descriptive Statistics of the Dataset	28
Table 4: Result of Baseline Model.....	49
Table 5: Result of Logistic Regression.....	50
Table 6: Result of Logistic Regression before and after SMOTE	52
Table 7: Result of Random Forest.....	54
Table 8: Result of Random Forest before and after SMOTE	56
Table 9: Result of XGBoost Classifier.....	62
Table 10: Result of XGBoost Classifier before and after SMOTE	64
Table 11: Overall Comparison of all Models	71
Table 12: Comparison of Top predictors	72

Contents

Declaration:.....	1
Acknowledgement:	1
Abstract:.....	2
List of Figures:.....	3
List of Tables:.....	4
CHAPTER ONE: INTRODUCTION.....	9
1. INTRODUCTION:	10
1.1 Background of E-commerce:	10
1.2 The Crucial Role of Customer Service in Driving E-Commerce Success:.....	11
1.3 Revolutionizing E-Commerce Customer Service with Machine Learning:.....	12
1.4 Aim, Research Questions, and Impact of Research:	13
1.4.1 Aim of the Research:.....	13
1.4.2 Research Questions	13
1.4.3 Impact of Research.....	14
1.5 Structure of the Research:.....	14
CHAPTER TWO: LITERATURE REVIEW	15
2. LITERATURE REVIEW:	16
2.1 Enhancing Customer Service through Machine Learning:	16
2.1.1 Neural Networks for Predicting Customer Behaviours	16
2.1.2 Automation of Ticket Classification	16
2.1.3 NLP and Real-Time Data Processing	17
2.2 Customer Satisfaction and Retention Strategies with Machine Learning:.....	17
2.2.1 Ensemble Learning and Boosting Techniques	18
2.2.2 Key Predictors of Satisfaction	18
2.2.3 ML Techniques for Customer Satisfaction	18
2.3 Sentiment Analysis and Its Role in Customer Experience Enhancement:.....	19
2.3.1 Advancements in Sentiment Analysis	19
2.3.2 Recommendations for Improvement	19
2.4 Revolutionizing E-Commerce Through Personalized Machine Learning Solutions:	20
2.4.1 Predictive Models for tailored Customer Experience	20
2.4.2 Hyper-Personalization Strategies.....	21
2.5 Emerging Trends in Machine Learning for E-Commerce:	21
2.5.1 Predicting Customer Behaviour and Automating Decisions	21

2.5.2 Broader Applications in E-Commerce.....	22
2.5.3 Advancements in Voice Recognition Technology	22
2.6 Summary of the Literature Review:	23
2.7 Literature Gap:.....	23
CHAPTER THREE: METHODOLOGY	25
3. METHODOLOGY:	26
3.1 Dataset:	26
3.1.1 Overview of the Dataset:	27
3.2 Sampling Technique:.....	28
3.3 Descriptive Statistics of the Dataset:	28
3.4 Exploratory Data Analysis:	29
3.4.1 Visualization of the target variable:	29
3.4.2 Univariate Visualization:	30
3.4.3 Bivariate Visualization:.....	32
3.5 Data Preprocessing:	34
3.5.1 Handling Missing values:	34
3.5.2 Outliers Detection:	37
3.5.3 Feature Engineering:	37
3.5.4 Encoding:.....	38
3.6 Scaling:.....	38
3.7 Data Splitting:	39
3.8 Sentiment Analysis:	39
3.8.1 Pre-Processing:	39
3.8.2 Implementation of Sentiment Analyzer:	40
3.8.3.1 Visualization of the Sentiments:	41
3.8.3.2 Distribution of Polarity and Subjectivity:.....	42
3.9 Machine Learning Models:.....	42
3.9.1 Logistic Regression:.....	42
3.9.2 Random Forest:	43
3.9.3 XGBoost:	44
3.10 Justification for Model Selection:	44
3.11 Evaluation Metrics:	45
3.11.1 Accuracy:	45
3.11.2 F1 Score:	45

3.11.3 Precision:	46
3.11.4 Recall:	47
3.11.5 Confusion Matrix:	47
CHAPTER FOUR: MODEL ANALYSIS AND RESULTS.....	48
4. MODEL ANALYSIS AND RESULTS:.....	49
4.1 Baseline Model:	49
4.2 Logistic Regression:	50
4.2.1 Hyperparameter Tuning and Model Optimization:	50
4.2.2 Strategies to Improve Minority Class Performance:	52
4.3 Random Forest:	54
4.3.1 Hyperparameter Tuning and Model Optimization:	55
4.3.2 Strategies to Improve Minority Class Performance:	56
4.3.2 Feature Importance:	61
4.4 XGBoost Classifier:	62
4.4.1 Hyperparameter Tuning and Model Optimization:	62
4.4.2 Strategies to Improve Minority Class Performance:	64
4.4.2 Feature Importance:	69
CHAPTER FIVE: SUMMARY OF FINDINGS, CONCLUSION AND RECOMMENDATIONS	70
5.1 Summary of Findings:	71
5.1.1 Top Predictors for CSAT score:	72
5.2 Conclusion:	73
5.2.1 Limitations of the study:	74
5.3 Recommendations:	75
5.3.1 Future Studies:	76
6. References:.....	77
Appendix:	81
Appendix 1: Stratified Sampling	81
Appendix 2: Feature Engineering.....	81
Appendix 3: Encoding.....	82
Appendix 4: Scaling.....	83
Appendix 5: Data Splitting	83
Appendix 6: Sentiment Analysis.....	84
Appendix 7: Baseline Method:	86
Appendix 8: Logistic Regression:.....	88

Appendix 9: Random Forest:.....	94
Appendix 10: XGBoost Classifier:.....	101

CHAPTER ONE: INTRODUCTION

1. INTRODUCTION:

1.1 Background of E-commerce:

E-commerce, or electronic commerce, refers to the buying and selling of goods and services over the Internet. Its origins can be traced back to the 1960s with the development of technologies like Electronic Data Interchange (EDI) and Electronic Funds Transfer (EFT), which allowed businesses to exchange documents and conduct transactions electronically (Britannica, n.d.). However, the 1990s marked a breakthrough with the commercialization of the Internet and the rise of online marketplaces like Amazon in 1994 and eBay in 1995, which revolutionized digital retail and connected buyers and sellers globally (Laudon & Traver, 2020).

The rapid growth of e-commerce has been driven by advancements in digital infrastructure, mobile technology, and innovative tools. By 2023, global retail e-commerce sales reached an estimated 5.8 trillion U.S. dollars, with projections indicating a 39% growth over the coming years, expected to surpass 8 trillion dollars by 2027 (Statista, 2023). Platforms like Amazon and Alibaba dominate the market by offering seamless, cross-device shopping experiences. While e-commerce provides unparalleled convenience for consumers and cost efficiencies for businesses, challenges such as maintaining traditional business sustainability and building robust digital infrastructure remain (Sharma, Mishra, & Srivastav, 2023).

The COVID-19 pandemic further accelerated the shift to e-commerce as lockdowns and social distancing measures forced consumers and businesses to rely heavily on online channels. Companies adapted by expanding their digital presence, offering contactless delivery, and adopting innovative technologies like artificial intelligence and machine learning to enhance customer experience (Supriya Jawale et al., 2024). As of 2023, e-commerce continues to grow, with projections indicating a compound annual growth rate (CAGR) in retail e-commerce sales worldwide from 2024 to 2029, varying by country (Statista, 2023). This comprehensive transformation reflects the evolutionary nature of e-commerce, driven by technological advancements and adaptive integration into broader business ecosystems, making it a vital component of the modern digital economy. The E-Commerce Workflow is shown in Figure 1.

E-Commerce Workflow Diagram

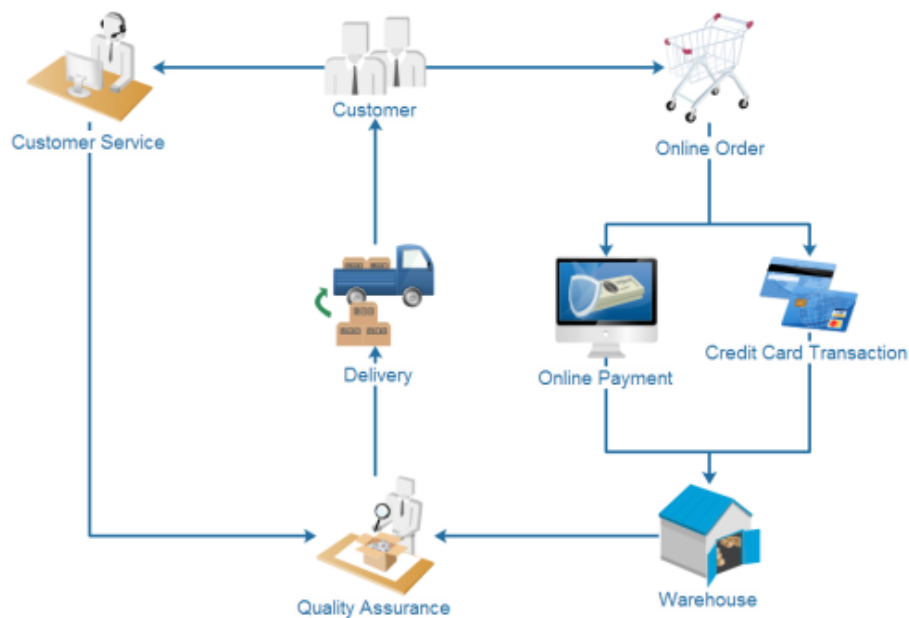


Figure 1: E-Commerce Workflow Diagram

Source: E-Commerce Workflow Template - edrawsoft

1.2 The Crucial Role of Customer Service in Driving E-Commerce Success:

The rapid growth of e-commerce, fueled by advancements in digital infrastructure, mobile technology, and global connectivity, has transformed how businesses connect with their customers. Unlike traditional retail, where face-to-face interactions build trust, e-commerce relies on virtual communication tools like live chat, emails, social media, and AI-powered platforms to foster engagement. As competition increases and customer expectations continue to rise, delivering exceptional customer service has become a critical factor for success in today's digital marketplace.

In e-commerce, effective customer service means providing quick responses, accurate solutions, and personalized experiences tailored to individual needs. However, businesses face challenges such as handling large volumes of inquiries, resolving complaints efficiently, and ensuring consistency across multiple platforms. To tackle these issues, companies increasingly rely on tools like Customer

Satisfaction Scores (CSAT), feedback surveys, and review mechanisms to measure performance and improve service.

Providing exceptional customer service not only resolves immediate concerns but also strengthens customer loyalty, encourages repeat purchases, and enhances brand reputation (Sharma, Mishra, & Srivastav, 2023). On the other hand, poor service can result in negative reviews, break trust, and lead to customer attrition, underscoring its critical importance in the e-commerce industry.

1.3 Revolutionizing E-Commerce Customer Service with Machine Learning:

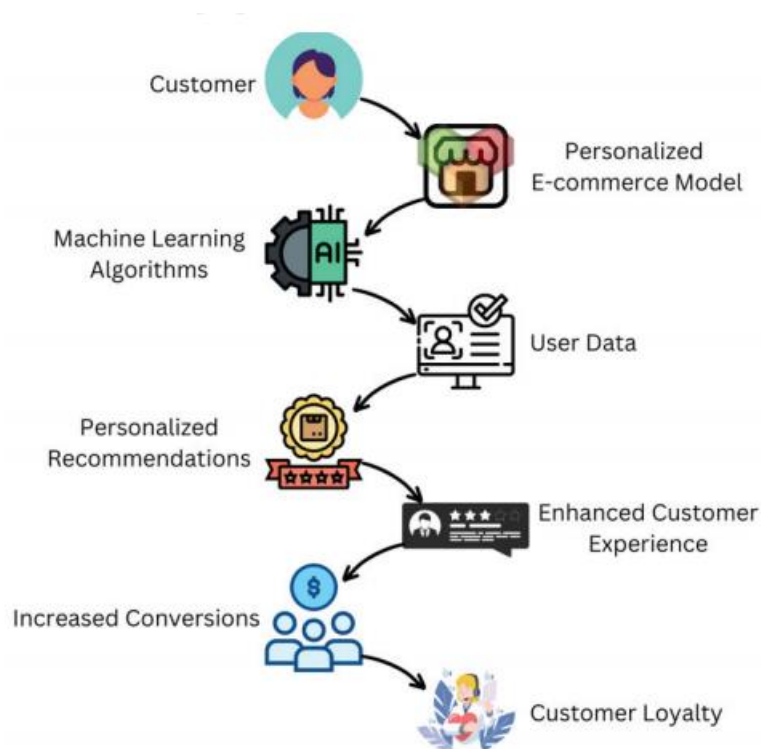


Figure 2: Machine Learning in E-Commerce

Source: (Gowda, Su and Kuo, 2024)

Machine learning (ML) is transforming the way e-commerce businesses deliver customer service, offering innovative solutions to meet the changing needs of customers while addressing operational challenges. By harnessing advanced algorithms, ML allows businesses to analyze large volumes of data, uncover meaningful patterns, and gain actionable insights that help improve service quality. Figure 2 shows the importance of Machine Learning in increasing customer satisfaction.

One popular application of ML in customer service is sentiment analysis, which helps businesses understand the emotions behind customer feedback. This insight allows companies to tailor their responses, improve communication, and ultimately enhance customer satisfaction (Panduro-Ramirez, 2024). ML also powers predictive analytics, which identifies customers who may be at risk of leaving, enabling businesses to take proactive steps to strengthen retention and reduce churn. Additionally, AI-driven chatbots like SuperAgent, designed for e-commerce, have transformed customer service by automating routine inquiries, delivering instant support, and enabling human agents to focus on complex issues, all while maintaining human-like interaction quality (Cui et al., 2017).

The integration of machine learning into customer service frameworks empowers e-commerce businesses to deliver scalable, proactive, and personalized support. These technologies not only improve customer satisfaction but also position organizations as industry leaders in a highly competitive, technology-driven market. By leveraging machine learning, businesses can foster stronger relationships with customers, optimize operations, and secure a sustainable competitive edge in the evolving digital landscape.

1.4 Aim, Research Questions, and Impact of Research:

1.4.1 Aim of the Research:

The aim of this research is to evaluate customer service performance by using machine learning techniques in e-commerce sector and develop a reliable model for analyzing and predicting customer satisfaction score (CSAT). As customer service continues to evolve with technological advancements, businesses are increasingly leveraging machine learning algorithms to optimize service quality, predict satisfaction levels, and improve operational efficiency. This study compares and evaluates the performance of various machine learning models, identifying the most effective approaches to predict and analyze customer satisfaction in the e-commerce industry.

1.4.2 Research Questions

The research questions formulated to achieve the above aim are as follows:

- What are the top predictors influencing customer satisfaction in e-commerce?
- What is the predictive accuracy of machine learning algorithms for customer satisfaction?
- What insights can predictive models provide about customer service performance and potential optimization strategies?

- How do various machine learning techniques compare in terms of performance and scalability when evaluating customer service?

1.4.3 Impact of Research

Answering the research questions will have significant implications for various stakeholders, including businesses, Customers, and researchers:

- **Businesses** can leverage findings to improve service quality, optimize resource allocation, and enhance operational efficiency through predictive analytics.
- **Customers** will benefit from improved service responsiveness, personalized solutions, and better overall experiences.
- **Researchers** gain insights into the applicability of machine learning in performance evaluation, providing a foundation for future studies.

In summary, this research provides a comprehensive evaluation of machine learning techniques for customer service performance analysis. It offers practical recommendations for businesses to implement predictive models, enhance customer satisfaction, and streamline operations. Additionally, it contributes to the academic domain by advancing the understanding of machine learning's role in performance evaluation, laying a strong foundation for future research and innovation.

1.5 Structure of the Research:

The research paper is structured as follows: Section two provides a thorough Literature Review, focusing on previous studies related to customer service optimization, machine learning applications, and gaps in the existing literature. Section three details the Methodology, covering data collection, preprocessing, sampling techniques, and the implementation of machine learning algorithms such as Logistic Regression, Random Forest, and XGBoost. Section four presents the Results and Discussion, showcasing findings through visualizations and metrics like accuracy, precision, recall, and F1-score, while comparing model performances and discussing challenges and their resolutions. Finally, Section five concludes with key findings, strategies for improving service performance, and recommendations for future research directions.

CHAPTER TWO: LITERATURE REVIEW

2. LITERATURE REVIEW:

The role of customer service has evolved from a basic support function to a strategic driver of organizational success in recent years. With the rise of digital channels such as social media, chatbots, and AI-driven systems, the scope of customer service has expanded, pushing businesses to leverage advanced analytics, machine learning (ML), and natural language processing (NLP). This literature review examines the key research findings, methodologies, and innovations in customer service performance. It also highlights several gaps in existing literature that provide opportunities for future research

2.1 Enhancing Customer Service through Machine Learning:

Machine learning (ML) has become an essential tool for modern e-commerce platforms, revolutionizing customer service by enhancing automation, improving response times, and boosting customer satisfaction. Studies highlight various ML applications, such as neural networks, natural language processing (NLP), and real-time data processing, which collectively optimize customer service operations. This critical analysis synthesizes findings from key research, emphasizing their transformative impact and identifying future research directions.

2.1.1 Neural Networks for Predicting Customer Behaviours

Customer behavior prediction is a critical challenge in modern customer service, particularly in minimizing wait times and optimizing resources. Neural networks have demonstrated exceptional capabilities in addressing these challenges. Kyritsis and Deriaz (2019) demonstrate that fully connected neural networks outperform traditional queueing theory, achieving a Mean Absolute Error (MAE) of just 3.35 minutes in bank queues. Neural networks' ability to adapt dynamically through continuous training ensures reliability across changing conditions, with future research emphasizing the expansion of such predictive capabilities to other industries.

2.1.2 Automation of Ticket Classification

Classifying service tickets efficiently remains a challenge for IT helpdesks dealing with high ticket volumes and diverse queries. ML-based solutions have transformed this process by automating classification tasks with high accuracy. Dlodlo and Sibanda (2023) evaluate algorithms like SVM, KNN, and Random Forest, finding SVM to be the most effective with 94.6% accuracy on test data and 99.9% on training data using preprocessing techniques such as TF-IDF vectorization. However, issues like dataset imbalance and overfitting in other algorithms highlight the need for robust data preprocessing and exploring deep learning models to improve scalability and reliability.

2.1.3 NLP and Real-Time Data Processing

Natural language processing (NLP) has emerged as a powerful tool for addressing communication gaps in customer service. By automating request classification and sentiment analysis, NLP reduces the time and effort needed to address customer queries. Arias-Barahona et al. (2023) reported an impressive 98.97% accuracy using SVM after preprocessing, hyperparameter optimization, and dataset balancing. This integration improves response times and customer satisfaction but relies heavily on high-quality datasets, which are not always available. Challenges such as scalability for diverse datasets and risks of data bias emphasize the need for advanced AI techniques to improve adaptability and reliability.

Real-time data processing has also become critical for customer service optimization. Nitesh Upadhyaya (2024) highlights the use of platforms like Apache Kafka combined with ML models to detect complaints, provide personalized responses, and adapt to customer behavior. By employing supervised learning for complaint prediction and unsupervised learning for pattern detection, these systems have significantly reduced churn and improved satisfaction in sectors like retail and telecommunications. Despite their success, challenges like scalability, privacy risks, and maintaining real-time accuracy persist, suggesting future integration with IoT and advanced AI techniques to enhance performance.

The integration of machine learning (ML) into customer service is helping tackle real-world challenges like inefficiencies, handling large volumes of data, and staying responsive in real-time. Neural networks make it easier to predict customer behavior and use resources more effectively, while natural language processing (NLP) enhances communication and ensures quicker, more accurate responses. Real-time data processing adds a layer of personalization and instant support, creating smoother, more customer-friendly interactions. Beyond operational improvements, ML boosts customer satisfaction and retention by addressing common problems such as delays, miscommunication, and lack of personalization. It also paves the way for powerful predictive tools to identify at-risk customers and enable proactive engagement strategies.

2.2 Customer Satisfaction and Retention Strategies with Machine Learning:

In today's competitive business landscape, customer retention has become a crucial focus for organizations seeking sustainable growth and profitability. With the advent of machine learning, businesses are now equipped with advanced tools to predict customer churn and enhance retention strategies. A series of studies highlight the evolution of machine-learning techniques for churn

prediction and their application across industries, providing valuable insights into improving customer satisfaction and loyalty.

2.2.1 Ensemble Learning and Boosting Techniques

Sabbeh (2018) evaluated ten ML algorithms for predicting customer churn in the telecommunications industry. The study highlighted the superior performance of ensemble learning techniques such as Random Forest and AdaBoost, achieving 96% accuracy. However, the study's reliance on a single dataset and static data posed challenges, leading to recommendations for hybrid models and deep learning techniques to improve scalability and runtime efficiency.

2.2.2 Key Predictors of Satisfaction

Khalid, Jugurnauth, and Muhammad (2022) explored customer satisfaction predictors in telecom services, emphasizing call quality and call-drop rates as critical determinants. Their ML models achieved 90.3% accuracy, and 92.1% F1-score, showcasing robust performance. However, the study identified gaps in using multiple performance metrics and recommended integrating deep learning and real-time analytics to enhance scalability and effectiveness.

Yu and Weng (2022) tested eight machine-learning classifiers using IBM Telco Customer Churn data. Boosting algorithms emerged as top performers, achieving 81.0% and 81.1% accuracy, respectively. The study identified tenure, monthly charges, and contract type as key predictors. Boosting algorithms demonstrated efficiency in handling large datasets, though issues with imbalanced data and hyperparameter optimization highlighted the need for advanced tuning techniques like grid search and Bayesian optimization.

Nagaraj et al. (2023) extended ML applications to e-commerce churn prediction, focusing on Random Forest as the most effective algorithm. The model utilized features such as customer tenure, order history, and satisfaction scores to predict churn. While the study validated the cost-effectiveness of ML-based retention strategies, it emphasized the need for hybrid models and deep learning to improve accuracy and scalability further.

2.2.3 ML Techniques for Customer Satisfaction

Zaghoul et al. (2024) compared traditional ML techniques with deep learning models for customer satisfaction prediction in e-commerce. Random Forest achieved high accuracy (92%) and strong performance across weighted metrics, such as F1 scores and precision. The study emphasized the importance of feature engineering and resampling techniques to improve model performance.

Manzoor et al. (2024) reviewed 212 studies on churn prediction and highlighted the effectiveness of boosting algorithms (e.g., XGBoost, LightGBM) and deep learning models. However, challenges like interpretability and real-time prediction persisted. The study recommended the integration of profit-driven evaluation metrics, and behavioral data to enhance trust and scalability.

The collective findings underscore the transformative role of ML in customer retention and satisfaction strategies. Ensemble techniques, boosting algorithms, and deep learning models consistently outperform traditional methods, offering high accuracy, adaptability, and predictive power. However, challenges such as imbalanced data, scalability, and model interpretability remain. Addressing these issues through hybrid frameworks, advanced tuning techniques, and explainable AI is essential for future progress. Additionally, knowing the customer feedback is very important for the organization. Analyzing customer feedback through Sentiment analysis is very necessary to understand customer emotions retain the customers and improve the performance.

2.3 Sentiment Analysis and Its Role in Customer Experience Enhancement:

Customer feedback has become necessary for every E-Commerce sector to refine their services and enhance user satisfaction. Sentiment analysis, a powerful application of machine learning, has emerged as a key tool in analyzing customer reviews to deliver personalized product recommendations and improve decision-making strategies.

2.3.1 Advancements in Sentiment Analysis

Panduro-Ramirez (2024) demonstrates how traditional models like Logistic Regression achieve high accuracy, ranging from 80–90%, in classifying sentiments as positive, negative, or neutral. However, Huang et al. (2023) argue that deep learning models, such as LSTM, CNN, and BERT, are better suited for handling complex linguistic patterns and contextual nuances, making them ideal for advanced sentiment analysis tasks. Despite these advancements, the studies collectively highlight challenges such as detecting fake reviews, managing sarcasm, and processing multilingual data effectively.

2.3.2 Recommendations for Improvement

To address these limitations, researchers suggest integrating multimodal data, including images and videos, and adopting hybrid approaches that combine machine learning and deep learning techniques. Panduro-Ramirez (2024) emphasizes the need for ensemble methods to improve accuracy and fairness, while Jabbar et al. (2019) highlight the importance of real-time processing systems for faster insights, as demonstrated in their Flask-based application. Additionally, Huang et al. (2023) propose

developing universal language models and fine-grained sentiment analysis techniques to tackle scalability and language diversity challenges.

By leveraging machine learning advancements, integrating multimodal inputs, and enhancing scalability, e-commerce platforms can unlock deeper insights into customer behavior, improve marketing strategies, and boost sales. While current systems have set a strong foundation, the future of sentiment analysis promises even greater efficiency, transparency, and adaptability, positioning it as a critical tool for shaping the future of e-commerce.

2.4 Revolutionizing E-Commerce Through Personalized Machine Learning Solutions:

In the fast-paced world of e-commerce, businesses are increasingly turning to machine learning and predictive analytics to decode customer behavior, optimize marketing strategies, and drive sales. The ability to harness vast amounts of data—from browsing histories to purchase patterns—has revolutionized how companies make decisions, enhance personalization, and retain customers. Several studies highlight the foundational role of predictive analytics and machine learning in transforming raw data into actionable intelligence. Predictive models built using classification, clustering, and regression algorithms enable businesses to forecast trends, identify purchase likelihood, and develop targeted strategies for customer retention and revenue growth.

2.4.1 Predictive Models for tailored Customer Experience

Machine learning has emerged as a disruptive force in e-commerce, redefining customer experiences and reshaping business models. Gupta and Bansal (2024) emphasize how AI algorithms, including Support Vector Machines (SVM) and K-Nearest Neighbors (KNN), can process large datasets and uncover complex patterns that traditional techniques often miss. Similarly, Krishnan and Mariappan (2024) discuss how AI-driven customization enhances engagement and loyalty by tailoring content and product recommendations to individual preferences. The work of Paripati et al. (2024) further explores how recommender systems powered by AI optimize user experiences, increasing conversion rates and customer satisfaction. As businesses shift towards hyper-personalization, predictive analytics becomes indispensable in turning big data into actionable insights, fine-tuning operations, and streamlining decision-making processes (Vanneschi et al., 2018).

While machine learning and predictive analytics have unlocked new possibilities, they also bring challenges related to data diversity, scalability, and transparency. Kasemrat and Kraiwanit (2024) address these concerns through their benchmarking study on machine-learning models for predictive

analytics in Thailand's e-commerce sector. The findings underscore the importance of integrating demographic and behavioral data to refine predictions and support dynamic pricing, inventory optimization, and personalized marketing strategies. However, the study highlights gaps in real-time analytics and multi-platform data integration, recommending the adoption of hybrid and deep learning approaches to overcome these limitations.

2.4.2 Hyper-Personalization Strategies

Gowda, Su, and Kuo (2024) take this vision further by exploring the role of machine learning in personalization, leveraging Long Short-Term Memory (LSTM) networks and attention mechanisms to generate dynamic recommendations. Their findings demonstrate how personalized e-commerce powered by AI improves customer engagement, satisfaction, and loyalty. The study also emphasizes ethical considerations, calling for explainable AI models to address privacy concerns and algorithmic biases. Future research recommends integrating multimodal inputs, such as visual and textual data, and enhancing real-time analytics to scale solutions and build customer trust.

The evolution of machine learning in e-commerce represents a paradigm shift, empowering businesses to predict customer behavior, deliver tailored experiences, and adapt to market demands in real-time. While the journey is not without challenges—such as ethical concerns, data scalability, and interpretability—the potential for growth and innovation is undeniable. As researchers continue to develop hybrid models, real-time systems, and ethical frameworks, the future of e-commerce is poised to become smarter, faster, and more customer-centric than ever before.

2.5 Emerging Trends in Machine Learning for E-Commerce:

Businesses are increasingly relying on machine learning (ML) and artificial intelligence (AI) to optimize processes, enhance decision-making, and deliver personalized customer experiences. From e-commerce platforms to IT service management, these technologies have redefined efficiency and scalability.

2.5.1 Predicting Customer Behaviour and Automating Decisions

Machine learning, as a subfield of AI, has shown remarkable success in predicting customer behaviour and automating decision-making. Tai et al. (2023) investigate the application of Decision Trees, Random Forest, and eXtreme Gradient Boosting (XGBoost) to forecast resolution times for customer service tickets, achieving up to 70.21% accuracy with Random Forest and superior error rates with XGBoost. Their findings emphasize the importance of data preprocessing, feature engineering, and model regularization to prevent overfitting while recommending hybrid approaches and deep learning

models for improved performance and scalability. Similarly, Zuev et al. (2018) highlights feature engineering and sampling techniques, such as SMOTE oversampling, to address imbalanced data and enhance predictive accuracy, ultimately optimizing Service Level Agreements (SLAs) and reducing costs.

2.5.2 Broader Applications in E-Commerce

The broader role of AI in e-commerce, as explored by Bawack et al. (2022), underscores its impact on recommender systems, sentiment analysis, and personalization. Their bibliometric study highlights a surge in AI research, led by China and the USA, focused on improving customer experiences and decision-making. Future directions include sustainable AI, voice assistants, and experimental methods to boost AI adoption in e-commerce.

2.5.3 Advancements in Voice Recognition Technology

Voice recognition technology, another AI advancement, has significantly improved user interactions and service automation. Barnwal and Gupta (2022) evaluate AI-based voice recognition systems, demonstrating up to 90.5% accuracy in social conversations using Natural Language Processing (NLP) and Deep Learning (DL) models. Their findings emphasize AI's role in healthcare, security, and education, particularly in automating customer service and enabling multilingual support. However, challenges related to data privacy and real-time processing persist, prompting recommendations for integrating conversational AI with biometric authentication for enhanced security and personalization.

Even Zuev, Kalistratov and Zuev, 2018 explores on SMOTE oversampling to handle imbalanced data, achieving an F1 score of 82.39%. The study highlights machine learning's role in improving incident management, reducing costs, and enhancing customer satisfaction. Future research may focus on ensemble methods and real-time applications for further optimization.

As machine learning and AI continue to evolve, their applications are reshaping industries by enabling businesses to analyze patterns, predict outcomes, and respond proactively to customer needs. In e-commerce, AI-driven personalization and predictive analytics empower businesses to optimize marketing strategies, improve engagement, and boost sales. Meanwhile, IT service management benefits from AI's ability to streamline workflows, reduce resolution times, and enhance service delivery. Despite challenges such as scalability, transparency, and ethical concerns, research emphasizes the need for hybrid approaches, multimodal AI systems, and explainable AI models to build trust and ensure long-term sustainability.

2.6 Summary of the Literature Review:

Table 1 provides the detailed summary of the relevant literatures and their findings, methodologies followed and limitations of the literature.

Authors	Year	Purpose	Methodology Adopted	Findings	Limitations
Dlodlo and Sibanda	2023	Automate IT ticket classification using ML algorithms.	SVM, KNN, Random Forest	SVM achieved 94.6% accuracy on test data and 99.9% on training data	Issues with dataset imbalance and overfitting;
Sabbbeh	2018	Predict customer churn and retention strategies.	Random Forest, AdaBoost, SVM	Ensemble techniques achieved 96% accuracy; MLP and SVM reached 94%.	needs hybrid models and runtime improvements.
Khalid et al.	2022	Analyze customer satisfaction predictors.	ML Models, Precision Metrics	Achieved 90.3% accuracy, 93.4% precision, and 92.1% F1-score.	Lacks scalability and real-time analytics.
Zaghloul et al.	2024	Compare traditional ML vs. deep learning for e-commerce satisfaction.	Random Forest, Deep Learning Models	Achieved 92% accuracy.	Requires resampling techniques for better performance.
Manzoor et al.	2024	Review ML methods for churn prediction and business recommendations.	XGBoost, LightGBM	Highlighted the effectiveness of boosting algorithms.	Requires profit-driven evaluation metrics for scalability.
Panduro-Ramirez	2024	Sentiment analysis for product recommendations in e-commerce.	Logistic Regression, SVM	Achieved 80–90% accuracy in classifying sentiments.	Lacks multimodal integration with sarcasm and fake reviews.
Jabbar et al.	2019	Real-time sentiment analysis for e-commerce applications.	SVM and NLP	High precision (87.88%), recall (99.98%), and F1-score (93.54%).	Needs faster insights for real-time processing.
Kasemra t and Kraiwani t	2024	Benchmark ML models for predictive analytics in e-commerce.	Logistic Regression, Random Forest, SVM, KNN	KNN achieved 94% accuracy and 96% AUC scores.	Gaps in real-time analytics and multi-platform integration.
Tai et al.	2023	Evaluate ML models for resolution time prediction in customer service.	Decision Trees, Random Forest, XGBoost	Achieved 70.21% accuracy.	Requires real-time applications, feature engineering, and model regularization.

Table 1: Literature Review Summary

2.7 Literature Gap:

A noticeable research gap emerged from the evaluation of existing literature on customer service optimization using machine learning (ML) and sentiment analysis techniques. Key issues identified include the lack of quality datasets and the presence of imbalanced data, leading to noise and

inaccurate model evaluations. Many studies failed to address these limitations effectively, reducing the reliability and generalizability of their findings. To overcome these challenges, this study employs robust data preprocessing techniques, including scaling, encoding, feature engineering, and the removal of columns with high missing values based on a correlation matrix, ensuring a clean and optimized dataset. Another significant gap observed is the narrow focus on either sentiment analysis or machine learning models, with limited integration between the two approaches. While some studies attempted to combine sentiment analysis with ML, they failed to incorporate sentiment analysis outputs as features for model training, potentially limiting predictive accuracy. This research addresses this gap by integrating sentiment analysis results directly into ML models, leveraging contextual and behavioral insights to enhance predictive performance. Furthermore, many studies lacked hyperparameter tuning, which is essential for optimizing performance and reducing overfitting or underfitting. This study rigorously applies hyperparameter tuning and selects best estimators for testing datasets, ensuring higher accuracy and reliability. Additionally, prior research showed inconsistencies in identifying the best-performing models, often lacking standardized comparisons. To resolve this, the current study benchmarks and tests multiple ML algorithms, including Random Forest, XGBoost, Decision Tree, using their optimized hyperparameters to validate performance. By addressing these gaps in data quality, integration of sentiment analysis, hyperparameter optimization, and comparative model evaluation, this research significantly enhances the predictive accuracy, scalability, and interpretability of ML models in customer service applications, offering a comprehensive and reliable methodology that contributes to both academic advancements and practical implementations.

CHAPTER THREE: METHODOLOGY

3. METHODOLOGY:

The main objective of this project is to evaluate Customer Service Performance using Machine Learning models in the E-Commerce Sector. We used the dataset of an E-commerce company from the Kaggle website. We followed data pre-processing to clean the data and performed various machine learning models to check their performance.

3.1 Dataset:

The dataset utilized for this project is taken from the trusted website [Kaggle](#). The dataset consists of 20 different columns and 85907 rows of detailed information about customer service interactions, including issue reporting times, the agents handling them, and how long it took to resolve issues. The dataset is representative of the broader e-commerce sector as it covers a wide range of common customer service scenarios, including product queries, returns, and cancellations, which are standard across the industry. The dataset's large size and diverse attributes ensure that the findings are generalizable across the e-commerce sector, as it captures a wide range of customer behaviours, service processes, and performance metrics. Key metrics such as resolution times, agent details, and CSAT scores align with industry standards, making the insights gained from this data applicable to a wide range of e-commerce platforms. This comprehensive dataset is ideal for exploring and predicting the factors that influence the Customer Satisfaction Score (CSAT), which is the target variable of the project.

3.1.1 Overview of the Dataset:

Variable Name	Description	Variable Type	Role
CSAT Score	Customer Satisfaction Score	Categorical	Target/ Dependent Variable
Unique Id	Unique Number	Categorical	Independent Variable
Channel Name	Communication channel used	Categorical	Independent Variable
Category	Issue Category	Categorical	Independent Variable
Sub-Category	Sub- Category of the issue	Categorical	Independent Variable
Customer Remarks	Remarks given by the customer	Categorical	Independent Variable
Order Id	Unique identifier of the order	Categorical	Independent Variable
Order Date Time	Date and time of the order	Datetime	Independent Variable
Issue Reported At	Date and time the issue reported	Datetime	Independent Variable
Issue Responded	Date and time the issue responded	Datetime	Independent Variable
Survey Response Date	Date and Survey of the response	Datetime	Independent Variable
Customer City	City of the Customer	Categorical	Independent Variable
Product Category	Product Category associated with the order	Categorical	Independent Variable
Item Price	Price of the item	Numerical	Independent Variable
Connected Handling Time	Connection handling time of the issue	Numerical	Independent Variable
Agent Name	Agent handling the issue	Categorical	Independent Variable
Supervisor	Supervisor of the agent	Categorical	Independent Variable
Manager	Manager of the Supervisor	Categorical	Independent Variable
Tenure Bucket	Tenure Bucket of the Agent	Numerical	Independent Variable
Agent Shift	Shift detail of the agent	Categorical	Independent Variable

Table 2: DataSet Overview

3.2 Sampling Technique:

We have used the Stratified Sampling technique for the dataset using the target variable CSAT Score. There are different types of sampling techniques, such as Stratified Sampling, Random Sampling, and Cluster Sampling. In this case, Stratified Sampling is preferred because it samples from a population that can be divided into sub-populations (strata) based on the target variable, CSAT Score. In this code, the strata are defined by the unique values in the CSAT Score column, ensuring each category of customer satisfaction is proportionally represented in the sample. A sample size of 40% of the data from each stratum is selected, preserving the equal distribution of the target variable across the dataset. This ensures that the sampled dataset is representative of the overall population and that all CSAT Score categories are equally distributed, maintaining the integrity of the analysis while reducing the dataset size. The code is provided in [Appendix 1](#).

3.3 Descriptive Statistics of the Dataset:

	Column	Data Type	Number of Missing Values	Percentage of Missing Values	Number of Duplicates	Number of Unique Values	Min Value	Max Value	Mean	Standard Deviation	Frequency
0	Unique id	object	0	0.000000	0	34363	nan	nan	nan	nan	1
1	channel_name	object	0	0.000000	0	3	nan	nan	nan	nan	27322
2	category	object	0	0.000000	0	12	nan	nan	nan	nan	17625
3	Sub-category	object	0	0.000000	0	56	nan	nan	nan	nan	8990
4	Customer Remarks	object	22803	66.359165	0	7680	nan	nan	nan	nan	564
5	Order_id	object	7375	21.462038	0	26988	nan	nan	nan	nan	1
6	order_date_time	object	27438	79.847510	0	6293	nan	nan	nan	nan	5
7	Issue_reported at	object	0	0.000000	0	21025	nan	nan	nan	nan	8
8	issue_responded	object	0	0.000000	0	20537	nan	nan	nan	nan	1330
9	Survey_response_Date	object	0	0.000000	0	31	nan	nan	nan	nan	1402
10	Customer_City	object	27492	80.004656	0	1275	nan	nan	nan	nan	275
11	Product_category	object	27443	79.862061	0	9	nan	nan	nan	nan	1925
12	Item_price	float64	27439	79.850421	0	1821	1.000000	164999.000000	5771.746389	12934.216967	nan
13	connected_handling_time	float64	34254	99.682798	0	100	0.000000	1115.000000	439.422018	215.990367	nan
14	Agent_name	object	0	0.000000	0	1371	nan	nan	nan	nan	188
15	Supervisor	object	0	0.000000	0	40	nan	nan	nan	nan	1691
16	Manager	object	0	0.000000	0	6	nan	nan	nan	nan	10056
17	Tenure Bucket	object	0	0.000000	0	5	nan	nan	nan	nan	12339
18	Agent Shift	object	0	0.000000	0	5	nan	nan	nan	nan	16579
19	CSAT Score	int64	0	0.000000	0	5	1.000000	5.000000	4.242179	1.378899	nan

Table 3: Descriptive Statistics of the Dataset

Table 3 describes the statistics of the dataset. It has a minimum number of duplicates but Columns such as Connected Handling Time, order date time, customer city, product category, and item price have quite many missing values. In terms of CSAT Scores the values are ranges from 1 to 5. 1 be the lowest and 5 be the highest.

3.4 Exploratory Data Analysis:

Exploratory Data Analysis is a crucial step to understand the data in depth visually and statistically.

Key Outcomes from EDA:

- 1) **Outlier Detection:** It will help to know the outliers of the dataset
- 2) **Data quality assessment:** It will help to understand the quality of data by identifying the missing values, or any inconsistencies in data.
- 3) **Patterns and Insights:** It will also help to understand the hidden patterns and insights within the data.
- 4) **Data Preprocessing technique:** It will also help to know what data cleaning techniques need to be performed to improve the data quality.

3.4.1 Visualization of the target variable:

The **CSAT score** (Customer Satisfaction score) is the target variable used in this model. Knowing the distribution of target variable is essential to understand how the target variable is distributed.

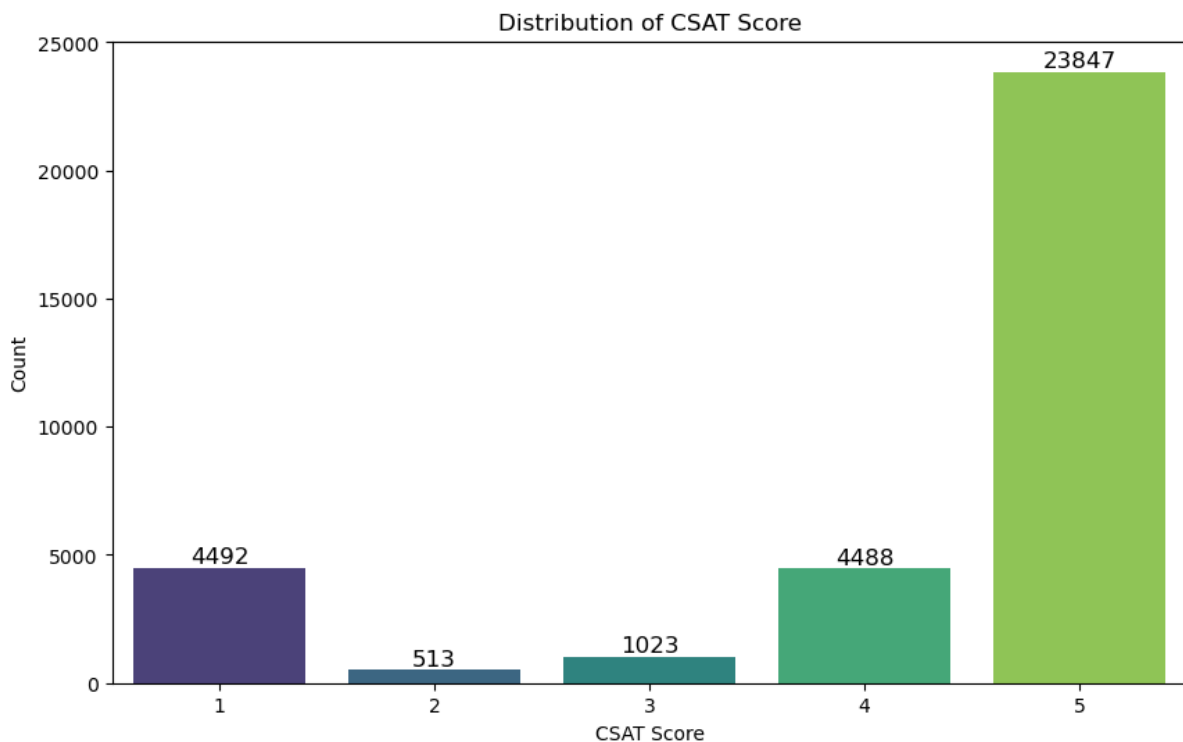


Figure 3: Distribution of CSAT Score

Figure 3 shows the distribution of the CSAT score ranges from 1(least satisfied) to 5 (most satisfied). Most responses are highly positive, with 23,847 customers rating 5 and 4,488 rating 4. Very few customers (4492) rated 1 and quite a less number rated 2. This suggests our dataset has a high level of customer satisfaction.

Type of Visualization:

There are two main types of Visualization

- 1)Univariate Visualization: used to understand the distribution of the single variable
- 2)Bivariate Visualization: used to understand the distribution of two variables.

Histograms, Bar plots, and Box plots are the most common plots used to evaluate the distribution of the data.

3.4.2 Univariate Visualization:

3.4.2.1 Distribution of Channel Name:

It is good to know what type of channels customers use to communicate and knowing that in a visually will help to understand the distribution very clearly.

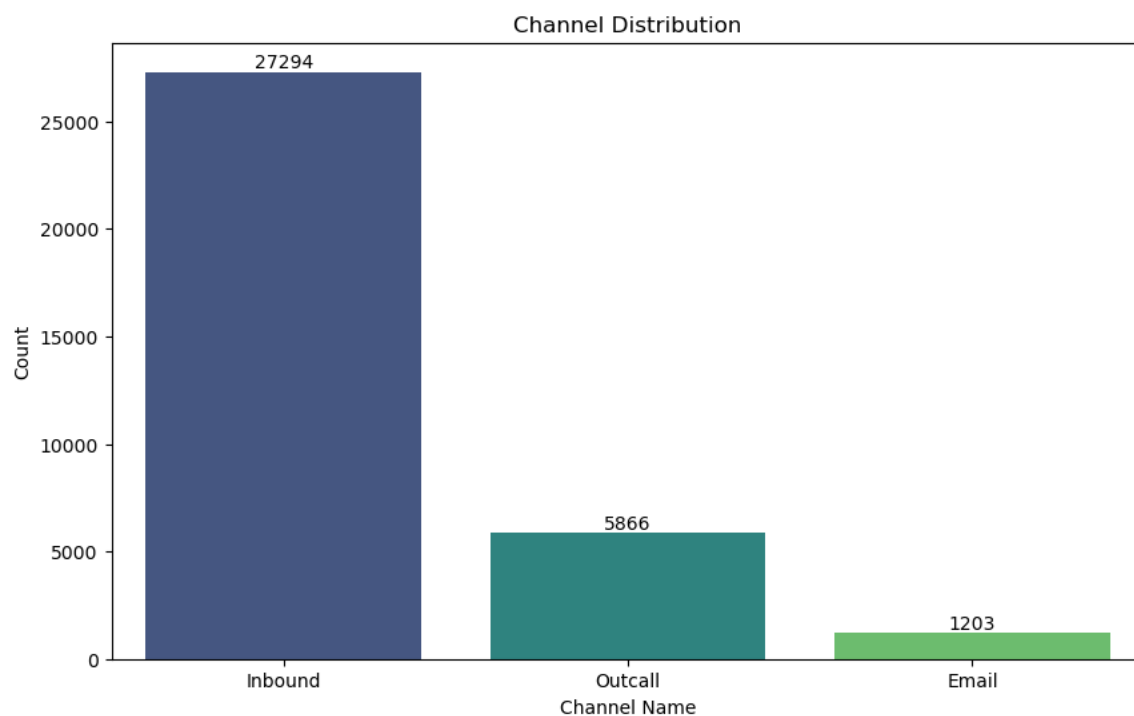


Figure 4: Distribution of Channel

Figure 4 shows the Channel Distribution based on the count of interactions through different communication channels. The Inbound channel has the highest count, exceeding 26,000 interactions, followed by Outcall with around 6,000 interactions, and Email with the lowest count, below 2,000 interactions. This suggests that most interactions occur through inbound calls, while email is the least used channel.

3.4.2.2 Distribution of Category:

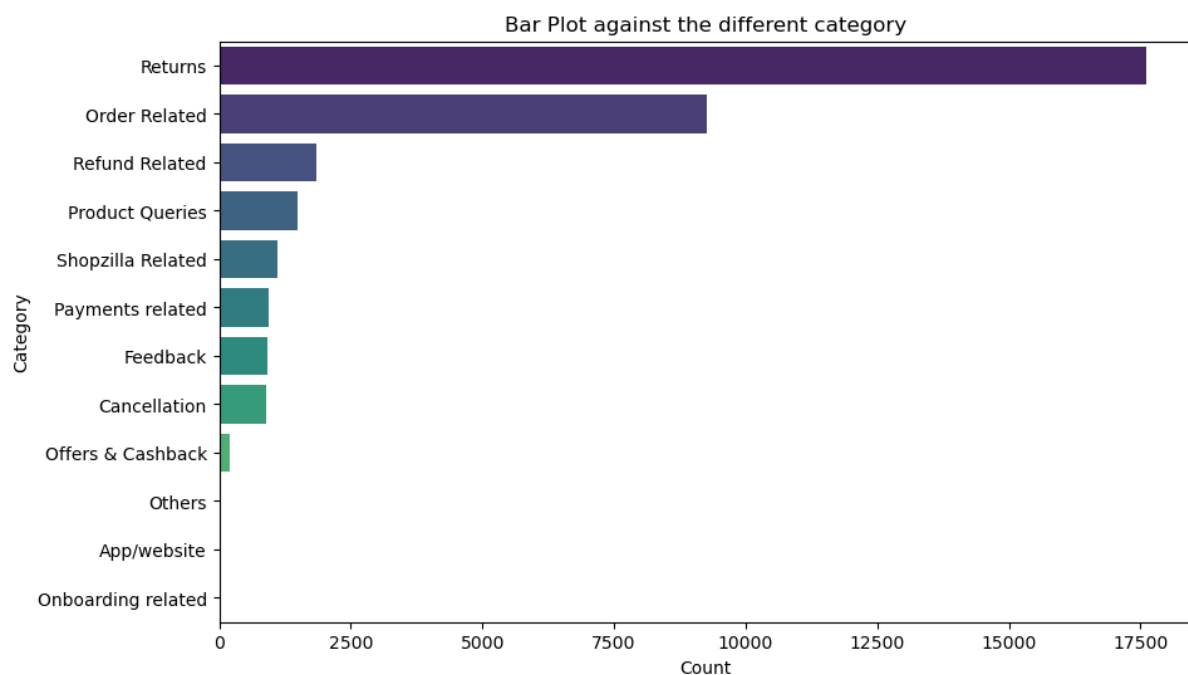


Figure 5: Bar Chart Showing Category Distribution

Figure 5 shows the types of customer queries and how frequently they occur. The most common issue is Returns, followed closely by Order Related queries. There are also a fair number of questions about Refunds and Product Queries. On the other hand, topics like Offers & Cashback, App/website, and Onboarding seem to generate very few inquiries, indicating they may not be major concerns for most customers.

3.4.3 Bivariate Visualization:

3.4.3.2 Distribution on Category and Sub-Category:

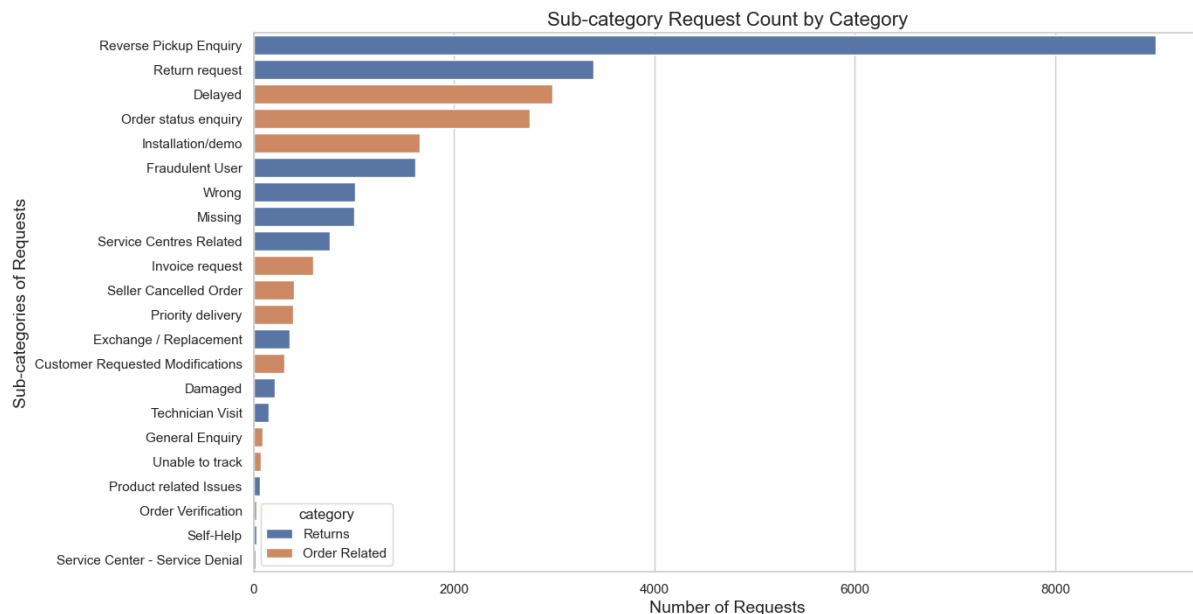


Figure 6: Bar plot against Category and Sub-Category

Figure 6 gives an overview of the customer service requests that can be summarized into two broad categories: Returns and Order Related. Evidently, Returns are a major concern, with Reverse Pickup Enquiries and Return Requests leading the list. These numbers are high, showing that many customers need help returning products or have problems scheduling pickups. Other frequent issues in the category of Order Related include Delayed Orders and Order Status Enquiries, which point to a problem in delivery timelines and tracking updates. Installation/Demo Services and Fraudulent User reporting are other requests that show the need for better support in service setup and security. Meanwhile, low-frequency issues include Technician Visits, Damaged Products, and Self-Help inquiries, which indicate that some customers experience specific and more specialized problems. Overall, the data indicates that most customer concerns revolve around returns and delivery-related issues, which present opportunities for businesses to streamline processes and improve communication to enhance customer satisfaction.

3.4.3.3 Distribution on top Agents having high CSAT Score:

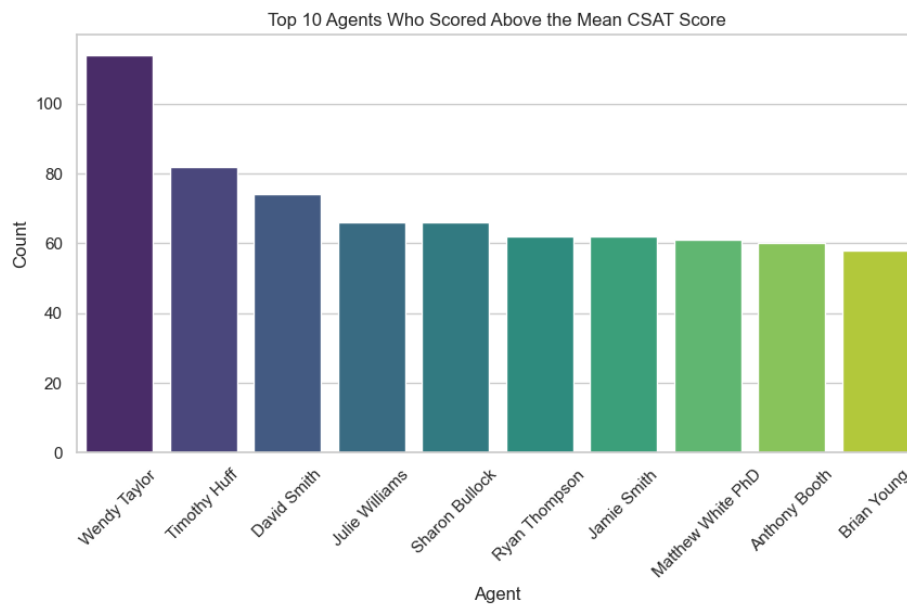


Figure 7: Top Agents of the dataset

Figure 7 depicts the top 10 agents who scored above average in CSAT, with their performance repetition against the number of instances surpassing the average satisfaction. Wendy Taylor is leading with more than 110 instances, closely followed by Timothy Huff and David Smith, both also performed very well. Other agents, like Julie Williams, Sharon Bullock, and Brian Young, had somewhat lower counts but were still in the top 10 performers. This chart clearly presents the visual comparisons of agent's performances, underlining much more strongly those that consistently deliver outstanding service and may be used for performance reviews, rewards, or further training.

3.5 Data Preprocessing:

It is necessary to perform data cleaning to evaluate the machine learning models. We have followed the following data-cleaning process to have a good dataset for performing a machine learning model.

- 1) Handling Missing Values
- 2) Handling Outliers
- 3) Feature Engineering
- 4) Correlation Matrix
- 5) Encoding
- 6) Scaling
- 7) Data Splitting

3.5.1 Handling Missing values:

Handling missing values is very important otherwise it will introduce bias to the result. In our dataset there are few columns which have more missing values.

Number of Missing Values:		Percentage of Missing Values:	
Unique id	0	Unique id	0.00
channel_name	0	channel_name	0.00
category	0	category	0.00
Sub-category	0	Sub-category	0.00
Customer Remarks	22802	Customer Remarks	66.36
Order_id	7406	Order_id	21.55
order_date_time	27524	order_date_time	80.10
Issue_reported at	0	Issue_reported at	0.00
issue_responded	0	issue_responded	0.00
Survey_response_Date	0	Survey_response_Date	0.00
Customer_City	27578	Customer_City	80.25
Product_category	27529	Product_category	80.11
Item_price	27526	Item_price	80.10
connected_handling_time	34278	connected_handling_time	99.75
Agent_name	0	Agent_name	0.00
Supervisor	0	Supervisor	0.00
Manager	0	Manager	0.00
Tenure Bucket	0	Tenure Bucket	0.00
Agent Shift	0	Agent Shift	0.00
CSAT Score	0	CSAT Score	0.00
dtype: int64		dtype: float64	

Figure 8: Details of Missing Values

Rationale for removing missing Values:

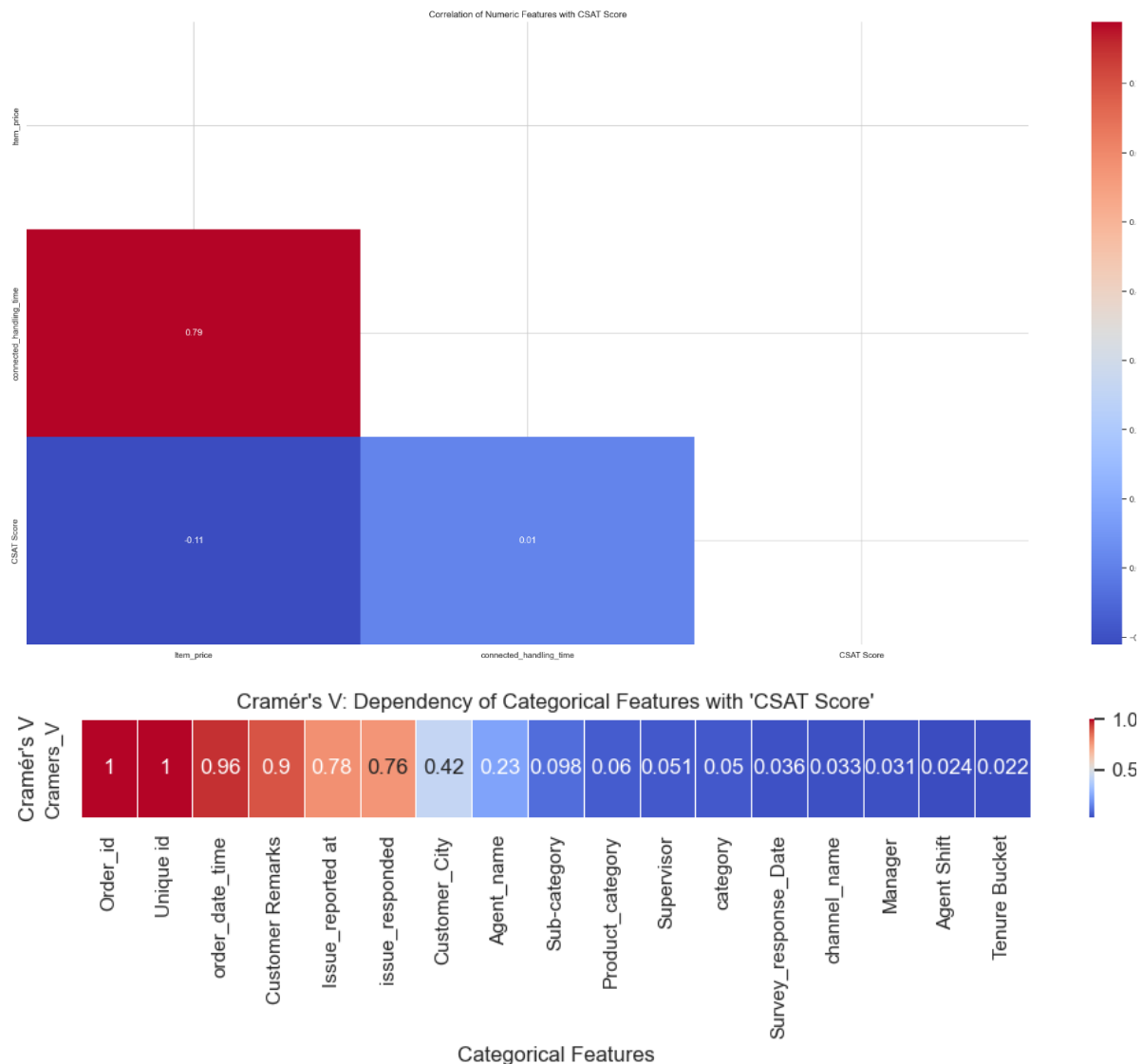


Figure 9: Correlation and Cramer's Matrix to check relation with CSAT Score

From the Figure 8 and 9, We could see the dataset has over 75% missing values in Order Date Time, Customer City, Product Category, Item Price, and Connected Handling Time. Imputing these with mean or mode would introduce noise. A correlation matrix showed Connected Handling Time and Item Price had weak correlations with CSAT Score and high missing values, leading to their removal. For categorical variables, Cramer's Matrix revealed Customer Remarks as highly correlated to CSAT Score. Missing values in this column were imputed as "No Remarks." Columns like Unique Id and Order Id (correlation = 1) and Product Category (high missing values, low significance) were dropped. These steps reduced noise and improved dataset quality.

While columns like Order Date Time and Customer City showed significant correlations with the target variable (CSAT Score), imputing their missing values through predictive modelling or domain-specific imputation was not considered for the following reasons:

1. **Extensive Missing Data:** Over 80% of the values in these columns were missing. Relying on limited available data for imputation would have significantly reduced the reliability of the results.
2. **Risk of Adding Noise or Bias:** Using predictive modeling or domain-specific rules for imputation could have introduced artificial patterns or biases into the dataset, potentially compromising its integrity and affecting the performance of machine learning models in subsequent analysis.
3. **Practical Constraints:** Imputing such a large volume of missing data would have required substantial time and computational resources. Since these features were expected to add only minimal value, the effort required to address them was considered unnecessary.

Given these challenges, the decision was made to drop these columns. This approach helped maintain data quality, reduce noise, and simplify the dataset for more efficient and accurate analysis.

Figure 10 shows the final dataset after removing the least significant columns and handling missing values.

Number of Missing Values:		Percentage of Missing Values:	
channel_name	0	channel_name	0.0
category	0	category	0.0
Sub-category	0	Sub-category	0.0
Customer Remarks	0	Customer Remarks	0.0
Issue_reported at	0	Issue_reported at	0.0
issue_responded	0	issue_responded	0.0
Survey_response_Date	0	Survey_response_Date	0.0
Agent_name	0	Agent_name	0.0
Supervisor	0	Supervisor	0.0
Manager	0	Manager	0.0
Tenure Bucket	0	Tenure Bucket	0.0
Agent Shift	0	Agent Shift	0.0
CSAT Score	0	CSAT Score	0.0
dtype: int64		dtype: float64	

Figure 10: Final Columns after handling Missing Values

3.5.2 Outliers Detection:

Outliers means the abnormal values that differ significantly from the dataset and also it is performed for numeric variables. Since there are no numeric columns in the final dataset Outliers handling is not performed.

3.5.3 Feature Engineering:

We have created a new column called response time based on the Issue reported at date and issue responded time. This will help in knowing how much time they have taken to resolve the issue. Since we can't use date columns directly converted those columns to issue reported day of week and issue responded hour. This will be helpful to know what are the day's issues are reported mostly. Created an extra column called 'is_urgent' this will help to know if the issue is created during non-business hours and whether to check whether the issue is urgent and needs immediate resolution. The code is in [Appendix 2](#).

CORRELATION MATRIX:

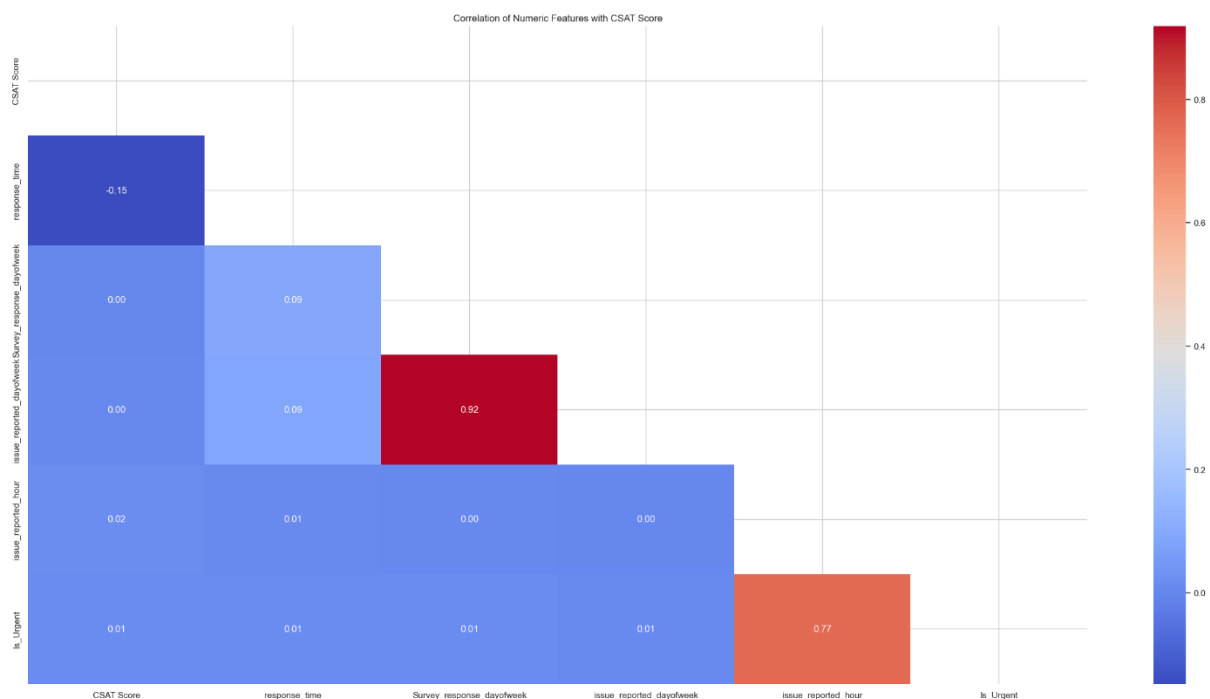


Figure 11: Correlation of the new columns

Figure 11 shows the correlation of the new features against the target variable. Among the new features, `response_time` shows the strongest correlation with CSAT Score (-0.15), indicating that longer response times are associated with lower customer satisfaction, as expected. In contrast, other features, such as `Survey_response_dayofweek` (0.00) and `issue_reported_hour` (0.02), display negligible correlations, suggesting that the timing of survey responses or issue reporting has little to no influence on satisfaction levels.

From the Figure 9, the column 'Customer Remarks' shows the highest dependency on the 'CSAT Score', whereas columns such as Agent Shift, Tenure Bucket, Manager and Channel Name have very minimum dependency. Hence, removed those columns.

3.5.4 Encoding:

The next important data pre-processing step is data encoding. Since machine learning algorithms do not understand the categorical variables, it is mandatory to convert the categorical variables into numerical variables. The code is in [Appendix 3](#).

3.5.4.1 One hot Encoding:

One-hot encoding is a data preprocessing technique that converts categorical data into a numerical format by creating a new binary column for each unique category within the original feature, where only one column will be set to "1" for the corresponding category while all other columns for that row will be "0". This will be more suitable for columns having low cardinality columns which means having low unique values. In our dataset I have implemented one hot encoding for the column "Category".

3.5.4.2 Label Encoding:

Label encoding is a data preprocessing technique in machine learning where each unique category within a categorical variable is assigned a unique numerical value. This will be helpful for the columns having more unique values (high cardinality columns) which will help reduce more columns than one hot encoding, this technique will be more efficient than one hot encoding for the high cardinality columns. In our dataset Label Encoding is implemented for the columns Agent Name, Supervisor, and Sub-category.

3.6 Scaling:

Scaling is the next data pre-processing technique used, which will be used only for numerical values. We have used a standard scaler for the project. A standard scaler is a technique that standardizes the

numerical values in a dataset. It will make sure to have an average value of 0 and a standard deviation of 1 as per the below formula. The code is in [Appendix 4](#).

$$Z = (x - u)/s$$

Where x is the original value, u is the mean and s is the standard deviation.

3.7 Data Splitting:

We have followed stratified shuffle split for splitting the data to training and testing. It ensures that the class distribution in the target variable is the same in both training and testing sets. So, the model for both training and testing sets will be appropriate. I have used 80 and 20 ratio which means 80% training data and 20% testing data. It is the most followed approach. The code is in [Appendix 5](#).

3.8 Sentiment Analysis:

Sentiment Analysis is applied for the customer remarks column which will be helpful to understand how many positive, negative, and neutral comments are there in the dataset. It will be very helpful for the organization to understand the feedback or opinions of the customers. Also, it helps in improving the business and key areas to focus. The code is in [Appendix 6](#).

3.8.1 Pre-Processing:

We have preprocessed the customer remarks column to get the data clean without any noise. The first thing we did was removing special characters in the column and then removed stop words like 'is', 'the', 'and' from the column and implemented lemmatizer for the words which will be very helpful to group different forms of same word.

Importance of Pre-processing:

- It will improve accuracy by removing irrelevant data and focus on meaningful words.
- It reduces noise by removing punctuation and stop words.
- It standardizes data by reducing variations caused by case differences or word forms.

3.8.2 Implementation of Sentiment Analyzer:

We have used VADER ("Valence Aware Dictionary and Sentiment Reasoner") Sentiment Analyzer for processing this will help in analyzing the sentiment of the text by assigning positive, negative and neutral sentiment scores for the remarks.

VADER SENTIMENT SCORE:

Vader sentiment score is a numerical value assigned to a piece of text using the VADER algorithm, indicating the overall sentiment of the text as positive, negative, or neutral. It is very helpful in categorizing the sentiments. It ranges from -1 to +1.

Polarity:

Polarity refers to the level of positivity or negativity expressed in the text. It is represented on a scale from -1 (very negative) to +1 (very positive). A sentence with a high positive polarity would be considered very positive, while a sentence with a low negative polarity would be considered somewhat negative.

Subjectivity:

Subjectivity indicates how much opinion or judgment is present in the text, as opposed to factual information. It is measured on a scale from 0 (completely objective) to 1 (completely subjective). A sentence stating a fact would have low subjectivity, while a sentence expressing an opinion would have high subjectivity.

3.8.3.1 Visualization of the Sentiments:

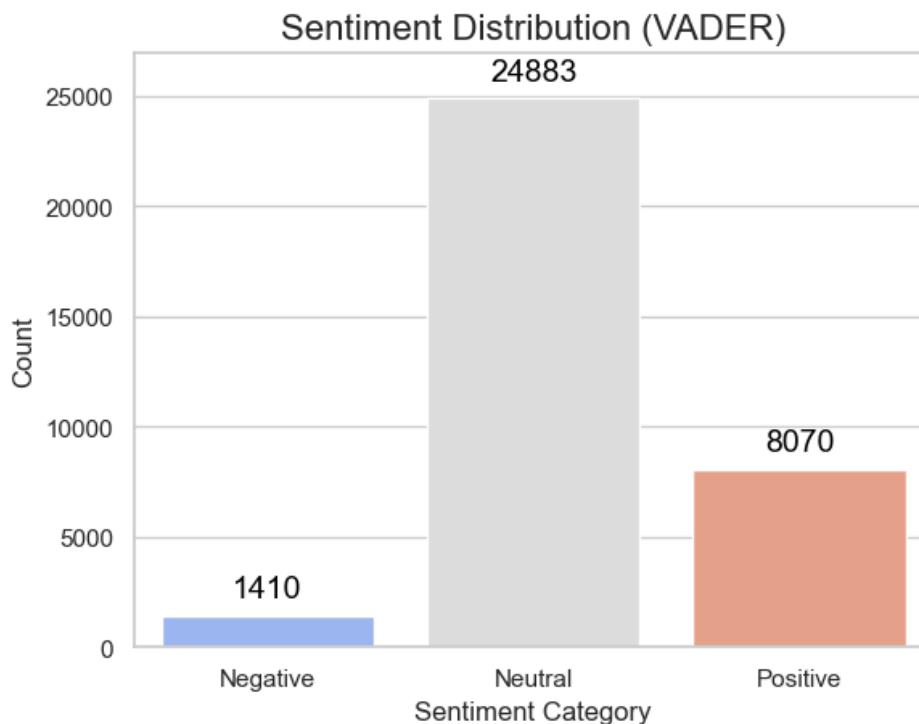


Figure 12: Distribution of Sentiments

Figure 12 is the output of the technique classifying text data into Neutral, Positive, and Negative sentiments. A count of 24,883 majority of data has fallen into the Neutral category, representing the fact that most text entries are factual, balanced, or not highly emotionally charged. The Positive category contains 8,007, which is quite high and shows a considerable portion of data with favorable opinions, satisfaction, or optimistic feedback. In contrast, the Negative category contains only 1,410 entries, hence forming a small fraction of the dataset, showing negligible dissatisfaction or criticism. This is because, for the missing values in the column, we have imputed them as 'No Remarks'. Thus, we can say that most of the customers are satisfied with the performance.

3.8.3.2 Distribution of Polarity and Subjectivity:

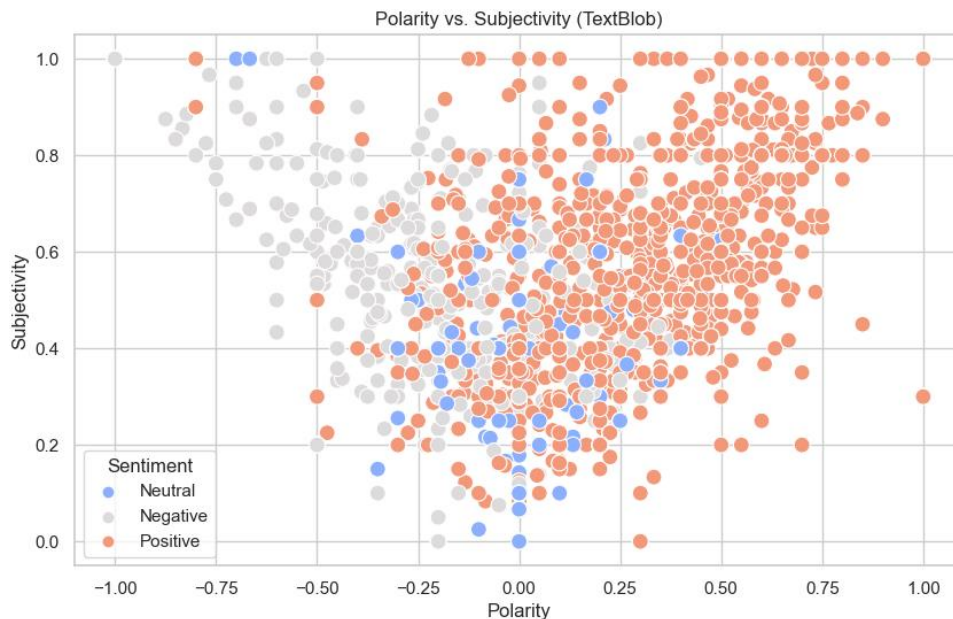


Figure 13: Distribution of Polarity and Subjectivity

Figure 13 visualizes the relationship between polarity and subjectivity based on TextBlob sentiment analysis. The x-axis represents polarity, measuring sentiment from -1 (negative) to +1 (positive), while the y-axis represents subjectivity, ranging from 0 (objective) to 1 (subjective), indicating whether the text is fact-based or opinion-based. Each point is color-coded to represent sentiment—Positive (orange), Neutral (blue), and Negative (grey). The plot shows that most points are concentrated in the positive polarity region with high subjectivity, indicating that much of the data contains positive and opinion-based content. The neutral sentiments, shown in blue, are scattered around the centre, reflecting balanced or factual statements. The negative sentiments, shown in grey, cluster toward the left side of the polarity axis, with varying subjectivity, suggesting fewer negative opinions.

3.9 Machine Learning Models:

We utilized three machine learning models for analysis: Logistic Regression for the traditional model, Random Forest for the ensemble model, and XGBoost for the boosting algorithm. These models were selected based on their frequent use in the literature and their strong performance on imbalanced data.

3.9.1 Logistic Regression:

Logistic Regression is a widely used statistical method for binary classification tasks. It is particularly effective in analyzing the relationships between variables to predict categorical outcomes, making it a

preferred choice in machine learning due to its simplicity, interpretability, and computational efficiency (Panduro-Ramirez, 2024). Logistic Regression is commonly applied in customer service optimization, including churn prediction, sentiment analysis, and other classification problems. For instance, in sentiment analysis tasks, Logistic Regression has demonstrated high accuracy (80–90%) in categorizing sentiments into positive, negative, and neutral, highlighting its reliability and robustness (Panduro-Ramirez, 2024).

Despite its strengths, Logistic Regression has limitations when dealing with non-linear relationships and complex datasets (Kasemrat and Kraiwanit, 2024). Historically, predictive analytics in the e-commerce sector has relied heavily on traditional techniques such as Logistic Regression and Linear Models (Zaghloul et al., 2024). However, these models struggle to handle the complexity and volume of data generated by modern e-commerce systems, underscoring the need for more advanced machine learning approaches to address these challenges effectively.

3.9.2 Random Forest:

Random Forest is a widely used ensemble learning model that combines predictions from multiple decision trees, each trained on random subsets of data, making it more robust and accurate compared to a single decision tree. Its scalability and versatility make it particularly effective in handling large datasets, imbalanced data, and complex prediction tasks. In e-commerce, Random Forest is extensively applied in predictive analytics and classification tasks, especially for customer service-related applications.

Sabbeh (2018) demonstrated its effectiveness by achieving 96% accuracy in predicting customer churn, while Nagaraj et al. (2023) validated it as the most effective algorithm for churn prediction using features such as customer tenure, order history, and satisfaction scores. These studies highlight its cost-effectiveness and adaptability for retention strategies. Similarly, Zaghloul et al. (2024) compared Random Forest with deep learning models and found that Random Forest achieved the highest accuracy for customer satisfaction prediction, though the study recommended feature engineering and resampling techniques to address data imbalances and enhance performance. Furthermore, Tai et al. (2023) reported its effectiveness in predicting resolution times for customer service tasks, with an accuracy of 70.21% when proper data preprocessing and regularization were applied to mitigate overfitting.

These studies collectively suggest that Random Forest's performance can be further improved through hyperparameter tuning, such as adjusting the number of trees, maximum depth, and split criteria, as

well as through comprehensive data preprocessing steps like feature scaling, handling missing values, and balancing datasets. Its reliability, scalability, and adaptability make Random Forest a competitive choice for modern machine-learning applications in e-commerce.

3.9.3 XGBoost:

eXtreme Gradient Boosting (XGBoost) is a highly efficient and scalable machine learning algorithm based on gradient boosting, widely used for classification and regression tasks. It is particularly effective in handling large datasets, high-dimensional data, and imbalanced datasets, making it a popular choice for complex predictive analytics.

Tai et al. (2023) found XGBoost outperformed Decision Trees and Random Forest in predicting resolution times for customer service tasks, achieving lower error rates and better performance. The study emphasized the role of data preprocessing, feature engineering, and regularization in enhancing performance and reducing overfitting. Similarly, Yu and Weng (2022) tested XGBoost on the IBM Telco Customer Churn dataset, where it excelled in churn prediction and outperformed other classifiers, highlighting the need for advanced tuning methods like grid search and Bayesian optimization. Manzoor et al. (2024) reviewed on customer churn prediction and emphasized the effectiveness of XGBoost and LightGBM for boosting algorithms. These models consistently outperformed traditional techniques, offering high accuracy, interpretability, and scalability.

The literature collectively suggests that XGBoost's performance can be improved by fine-tuning its hyperparameters using methods like Grid Search, Randomized Search, and Bayesian Optimization. Additionally, effective data preprocessing, such as scaling features, addressing missing values, and balancing datasets, plays a key role in boosting its accuracy, scalability, and overall performance. These enhancements make XGBoost a powerful tool for predictive analytics and customer service applications. However, it has some limitations, such as being computationally expensive for extremely large datasets, requiring careful hyperparameter tuning to avoid overfitting, and facing challenges with interpretability in complex models.

3.10 Justification for Model Selection:

In this study, we have chosen Logistic Regression, Random Forest, and XGBoost as they align well with the project goals and the nature of the dataset. While other algorithms like Neural Networks and Support Vector Machines (SVMs) were considered, they were excluded for practical reasons.

Neural Networks are powerful for handling complex and high-dimensional data, but they require much larger datasets to perform well and can easily overfit smaller datasets like ours. They also demand

significant computational resources and are less interpretable, making it harder to understand how individual features influence predictions, which was an important focus for this project. Similarly, SVMs are effective for smaller datasets or binary classification problems but struggle to scale with larger datasets. Training an SVM on our data would have been time-consuming and also computationally intensive. By contrast, the selected models offer a strong balance of accuracy, efficiency, and interpretability. They are designed to handle structured data effectively, perform well with imbalanced datasets, and provide clear insights into which features are most important, aligning perfectly with the objectives of our study.

3.11 Evaluation Metrics:

The performance of the machine learning models are evaluated through the below evaluation metrics.

3.11.1 Accuracy:

Accuracy is a metric that measures how often a machine learning model correctly predicts the outcome. You can calculate accuracy by dividing the number of correct predictions by the total number of predictions. In other words, it is the proportion of all classifications that were correct, whether positive or negative. It is mathematically defined as:

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- **TP (True Positive):** Correctly predicted positive cases.
- **TN (True Negative):** Correctly predicted negative cases.
- **FP (False Positive):** Incorrectly predicted positive cases.
- **FN (False Negative):** Incorrectly predicted negative cases.

3.11.2 F1 Score:

The F1 Score is a performance metric used in machine learning that combines precision and recall into a single value. It is particularly useful when dealing with imbalanced datasets, as it considers both false positives and false negatives. The F1 Score is the harmonic mean of precision and recall and is mathematically defined as:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where:

- **Precision (P)** = $\frac{TP}{TP+FP}$
- **Recall (R)** = $\frac{TP}{TP+FN}$

Key Terms:

- **TP (True Positive):** Correctly predicted positive cases.
- **FP (False Positive):** Incorrectly predicted positive cases.
- **FN (False Negative):** Incorrectly predicted negative cases.

It ranges from 0 to 1, where 1 represents perfect precision and recall, and 0 indicates poor performance.

3.11.3 Precision:

Precision is a performance metric in machine learning that measures the accuracy of positive predictions made by a model. It evaluates how many of the predicted positive cases are actually positive, making it particularly useful in scenarios where false positives (FP) are costly. It is calculated using the formula.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Where:

- **TP (True Positive):** Correctly predicted positive cases.
- **FP (False Positive):** Incorrectly predicted positive cases.

High Precision means most predicted positives are correct, minimizing false positives, while Low Precision indicates many predicted positives are actually false positives.

3.11.4 Recall:

Recall (also known as Sensitivity or True Positive Rate) is a performance metric in machine learning that measures the ability of a model to correctly identify positive cases out of all actual positive cases. It focuses on minimizing false negatives (FN).

$$Recall = \frac{TP}{TP + FN}$$

Where:

- **TP (True Positive):** Correctly predicted positive cases.
- **FN (False Negative):** Actual positive cases that were incorrectly predicted as negative.

High Recall means the model detects most actual positives, minimizing false negatives, while Low Recall indicates it misses many positive cases, leading to more false negatives.

3.11.5 Confusion Matrix:

A Confusion Matrix is a performance evaluation tool used in classification problems to visualize the accuracy of a machine learning model by comparing its predicted values with the actual values. It provides a detailed breakdown of correct and incorrect predictions across different classes.

CHAPTER FOUR: MODEL ANALYSIS AND RESULTS

4. MODEL ANALYSIS AND RESULTS:

This section presents the results of the machine learning models used to predict customer satisfaction scores and provides a comparison of their performance. The models were trained using the training dataset, with hyperparameter tuning and cross-validation performed to identify the best parameters and estimators for each model.

After obtaining the optimal parameters, the models were evaluated on both training and testing datasets to assess their performance. To address the issue of data imbalance, SMOTE (Synthetic Minority Oversampling Technique) was applied. Additionally, confusion matrices were used to visualize and analyze the models' classification performance.

To further evaluate and compare the models, AUC-ROC and Precision-Recall curves were plotted. These visualizations provide insights into the models' ability to distinguish between classes, helping the business better understand and interpret the models' effectiveness. This analysis ultimately aids in selecting the most reliable model for improving customer satisfaction predictions.

4.1 Baseline Model:

We have performed the baseline method because it provides a benchmark to evaluate the other model's performance. It also ensures the model learns meaningful patterns and also helps in highlighting the imbalanced dataset. The models performing better than the baseline indicate effective learning and better prediction of outcomes. The code for the baseline model is in [Appendix 7](#).

Model	Evaluation Metrics	Training Data	Testing Data
Baseline	Accuracy	0.69	0.69
	F1 score	0.57	0.57

Table 4: Result of Baseline Model

Table 4 provides a performance summary of the baseline model on both the training and testing datasets, using accuracy and F1 score as evaluation metrics. The accuracy of the baseline model is 0.69 for both datasets, indicating that it predicts the most frequent class correctly 69% of the time. The F1 score, which balances precision and recall, is 0.57 for both training and testing, highlighting the baseline's limitations in effectively handling class diversity. This baseline performance serves as a

benchmark to evaluate whether more advanced models provide meaningful improvements in predicting customer satisfaction (CSAT) scores.

4.2 Logistic Regression:

Model	Evaluation Metrics	Before hyperparameter tuning	After Hyperparameter Tuning & Cross Validation	
			Training Data	Testing Data
Logistic Regression	Accuracy	0.70	0.71	0.69
	F1 Score	0.59	0.60	0.61

Table 5: Result of Logistic Regression

Table 5 presents the result of Logistic Regression before and after Tuning. The model's accuracy was initially 0.70 before hyperparameter tuning. After tuning and cross-validation, the accuracy improved slightly to 0.71 on the training data and slightly decreased to 0.69 on the testing data. Similarly, the F1 score showed a minor improvement, increasing from 0.59 before tuning to 0.60 on the training data and 0.61 on the testing data. This indicates that the tuning process helped the model achieve a better balance between precision and recall, enhancing its ability to handle class diversity. The improved F1 score highlights that Logistic Regression became more effective than the baseline model in predicting CSAT scores accurately and managing imbalanced class distributions.

4.2.1 Hyperparameter Tuning and Model Optimization:

We employed GridSearch for hyperparameter tuning, utilizing a parameter grid to test various values for regularization strength (C) and solvers (liblinear and saga). Stratified k-fold cross-validation was conducted to evaluate the model's performance, using the F1 macro score as the evaluation metric. The best parameters identified were C=100 and solver='liblinear', which were used to evaluate the model. The cross-validation results yielded a mean F1 score of 0.21, reflecting the model's performance. Finally, the best model was saved for further evaluation on the dataset. The detailed information is in [Appendix 8](#).

Result of Confusion Matrix After Tuning:

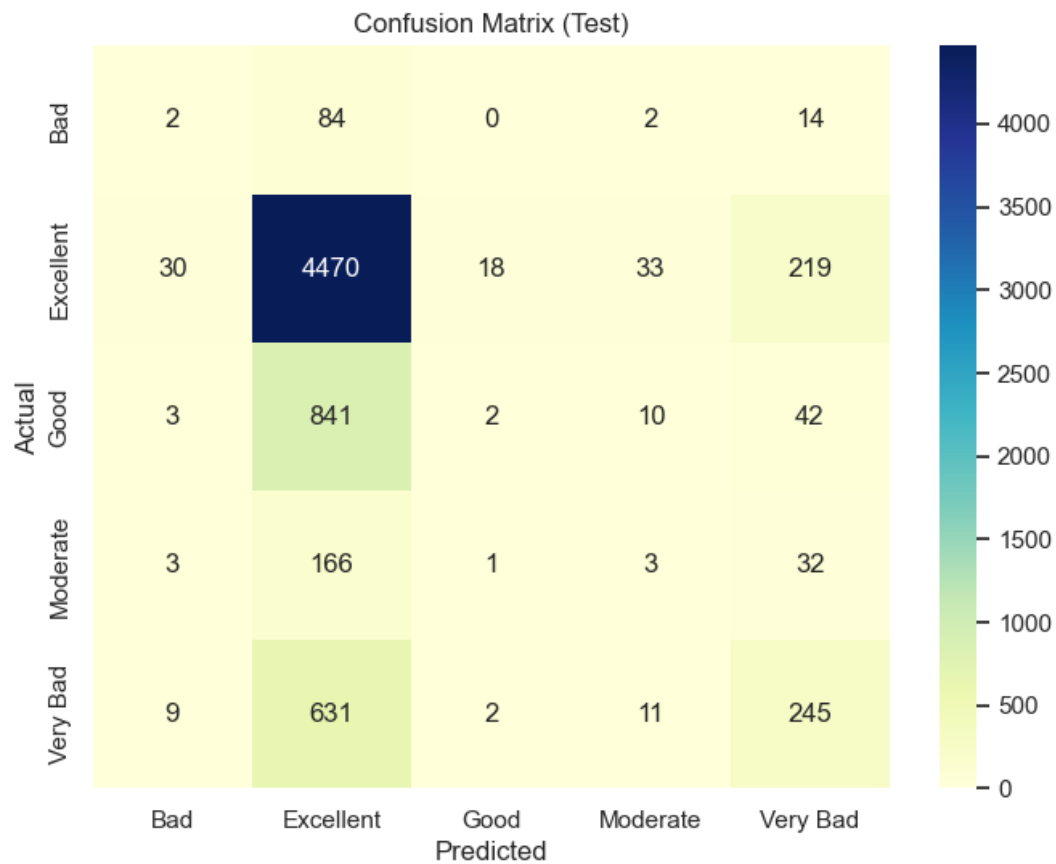


Figure 14: Result of Confusion Matrix on Testing Data

Figure 14 shows how well the model performed on the test data in distinguishing between different categories. The model does an excellent job of correctly identifying the majority class, Excellent, with 4,470 instances accurately classified. However, it struggles with the smaller classes like Bad, Good, Moderate, and Very Bad. For example, only 2 instances of Bad and 245 instances of Very Bad are correctly classified. A notable issue is the high number of misclassifications, especially with minority classes being predicted as Excellent. For instance, 631 instances of Very Bad and 166 instances of Moderate are incorrectly categorized as Excellent.

4.2.2 Strategies to Improve Minority Class Performance:

MODEL	Evaluation Metrics	Before SMOTE	After SMOTE
Logistic Regression	Precision	0.57	0.58
	Recall	0.69	0.47
	F1 Score	0.61	0.51

Table 6: Result of Logistic Regression before and after SMOTE

We have experimented with several techniques like SMOTE, ADASYN, and class weight distribution to increase the performance of the minority class. After trying several methods above and even performing Feature selection to remove irrelevant features and selected the top 10 features. But, we faced so many challenges in increasing the Accuracy and F1 score. In some combinations, the result predictions were not so good. After doing all these steps, the combination of SMOTE and changing the class weight distribution to balanced provided the better result with an accuracy of 0.47 and an F1 score of 0.51. Table 6 illustrates the performance of the Logistic Regression model before and after applying SMOTE, showing that while precision increased to 0.58, recall decreased significantly from 0.69 to 0.47, resulting in a drop in the F1 score from 0.61 to 0.51. The result of Confusion Matrix, AUC-ROC curve, and precision-recall curve are used to visualize the distribution of outcomes.

Result of Confusion Matrix After SMOTE:

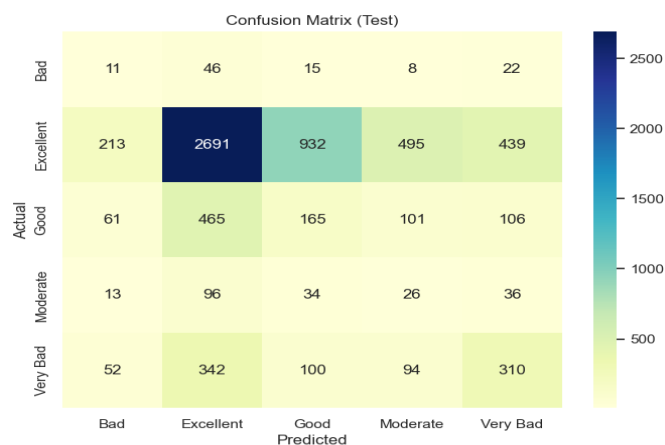


Figure 15: Confusion Matrix on Testing Data after SMOTE

The model without the SMOTE technique is highly biased toward the majority class ("Excellent") and gives very poor representation to minority classes like "Bad," "Moderate," and "Very Bad." There is

heavy misclassification in the prediction of minority classes. However, the model with SMOTE gives better representation for minority classes due to the balanced data. However, this is achieved at the cost of higher misclassification across similar classes, which reduced the precision and recall for some. The improvement can be seen in Figure 15.

AUC-ROC Curve:

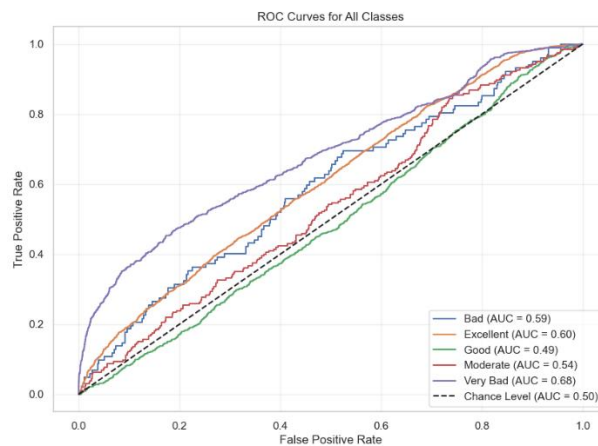


Figure 16: AUC-ROC Curve of Each Class

Figure 16 visualizes the classification performance of the model across all classes. The AUC (Area Under the Curve) values indicate how well the model separates the positive class from the negative class for each category. The model performs best for the Very Bad category with an AUC of 0.68, followed by Excellent (AUC = 0.60) and Bad (AUC = 0.59). The Moderate category has an AUC of 0.54, while the Good category performs poorly with an AUC of 0.49, nearly equivalent to random chance (AUC = 0.50).

Precision-Recall Curve:

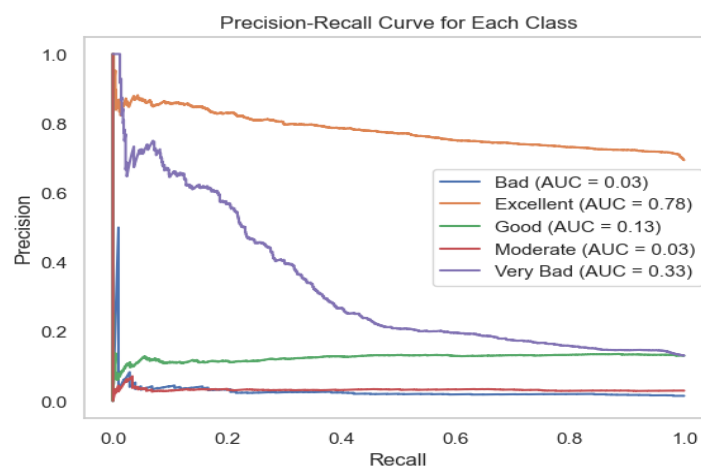


Figure 17: Precision-Recall Curve of each class

Figure 17 shows the model's performance across all classes. The class Excellent has the highest AUC (0.78), indicating strong precision and recall. However, for the minority classes like Bad (AUC = 0.03) and Moderate (AUC = 0.03), the model struggles, with very low precision and recall. The Very Bad class performs slightly better (AUC = 0.33), while Good shows moderate performance (AUC = 0.13). This indicates that the model is biased toward the majority class (Excellent), highlighting challenges in handling class imbalance and distinguishing minority classes effectively.

In summary, SMOTE reduces bias toward the majority class and improves minority class performance, but it introduces slight confusion between classes and also reduced the accuracy and F1 score. To improve the performance of the minority class and enhance the model's overall performance, the ensemble method Random Forest was utilized.

4.3 Random Forest:

Model	Evaluation Metrics	Before hyperparameter tuning	After Hyperparameter Tuning & Cross Validation	
			Training Data	Testing Data
Random Forest	Accuracy	0.72	0.73	0.72
	F1 Score	0.62	0.64	0.62

Table 7: Result of Random Forest

Table 7 shows the performance of the Random Forest model before and after hyperparameter tuning. The model's accuracy improved slightly from 0.72 to 0.73 on the training data, while it remained consistent at 0.72 on the testing data. The F1 score increased slightly from 0.62 to 0.64 on the training data but stayed the same at 0.62 on the testing data, indicating that hyperparameter tuning did not significantly enhance the model's ability to generalize to unseen data. The Random Forest model outperformed both the Baseline and Logistic Regression models in terms of overall performance.

4.3.1 Hyperparameter Tuning and Model Optimization:

```
# Define Parameter Grid for Hyperparameter Tuning
param_grid_rf = {
    'n_estimators': [50, 100], # Fewer trees to avoid overfitting
    'max_depth': [10, 20, None], # Restrict tree depth
    'min_samples_split': [10, 20], # More samples to split nodes
    'min_samples_leaf': [5, 10], # More samples per leaf
    'max_features': ['sqrt', 'log2'],
    'bootstrap': [True]
}
```

Figure 18: Details of Parameter Grid

```
Best parameters for Random Forest:
{'n_estimators': 50,
 'min_samples_split': 10,
 'min_samples_leaf': 5, 'max_features':
 'log2', 'max_depth': None, 'bootstrap':
 True}
```

Figure 19: Best Parameters of Random Forest

RandomizedSearchCV was chosen over Grid Search for its faster runtime and efficiency, making it a practical choice for hyperparameter optimization. By randomly sampling the hyperparameter space, it identifies optimal configurations without exhaustively testing all combinations, reducing computational time while maintaining strong performance. Parameters tuned used are shown in Figure 18. A 5-fold stratified cross-validation strategy ensured balanced and reliable evaluation. The best model identified in Figure 19 is used to evaluate the data. The detailed information is in [Appendix 9](#).

Result of Confusion Matrix on Testing Data:

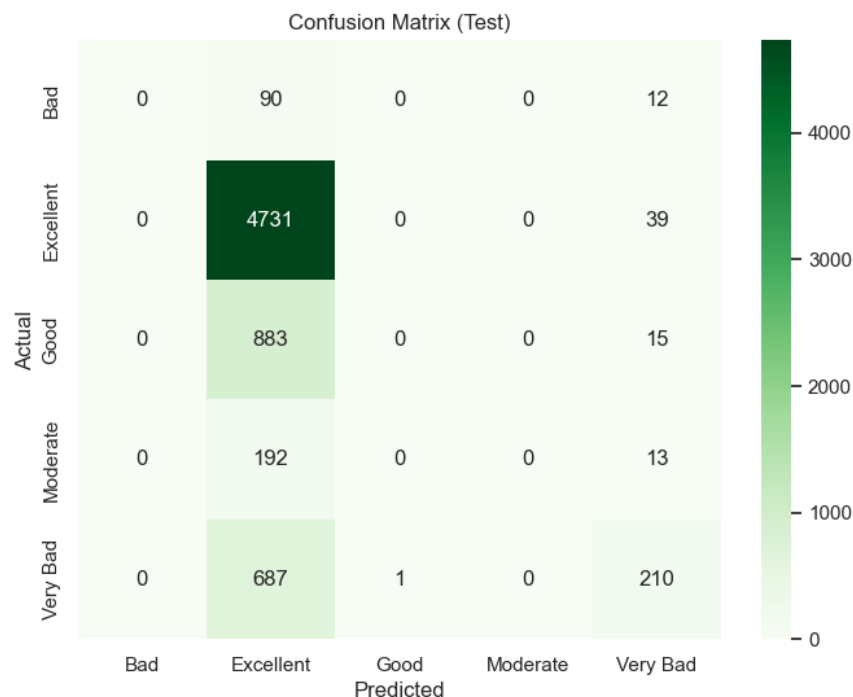


Figure 20: Result of Confusion Matrix on Testing Data

Figure 20 shows that the Random Forest model performs well for the "Excellent" class, with 4731 correct predictions, indicating strong accuracy for the majority class. However, the model struggles with minority classes like "Bad," "Moderate," and "Very Bad," which are often misclassified as "Excellent." For instance, 687 instances of "Very Bad" and 192 instances of "Moderate" were incorrectly predicted as "Excellent." While the "Good" class has 883 correct predictions, there are also misclassifications into "Excellent" and "Very Bad." This highlights the model's bias toward the majority class and suggests the need for techniques like data balancing or adjusting class weights to improve performance for minority classes.

4.3.2 Strategies to Improve Minority Class Performance:

MODEL	Evaluation Metrics	Before SMOTE		After SMOTE	
		Training Data	Testing Data	Training Data	Testing Data
Random Forest	Precision	0.73	0.59	0.91	0.59
	Recall	0.73	0.72	0.90	0.69
	F1 Score	0.64	0.62	0.91	0.62

Table 8: Result of Random Forest before and after SMOTE

In this approach, We focused on improving the performance of the minority class to overcome the issue of imbalance data. To address the class imbalance, I applied SMOTE to oversample the minority classes in the training dataset, ensuring the model had sufficient examples from all classes to learn effectively. I used label encoding to consistently convert categorical labels into a numeric format for both the training and test datasets. Additionally, I enabled class weighting distribution to balance to give appropriate importance to minority classes.

Table 8 compares the performance of the Random Forest model before and after applying SMOTE to address class imbalance. Before SMOTE, the model showed moderate performance, with an F1 score of 0.64 on training data and 0.62 on testing data. After applying SMOTE, the model's performance significantly improved on the training data, achieving an F1 score of 0.91, precision of 0.91, and recall of 0.90, reflecting better handling of the minority class. However, the performance on testing data remained the same, with the F1 score at 0.62, indicating overfitting and challenges in generalizing to unseen data despite the balanced training dataset.

Result of confusion matrix on Training data:

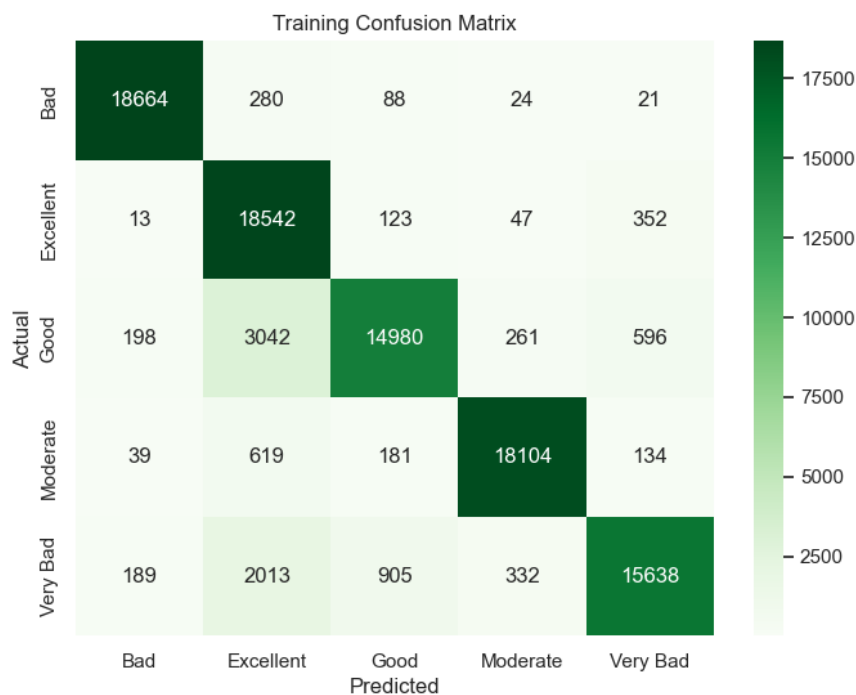


Figure 21: Result of Confusion matrix on Training Data after SMOTE

Figure 21 shows that the model performs exceptionally well on the training data, achieving high accuracy for all classes. The diagonal cells, which represent correct predictions, have significantly high values, indicating that the model has learned the training data effectively. However, slight misclassifications are observed, particularly for the "Good" and "Very Bad" classes, where some instances are misclassified as "Excellent" or other classes. This excellent performance is likely due to the balanced dataset created using SMOTE during training, which provided sufficient examples for all classes, enabling the model to handle the data well. However, this performance might not generalize to the test data due to overfitting or differences in class distributions.

Result of confusion matrix on Testing data:

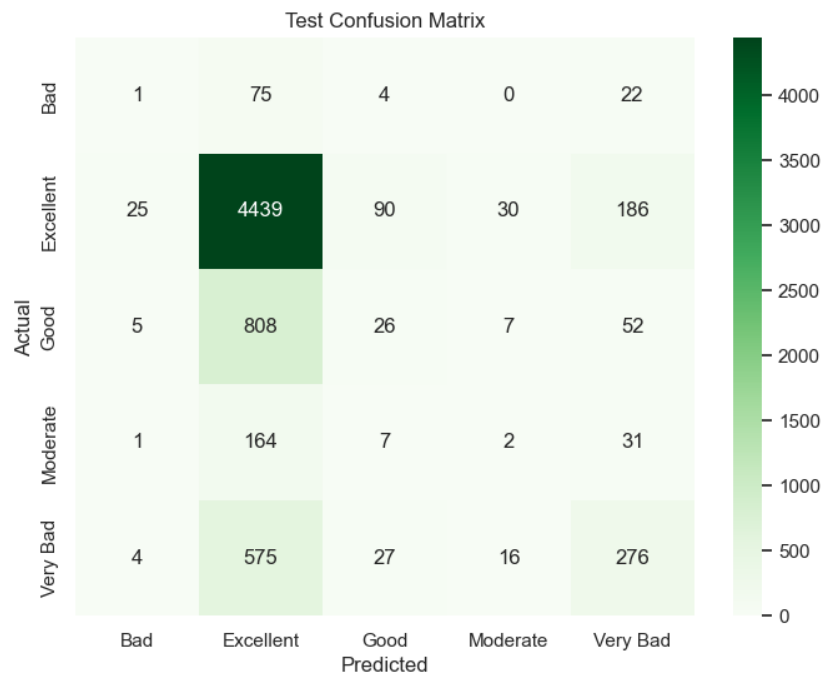


Figure 22: Result of confusion matrix on Testing data after SMOTE

Figure 22 highlights a performance drop compared to the training data. The model heavily predicts the "Excellent" class, as seen by the high value in the corresponding diagonal cell, but struggles with minority classes like "Bad" and "Very Bad," where predictions are sparse and mostly misclassified. For instance, many "Very Bad" samples are incorrectly classified as "Excellent," showing a bias toward the majority class. This indicates the model's inability to generalize well due to imbalanced test data. To increase the performance of testing data I have tried XGboost which will be more efficient for handling the imbalance data.

AUC-ROC Curve:

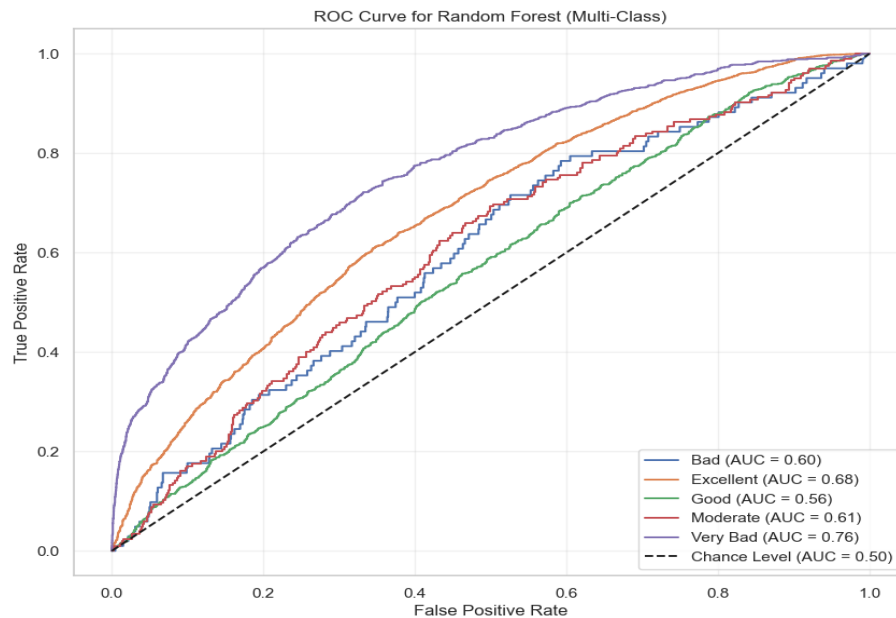


Figure 23: AUC-ROC Curve of each class

Figure 23 shows the performance for each class in a multi-class classification setting. The "Very Bad" class achieves the highest AUC (0.76), indicating the model performs relatively well for this class. The "Excellent" class follows with an AUC of 0.68, showing moderate separability. However, other classes, such as "Good" (AUC = 0.56) and "Bad" (AUC = 0.60), have lower AUC values, indicating poor discrimination between positive and negative samples for these classes.

Precision-Recall Curve:

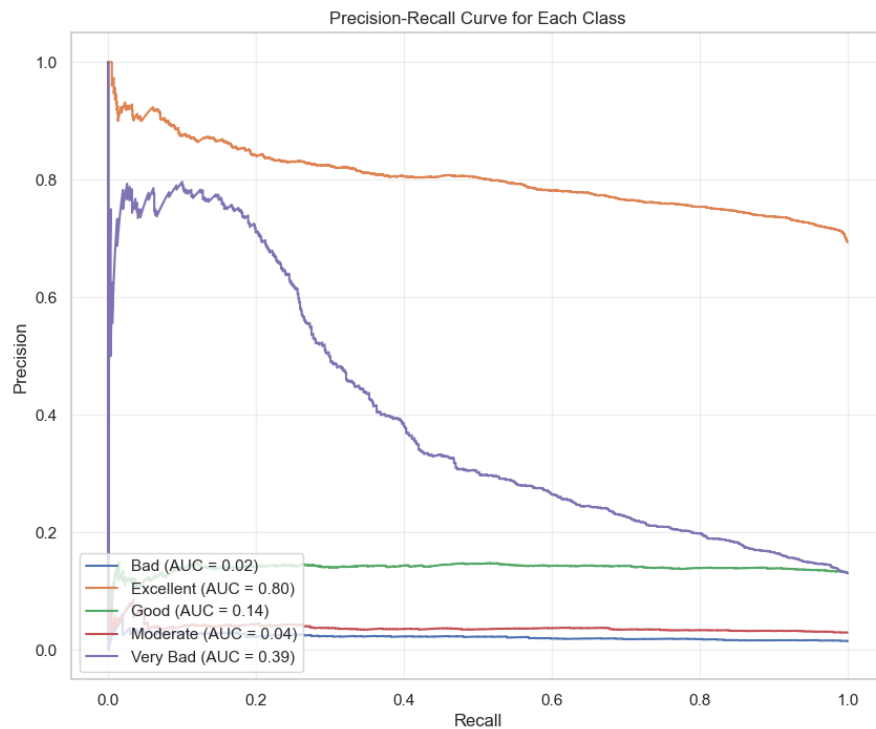


Figure 24: Precision-Recall Curve of each Class

Figure 24 shows the model's performance for each class. The class Excellent achieves the highest AUC (0.80), indicating strong performance with high precision and recall. In contrast, the minority classes such as Bad (AUC = 0.02), Moderate (AUC = 0.04), and Very Bad (AUC = 0.39) exhibit poor performance, with low precision and recall. The class Good (AUC = 0.14) performs slightly better than other minority classes but still shows significant limitations. This graph highlights the model's bias towards the majority class (Excellent) and its struggle to effectively predict minority classes, suggesting challenges related to class imbalance.

4.3.2 Feature Importance:

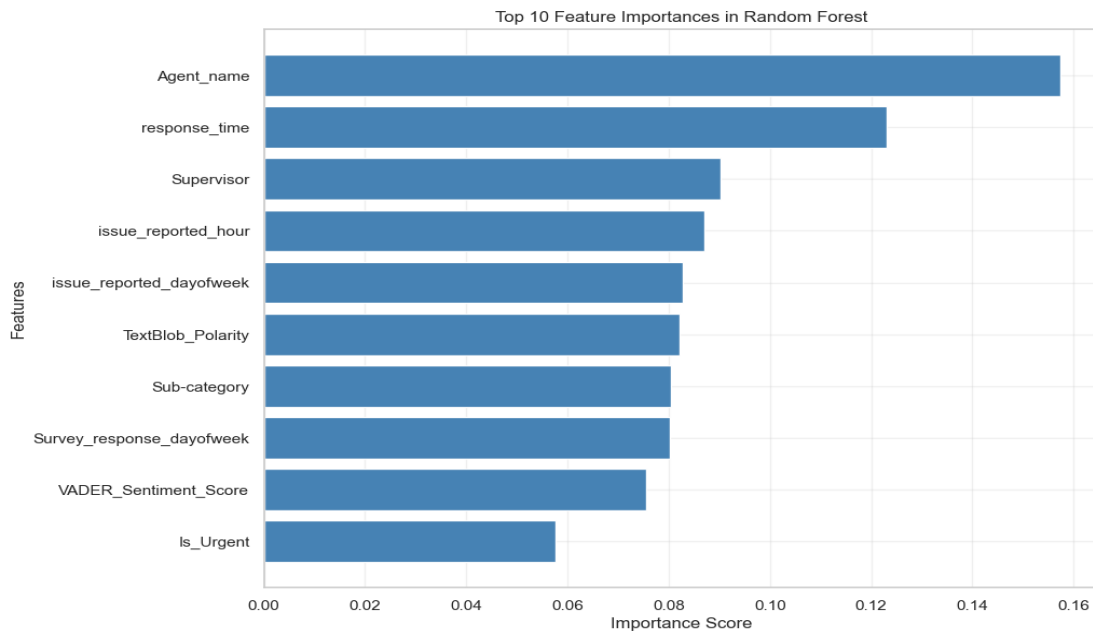


Figure 25: Top Features of Random Forest

Figure 25 illustrates the top 10 most important features in the Random Forest model ranked by their contribution to the predictions. The feature `Agent_name` stands out as the most significant factor, indicating that it plays a major role in the model's decision-making process. `response_time` and `Supervisor` follow as the next key contributors, suggesting their strong influence on the outcomes. Other important features include `issue_reported_hour` and `issue_reported_dayofweek`, highlighting moderate effects on the predictors. Sentiment-related features like `TextBlob_Polarity` and `VADER_Sentiment_Score` also play a role, along with categorical variables like `Sub-category`.

4.4 XGBoost Classifier:

MODEL	Evaluation Metrics	Before hyperparameter tuning	After Hyperparameter Tuning & Cross Validation	
			Training Data	Testing Data
XGBoost Classifier	Accuracy	0.77	0.79	0.71
	F1 Score	0.71	0.75	0.63

Table 9: Result of XGBoost Classifier

Table 9 highlights the strong performance of the XGBoost model, surpassing both Random Forest and Logistic Regression in terms of accuracy and F1 score. Without hyperparameter tuning, XGBoost achieved an accuracy of 0.77 and an F1 score of 0.71 on the training data. After tuning, the model's performance further improved, with accuracy increasing to 0.79 and the F1 score to 0.75 on the training data. On the testing data, XGBoost achieved an accuracy of 0.71 and an F1 score of 0.63, slightly trailing Random Forest in accuracy (0.72) but outperforming it in F1 score (0.62).

Despite the slight difference in testing accuracy, XGBoost demonstrated robust generalization by avoiding significant overfitting, unlike other models. These results emphasize XGBoost's ability to balance predictive power and consistency, making it a reliable and effective choice for this classification task.

4.4.1 Hyperparameter Tuning and Model Optimization:

```
# Defining the parameter grid
param_grid_xgb = {
    'n_estimators': [50, 100],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 5],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}
```

Figure 26: Details of Parameter Grid

```
Best parameters for XGBoost:
{'colsample_bytree': 1.0,
 'learning_rate': 0.1, 'max_depth': 5,
 'n_estimators': 100, 'subsample': 1.0}
```

Figure 27: Best parameters of XGBoost

GridSearchCV was utilized for hyperparameter optimization to systematically evaluate all possible combinations of predefined parameters, ensuring robust and thorough testing. The parameters tuned, as shown in Figure 26. A 5-fold stratified cross-validation strategy was applied to ensure balanced and reliable evaluation, using the F1 macro score as the performance metric. The best parameters identified were shown in Figure 27. This configuration achieved a mean F1 Macro score of 0.25,

reflecting the model's balanced performance across folds. Detailed information can be found in [Appendix 10](#).

Result of Confusion Matrix on Testing Data:

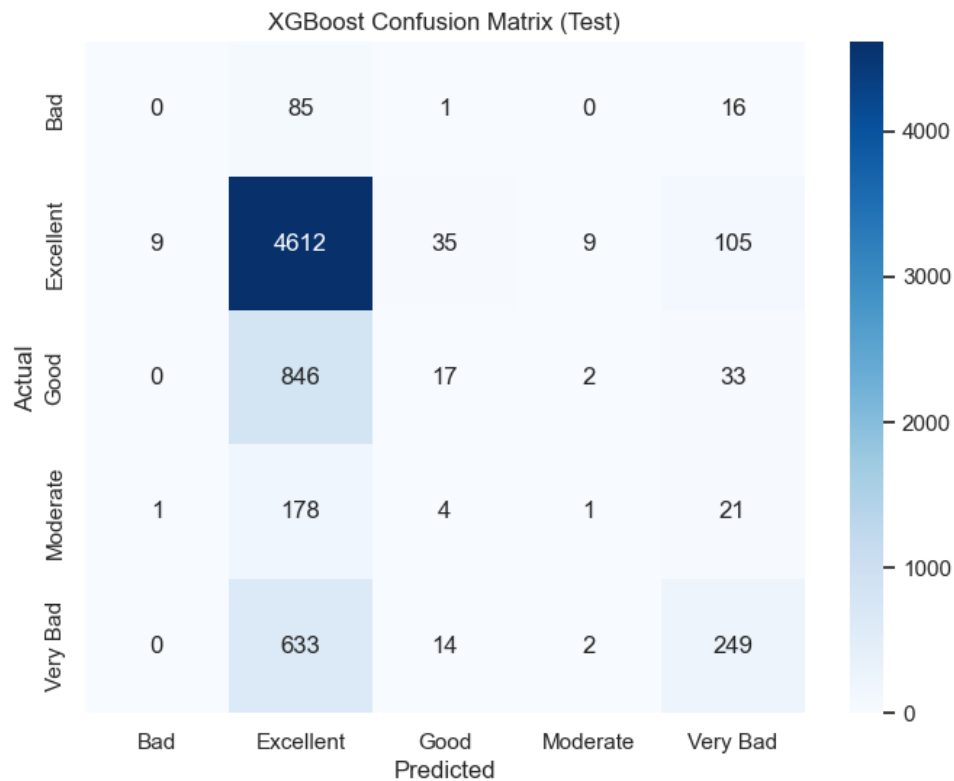


Figure 28: Result of Confusion Matrix on Testing Data

Figure 28 represents the performance of the XGBoost Classifier on the test dataset. The diagonal elements indicate the correctly predicted instances for each class, while the off-diagonal elements show misclassifications. The model performs well for the "Excellent" class, with 4612 correctly classified instances, but struggles with other classes, such as "Very Bad", where a significant number of instances (633) are misclassified as "Excellent". This highlights that while XGBoost excels in predicting the majority class, it has difficulty with minority or less represented classes, leading to imbalances in classification performance.

4.4.2 Strategies to Improve Minority Class Performance:

MODEL	Evaluation Metrics	Before SMOTE		After SMOTE	
		Training Data	Testing Data	Training Data	Testing Data
XGBoost Classifier	Precision	0.82	0.61	0.90	0.59
	Recall	0.79	0.71	0.89	0.71
	F1 Score	0.75	0.63	0.89	0.63

Table 10: Result of XGBoost Classifier before and after SMOTE

In this approach, we focused on improving the minority performance of the XGBoost model, particularly for handling imbalanced datasets. To address the class imbalance, I applied SMOTE to oversample the minority classes in the training data, ensuring that the model had sufficient examples from all classes to learn effectively. Label encoding was used to consistently convert categorical labels into numeric format for both the training and test datasets. Additionally, the model was evaluated using stratified cross-validation, which ensured class balance across folds for a fair and robust evaluation.

Table 10 compares the performance of the XGBoost Classifier before and after applying SMOTE to address class imbalance. Before SMOTE, the model achieved an F1 score of 0.75 on the training data and 0.63 on the testing data, with precision and recall values of 0.82 and 0.79 on the training data, and 0.61 and 0.71 on the testing data, respectively. After SMOTE, the training data performance improved significantly, with the F1 score rising to 0.89, and precision and recall increasing to 0.90 and 0.89, respectively. However, the testing data performance remained unchanged, with an F1 score of 0.63, precision of 0.59, and recall of 0.71. This indicates overfitting to the balanced training data, while generalization to the testing data did not improve.

Result of confusion Matrix on training data:

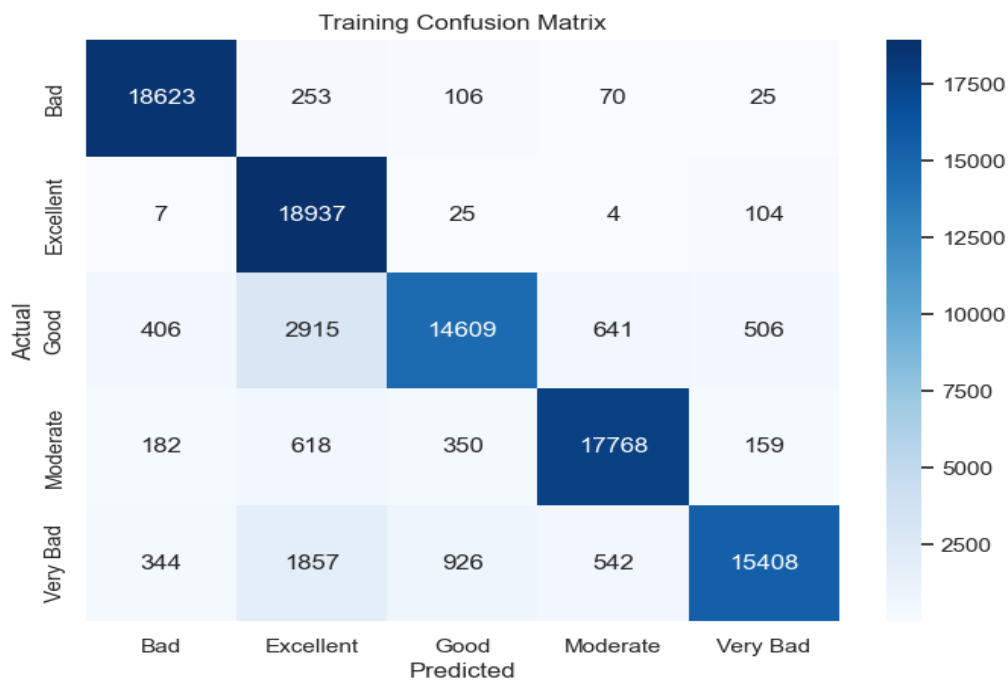


Figure 29: Result of confusion Matrix on training data after SMOTE

Figure 29 represents the performance of the XGBoost classifier on the training dataset. The diagonal elements indicate the number of correctly classified instances for each class, while the off-diagonal elements represent misclassifications. The model shows strong performance across all classes, with high correct classifications, such as 18,623 correctly classified instances in the "Bad" class and 18,937 in the "Excellent" class. However, some misclassifications are evident, such as 2,915 "Good" instances misclassified as "Excellent" and 1,857 "Very Bad" instances misclassified as "Excellent." This overall strong performance on the training data reflects the model's ability to fit the training data effectively, likely due to the use of SMOTE and hyperparameter tuning.

Result of confusion Matrix on Testing data:

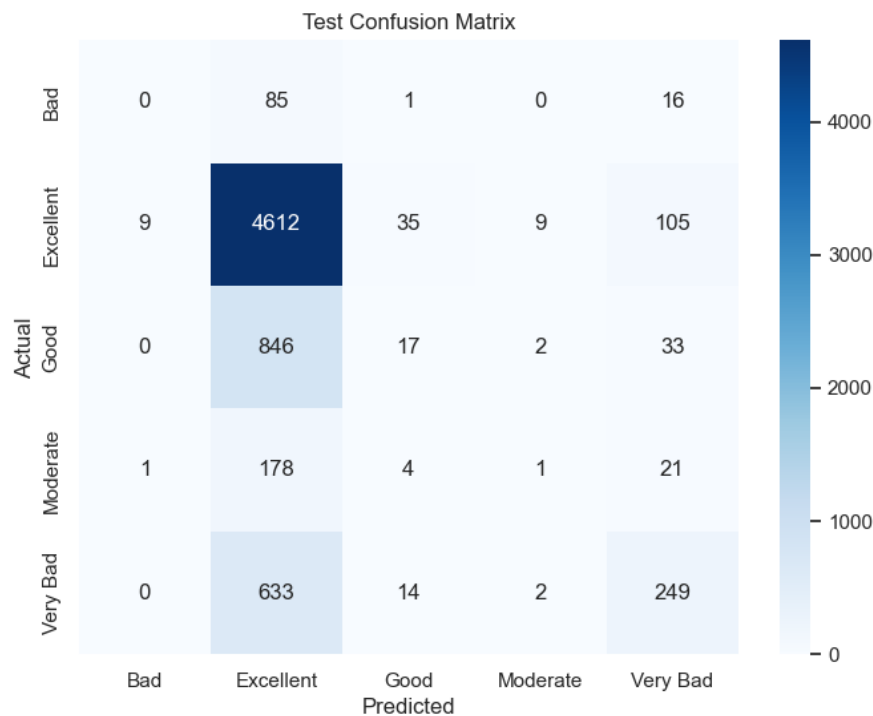


Figure 30: Result of confusion Matrix on Testing data after SMOTE

Figure 30 represents the performance of the XGBoost classifier on the test dataset. The diagonal elements show correctly classified instances for each class, while the off-diagonal elements highlight misclassifications. The "Excellent" class has the highest correct predictions, with 4,612 instances accurately classified. However, other classes, such as "Bad" and "Very Bad," show significant misclassifications. For example, most "Very Bad" instances (633) are misclassified as "Excellent," and the "Bad" class has no correctly classified instances.

These results indicate that while the model performs well on the "Excellent" class, it struggles to generalize effectively for minority or challenging classes in the test data, reflecting a potential imbalance or overlap in class features. But there is a good improvement when compared to other models. Because of the more imbalance dataset the model is not performing well on test data.

AUC-ROC Curve:

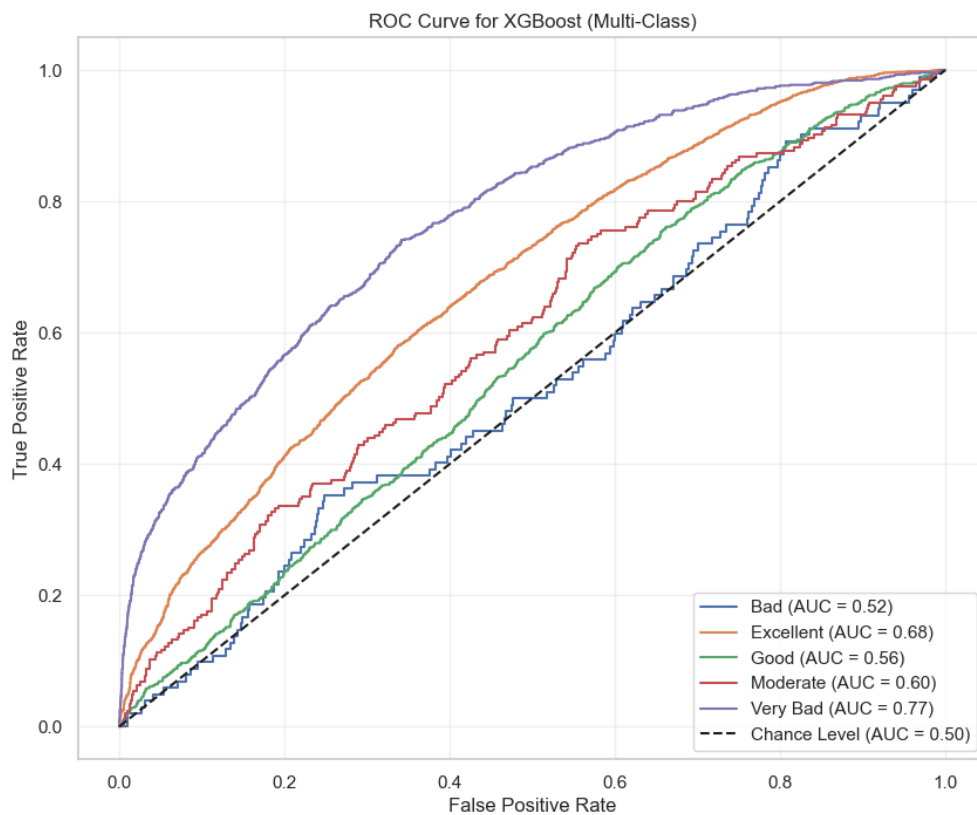


Figure 31: AUC-ROC Curve of each class

Figure 31 illustrates the multi-class performance of the XGBoost classifier, where each line represents the True Positive Rate (TPR) versus the False Positive Rate (FPR) for a specific class. The AUC (Area Under the Curve) values for the classes indicate their predictive performance. The "Very Bad" class has the highest AUC at 0.77, indicating relatively better separability for this class compared to others. The "Excellent" class follows with an AUC of 0.68, while the "Good" and "Moderate" classes have moderate AUC values of 0.56 and 0.60, respectively. The "Bad" class has the lowest AUC at 0.52, reflecting poor performance and difficulty in distinguishing this class from others.

Precision-Recall Curve:

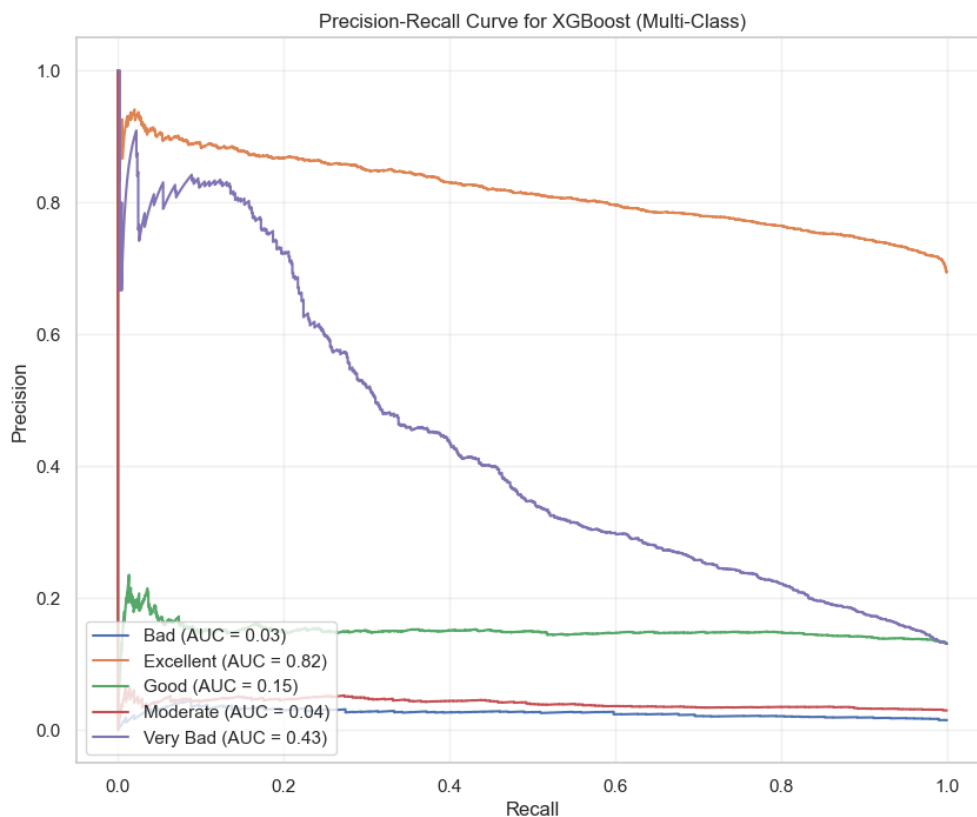


Figure 32: Precision-Recall Curve of each class

Figure 32 demonstrates its performance across multiple classes. The Excellent class achieves the highest AUC (0.82), indicating strong performance with high precision and recall. The Very Bad class follows with a moderate AUC (0.43), showing some ability to predict but with limitations. The minority classes, such as Bad (AUC = 0.03), Moderate (AUC = 0.04), and Good (AUC = 0.15), perform poorly, with low precision and recall. This indicates the model is heavily biased towards the majority class (Excellent) and struggles significantly with minority class predictions, highlighting challenges due to class imbalance.

4.4.2 Feature Importance:

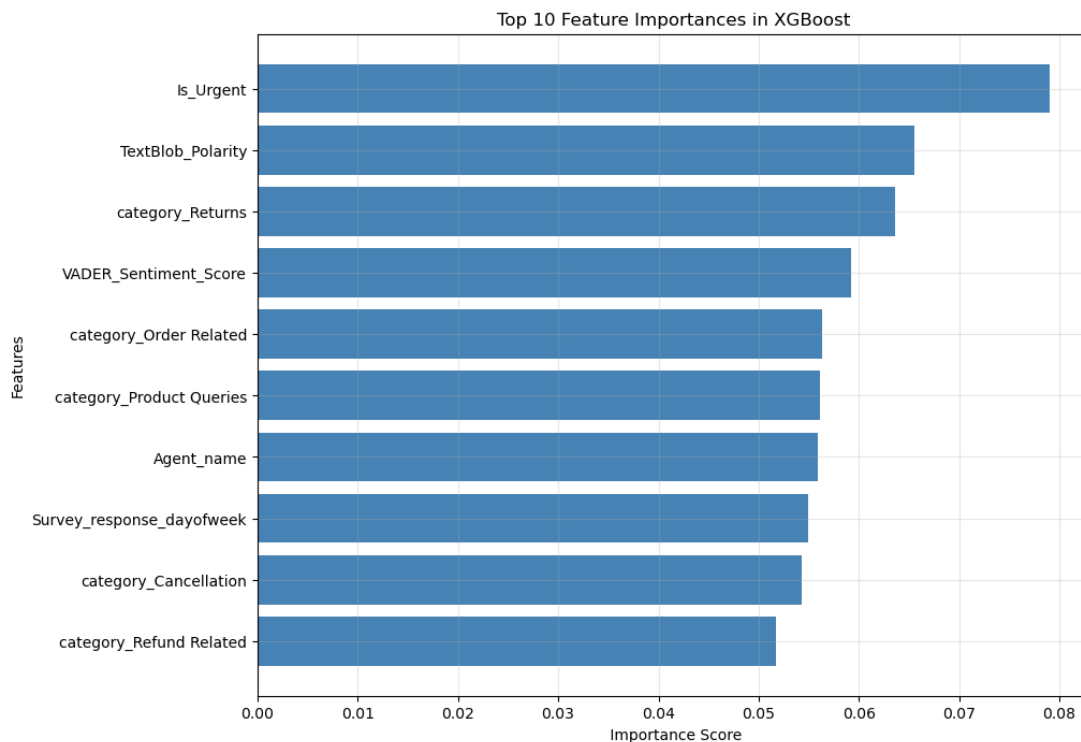


Figure 33: Top Features of XGBoost Classifier

Figure 33 illustrates the top 10 most important features for the XGBoost model based on their importance scores. The feature "Is_Urgent" has the highest importance, indicating it significantly impacts the model's predictions. Other important features include "TextBlob_Polarity", "category_Returns", and "VADER_Sentiment_Score", highlighting their relevance in the decision-making process. Features like "Agent_name" and "Survey_response_dayofweek" also contribute meaningfully but to a lesser extent. This ranking provides insights into the factors driving the model's predictions and highlights areas for potential feature engineering or data quality improvement.

CHAPTER FIVE: SUMMARY OF FINDINGS, CONCLUSION AND RECOMMENDATIONS

5.1 Summary of Findings:

The overall results of all the model on testing data

	Result	Logistic Regression	Random Forest	XG Boost
(With hyperparameter Tuning & Cross Validation)	Accuracy	0.69	0.72	0.71
	F1score	0.61	0.62	0.63
(With Oversampling SMOTE)	Accuracy	0.47	0.69	0.71
	F1score	0.51	0.62	0.63
Cohen Kappa Score		0.10	0.16	0.16

Table 11: Overall Comparison of all Models

Table 11 shows the overall comparison of all models. Logistic Regression performs reasonably well with hyperparameter tuning and cross-validation, achieving an accuracy of 0.69 and an F1 score of 0.61. This indicates that Logistic Regression can generalize well to the data when properly tuned. However, its performance drops significantly when SMOTE (Synthetic Minority Oversampling Technique) is applied, with accuracy declining to 0.47 and the F1 score falling to 0.51. This decline is likely due to Logistic Regression's linear nature, which makes it sensitive to artificially generated samples that may not align well with the true data distribution. Additionally, Logistic Regression records a Cohen's Kappa score of 0.10, highlighting a significant drop in agreement between predictions and actual labels. SMOTE adds artificial instances to minority classes, which may lead to distortion of the linear separability required by Logistic Regression.

Random Forest, on the other hand, performs strongly across both scenarios. With hyperparameter tuning and cross-validation, it achieves the highest accuracy of 0.72 and a decent F1 score of 0.62. When SMOTE is applied, Random Forest maintains a stable accuracy of 0.69 and F1 score of 0.62. This robustness can be attributed to Random Forest's ability to handle diverse data distributions and its ensemble nature, which makes it less sensitive to the synthetic data generated by SMOTE. Random Forest also demonstrates a significant improvement in Cohen's Kappa score, increasing from 0.10 to 0.16 after SMOTE compared to Logistic Regression, indicating better agreement with true labels due

to its ability to adapt to synthetic data effectively. By creating multiple decision trees and aggregating their results, Random Forest effectively mitigates the impact of oversampling techniques.

XGBoost demonstrates consistent performance in both scenarios, achieving an accuracy of 0.71 and an F1 score of 0.63 in both cases. This stability is a key strength of XGBoost, which leverages advanced boosting techniques to iteratively minimize error and adapt to data variations. XGBoost is less affected by SMOTE due to its ability to handle complex and imbalanced datasets effectively. Its tree-based boosting mechanism ensures that it remains resilient to the introduction of synthetic samples, making it a reliable choice for both original and oversampled datasets. XGBoost maintains a stable Cohen's Kappa score of 0.16 across both scenarios, reflecting consistent agreement between predictions and actual labels. Overall, XGBoost and Random Forest are more robust to SMOTE compared to Logistic Regression, which struggles with artificially generated data.

5.1.1 Top Predictors for CSAT score:

Random Forest	XG Boost
Agent Name	Is Urgent
Response Time	Text Blob Polarity
Supervisor	Category Returns
Issue Reported Hour	Vader Sentiment Score
Issue Reported Day of Week	Category Order Related
Text Blob Polarity	Category Product Queries
Sub-Category	Agent Name
Survey Response Day of Week	Survey Response Day of Week
Vader Sentiment Score	Category Cancellation
Is Urgent	Category Refund Related

Table 12: Comparison of Top predictors

Table 12 compares the top features identified by the Random Forest and XGBoost models. Both models highlight different factors as being most important for their predictions. Random Forest emphasizes features like Agent Name, Response Time, and Issue Reported Hour, while XGBoost prioritizes Is Urgent, Text Blob Polarity, and Category Returns. Some features, such as Survey Response Day of Week and Text Blob Polarity, are considered important by both models, though their relative rankings differ. Overall, Random Forest seems to focus more on operational and timing-related features, while XGBoost gives higher importance to urgency, sentiment, and categorical context, reflecting different approaches to identifying the drivers behind predictions.

IMPORTANCE OF NEW FEATURES:

The newly created features, Is Urgent and Response Time, play a critical role in enhancing model performance. In XGBoost, Is Urgent stands out as the most influential feature, helping the model effectively classify high-priority cases by capturing the urgency of an issue. Similarly, in Random Forest, Response Time is highly ranked, reflecting its significance in predicting outcomes by focusing on the duration taken to address issues, which directly impacts efficiency and customer satisfaction. These features add valuable dimensions of urgency and timeliness, enabling the models to better understand and predict real-world scenarios.

5.2 Conclusion:

The research successfully addressed the formulated questions, providing detailed insights into customer satisfaction in e-commerce and highlighting actionable strategies for service optimization. The key predictors influencing customer satisfaction were identified as response time, sentiment polarity, and issue categories such as returns and order-related queries. It was evident that faster resolution times, combined with effective handling of common issues, significantly improved Customer Satisfaction Scores (CSAT). Sentiment analysis, implemented using the VADER algorithm, contributed substantial value by categorizing customer feedback into positive, neutral, and negative sentiments, enabling businesses to better understand customer emotions. This insight allowed for the prioritization of dissatisfied customers and targeted improvements in areas with negative feedback, further enhancing overall satisfaction.

The study evaluated three machine learning models—XGBoost, Random Forest, and Logistic Regression—to compare their predictive accuracy and scalability. XGBoost emerged as the best-

performing model, achieving 71% accuracy on the test dataset, with robust handling of complex relationships and imbalanced data. It demonstrated scalability and was effective in delivering reliable results across diverse datasets. Random Forest performed similarly, achieving 72% accuracy, but its sensitivity to synthetic data from SMOTE slightly reduced its reliability for imbalanced datasets. Logistic Regression, while achieving a lower accuracy of 69%, played a crucial role as a baseline model, offering clear interpretability and insights into feature interactions, such as how response time and sentiment directly influenced CSAT.

Insights derived from the predictive models emphasized the importance of addressing frequently reported issues, such as returns, which were the most common customer concerns, and ensuring quicker resolution times. Additionally, sentiment analysis helped identify areas for proactive improvements, such as resolving complaints reported during non-business hours. By analyzing agent-specific performance, businesses could recognize top-performing agents and provide additional training to those underperforming. Furthermore, the study highlighted optimization strategies such as better resource allocation for frequently occurring issue categories and leveraging predictive analytics to prevent churn and enhance overall operational efficiency.

In terms of scalability and performance, XGBoost stood out for its ability to handle large, complex datasets efficiently, making it highly suitable for broader e-commerce applications. Random Forest offered reliable results for non-linear patterns but required careful tuning to manage imbalanced datasets effectively. Logistic Regression provided computational efficiency and interpretability, but its limitations in capturing complex relationships made it less scalable for larger datasets. Overall, the research showcased the transformative potential of machine learning in enhancing customer satisfaction, demonstrating how predictive models and sentiment analysis can drive data-driven decisions, improve service delivery, and optimize operational strategies in the competitive e-commerce sector.

5.2.1 Limitations of the study:

Though the study achieved commendable accuracy, it faced certain limitations. One significant challenge was the class imbalance in the dataset, where 'Bad' and 'Very Bad' satisfaction ratings were significantly underrepresented compared to 'Excellent' ratings. This imbalance created difficulties for the machine learning models, particularly in accurately predicting ratings for minority classes. SMOTE (Synthetic Minority Oversampling Technique) was applied to address this issue by enhancing the representation of underrepresented classes, which improved the models ability to detect patterns in

these ratings. However, despite the application of SMOTE, models such as XGBoost and Random Forest struggled to generalize effectively on the testing data for minority classes, primarily due to the inherent imbalance in the dataset. These findings highlight the importance of using more balanced and diverse datasets in future studies to improve predictive performance and ensure consistent accuracy across all satisfaction levels.

The study focused exclusively on textual feedback, omitting non-textual data like emojis, images, and audio inputs, which limited insights into customer satisfaction. Emojis, often used in reviews, convey nuanced emotions, while images and audio provide visual and tonal context, enriching the analysis. By excluding these elements, the study missed the opportunity for a more comprehensive understanding of feedback.

5.3 Recommendations:

The organization can significantly benefit from implementing machine learning models, such as XGBoost and Random Forest, to analyze Customer Satisfaction Scores (CSAT). These models provide valuable insights into customer feedback, enabling the organization to identify trends, proactively address service-related issues, and enhance customer satisfaction. By leveraging these predictive capabilities, the organization can build a more customer-centric approach to improve loyalty and retention.

A key application of these models is in monitoring and improving agent performance. CSAT scores can be linked to specific interactions, helping the organization evaluate individual agent performance. High-performing agents can be recognized and rewarded, fostering motivation and maintaining consistent service quality. At the same time, underperforming agents can be identified early, allowing for targeted training and support to address their challenges. This approach ensures efficient customer service delivery and boosts team morale.

The models also enable the organization to address recurring issues by analyzing patterns in low-rated customer feedback. By focusing on common pain points, the organization can implement targeted improvements to prevent repeated dissatisfaction. This proactive resolution of issues strengthens customer trust and enhances the overall experience. Furthermore, incorporating sentiment analysis tools like VADER adds depth to feedback analysis by capturing emotional nuances in textual feedback. Understanding customer sentiment allows the organization to tailor responses, providing a more personalized and engaging customer experience.

To further enhance the analysis, the organization can expand its scope to include multimodal feedback, such as emojis, images, and audio inputs. These additional sources of data provide a comprehensive understanding of customer emotions and experiences, enriching insights and improving predictive accuracy. Additionally, insights from these models can guide data-driven decision-making, enabling the organization to prioritize resource allocation and operational improvements. Investments in high-impact areas, such as addressing frequently reported issues or improving specific service channels, can maximize efficiency and customer satisfaction.

Finally, the models' ability to predict dissatisfaction offers opportunities for targeted customer retention strategies. By identifying customers at risk of churn, the organization can implement personalized follow-ups, offers, or solutions to retain them and build long-term loyalty. Implementing these recommendations will empower the organization to enhance customer satisfaction, optimize operations, and improve overall business performance.

5.3.1 Future Studies:

Future research can explore the integration of advanced feature extraction techniques like Word2Vec, GloVe, and BERT to improve text representation and enhance model accuracy. Word2Vec and GloVe excel at capturing semantic relationships between words, while BERT provides context-specific understanding, making it particularly effective for nuanced textual data. Comparing these methods across different datasets could help identify the most suitable approach for varying applications, ultimately improving prediction quality and adaptability.

Additionally, future studies can focus on hybrid models that combine machine learning and deep learning techniques, such as CNN-LSTM, to handle complex and imbalanced datasets more effectively. These models can capture both local text patterns and sequential dependencies, leading to improved accuracy. Incorporating ensemble methods could further enhance robustness, allowing models to adapt to diverse data types.

Moreover, Analyzing emotional nuances, such as emojis and tone variations, can deepen insights into customer satisfaction. Emojis convey subtle emotions, while tone analysis adds valuable context. Integrating these elements into predictive models can greatly enhance their accuracy and comprehensiveness.

6. References:

- Arias-Barahona, M.X., Arteaga-Arteaga, H.B., Orozco-Arias, S., Flórez-Ruíz, J.C., Valencia-Díaz, M.A. and Tabares-Soto, R. (2023). Requests classification in the customer service area for software companies using machine learning and natural language processing. *PeerJ Computer Science*, 9, p.e1016. doi:<https://doi.org/10.7717/peerj-cs.1016>
- Asniar and Surendro, K. (2019). Predictive Analytics for Predicting Customer Behavior. *2019 International Conference of Artificial Intelligence and Information Technology (ICAIIIT)*. doi:<https://doi.org/10.1109/icaaiit.2019.8834571>
- Ballestar, M.T., Grau-Carles, P. and Sainz, J. (2018). Predicting customer quality in e-commerce social networks: a machine learning approach. *Review of Managerial Science*, 13(3), pp.589–603. doi:<https://doi.org/10.1007/s11846-018-0316-x>
- Barnwal, S.K. and Gupta, P. (2022). *Evaluation of AI System's Voice Recognition Performance in Social Conversation*. [online] IEEE Xplore. doi:<https://doi.org/10.1109/IC3I56241.2022.10073242>
- Bawack, R.E., Wamba, S.F., Carillo, K.D.A. and Akter, S. (2022). Artificial Intelligence in E-Commerce: a Bibliometric Study and Literature Review. *Electronic Markets*, [online] 32(1). Available at: <https://link.springer.com/article/10.1007/s12525-022-00537-z>
- Boukrouh, I. and Azmani, A. (2025). Explainable machine learning models applied to predicting customer churn for e-commerce. *IAES International Journal of Artificial Intelligence (IJ-AI)*, 14(1), p.286. doi:<https://doi.org/10.11591/ijai.v14.i1.pp286-297>
- Britannica. (n.d.). E-commerce. In *Encyclopædia Britannica*. Retrieved January 19, 2025, from <https://www.britannica.com/technology/e-commerce>
- Cui, L., Huang, S., Wei, F., Tan, C., Duan, C. and Zhou, M. (2017). SuperAgent: A Customer Service Chatbot for E-commerce Websites. *Proceedings of ACL 2017, System Demonstrations*. [online] doi:<https://doi.org/10.18653/v1/p17-4017>
- Dihingia, H., Ahmed, S., Borah, D., Gupta, S., Phukan, K. and Muchahari, M.K. (2021). Chatbot Implementation in Customer Service Industry through Deep Neural Networks. *2021 International Conference on Computational Performance Evaluation (ComPE)*. doi:<https://doi.org/10.1109/compe53109.2021.9752271>

- Dlodlo, D. and Sibanda, K. (2023). Automated Ticket Classification for Information Technology Helpdesks using Machine Learning. *2023 2nd Zimbabwe Conference of Information and Communication Technologies (ZCICT)*, [online] pp.1–7. doi:<https://doi.org/10.1109/zcict59466.2023.10528578>
- Eid, M.I. (2011). Determinants of E-Commerce Customer Satisfaction, Trust, and Loyalty in Saudi Arabia. *Journal of Electronic Commerce Research*. [online] Available at: https://www.researchgate.net/publication/228867201_Determinants_of_E-Commerce_Customer_Satisfaction_Trust_and_Loyalty_in_Saudi_Arabia
- Gowda, A., Su, H.-K. and Kuo, W.-K. (2024). Personalized E-commerce: Enhancing Customer Experience through Machine Learning-driven Personalization. pp.1–5. doi:<https://doi.org/10.1109/iciteics61368.2024.10624901>
- Gupta, T. and Bansal, S. (2018). Machine Learning Algorithms for Predictive Analytics in E-Commerce. *International Journal of Science and Research (IJSR) ResearchGate Impact Factor*. [online] doi:<https://doi.org/10.21275/SR24302212009>
- Huang, H., Zavareh, A.A. and Mustafa, M.B. (2023). Sentiment Analysis in E-Commerce Platforms: A Review of Current Techniques and Future Directions. *IEEE Access*, [online] 11, pp.90367–90382. doi:<https://doi.org/10.1109/ACCESS.2023.3307308>
- Ingaldi, M. and Ulewicz, R. (2019). How to Make E-Commerce More Successful by Use of Kano's Model to Assess Customer Satisfaction in Terms of Sustainable Development. *Sustainability*, [online] 11(18), p.4830. doi:<https://doi.org/10.3390/su11184830>
- Jabbar, J., Urooj, I., JunSheng, W. and Azeem, N. (2019). Real-time Sentiment Analysis On E-Commerce Application. *2019 IEEE 16th International Conference on Networking, Sensing and Control (ICNSC)*. doi:<https://doi.org/10.1109/icnsc.2019.8743331>
- Jeidy Panduro-Ramirez (2024). Sentiment Analysis in Customer Reviews for Product Recommendation in E-commerce Using Machine Learning. *2022 International Conference on Advances in Computing, Communication and Applied Informatics (ACCAI)*, pp.1–5. doi:<https://doi.org/10.1109/accai61061.2024.10602027>
- Khalid, Jugurnauth, R.A. and Muhammad (2022). Application of Machine Learning in Predicting Customer Satisfaction of Telecom Service Providers. doi:<https://doi.org/10.1109/elecom54934.2022.9965212>

Krishnan, C. and Mariappan, J. (2024). 'The AI Revolution in E-Commerce: Personalization and Predictive Analytics'. *Studies in computational intelligence*, pp.53–64. doi:https://doi.org/10.1007/978-3-031-55615-9_4

Kyritsis, A.I. and Deriaz, M. (2019). *A Machine Learning Approach to Waiting Time Prediction in Queueing Scenarios*. [online] IEEE Xplore. doi:<https://doi.org/10.1109/AI4I46381.2019.00013>

Laudon, K.C. and Traver, C.G., 2020. *E-commerce 2020: Business, Technology, and Society*. 16th ed. Pearson Education. Available at: https://books.google.com/books/about/E_Commerce_2020_2021_Business_Technology.html?id=E8AoEAAAQBAJ [Accessed 19 January 2025].

Manzoor, A., M. Atif Qureshi, Kidney, E. and Longo, L. (2024). A Review on Machine Learning Methods for Customer Churn Prediction and Recommendations for Business Practitioners. *IEEE access*, pp.1–1. doi:<https://doi.org/10.1109/access.2024.3402092>

Nadaf, M.A. and Patil, S.S. (2015). Performance evaluation of categorizing technical support requests using advanced K-Means algorithm. *2015 IEEE International Advance Computing Conference (IACC)*, pp.409–414. doi:<https://doi.org/10.1109/iadcc.2015.7154740>

Nitesh Upadhyaya (2024). Enhancing Real-Time Customer Service through Adaptive Machine Learning. *International Journal of Advanced Research in Science, Communication and Technology*, pp.630–636. doi:<https://doi.org/10.48175/ijarsct-19381>

Rattapol Kasemrat and Tanpat Kraiwanit (2024). Benchmarking Machine Learning Models for Predictive Analytics in E-Commerce. doi:<https://doi.org/10.2139/ssrn.4832967>

Sabbeh, S.F. (2018). Machine-Learning Techniques for Customer Retention: a Comparative Study. *International Journal of Advanced Computer Science and Applications*, 9(2). doi:<https://doi.org/10.14569/ijacsa.2018.090238>

Sharma, D.A., Mishra, D.S.K. and Srivastav, M.V.K. (2023). The Evolution And Impact Of E-Commerce. *Journal of Namibian Studies : History Politics Culture*, [online] 33, pp.1838–1846. doi:<https://doi.org/10.59670/jns.v33i.3260>

Statista. (2023). Global retail e-commerce sales 2014-2027. Retrieved January 19, 2025, from <https://www.statista.com/statistics/379046/worldwide-retail-e-commerce-sales/>

Statista. (2023). Key figures of e-commerce. Retrieved January 19, 2025, from <https://www.statista.com/markets/413/e-commerce/>

Supriya Jawale, Aparna Bondade, Manish Sontakke, Aditya Sohagpure, Akash Muneshwar and Nisarga Wankhede (2024). Extended Reality E-Commerce: The Evolution of E-Commerce through XR Based Technologies. pp.1–7. doi:<https://doi.org/10.1109/otcon60325.2024.10687746>

Tai, T.-E., Haw, S.-C., Ng, K.-W., Palanichamy Naveen and Mutaz Al-Tarawneh (2023). Performance Evaluation on Resolution Time Prediction Using Decision Tree, Random Forest and eXtreme Gradient Boosting. [online] pp.74–79. doi:<https://doi.org/10.1109/ccat59108.2023.00021>

Vanneschi, L., Horn, D.M., Castelli, M. and Popovič, A. (2018). An artificial intelligence system for predicting customer default in e-commerce. *Expert Systems with Applications*, [online] 104, pp.1–21. doi:<https://doi.org/10.1016/j.eswa.2018.03.025>

Yu, W. and Weng, W. (2022). *Customer Churn Prediction Based on Machine Learning*. [online] IEEE Xplore. doi:<https://doi.org/10.1109/AIAM57466.2022.00176>

Zaghloul, M., Barakat, S. and Rezk, A. (2024). Predicting E-commerce customer satisfaction: Traditional machine learning vs. deep learning approaches. *Journal of Retailing and Consumer Services*, [online] 79, p.103865. doi:<https://doi.org/10.1016/j.jretconser.2024.103865>

Zuev, D., Kalistratov, A. and Zuev, A. (2018). Machine Learning in IT Service Management. *Procedia Computer Science*, 145, pp.675–679. doi:<https://doi.org/10.1016/j.procs.2018.11.063>

Appendix:

Appendix 1: Stratified Sampling

```
import pandas as pd

# Defining the proportion of samples to take from each column based on target variable
stratify_col = 'CSAT Score'
sample_size = 0.4 # 40% of the data

if not isinstance(df, pd.DataFrame):
    df = pd.DataFrame(df)

# Checking if the stratify column exists
if stratify_col not in df.columns:
    raise ValueError(f"The column '{stratify_col}' does not exist in the DataFrame")

# Perform stratified sampling
df = df.groupby(stratify_col, group_keys=False).apply(lambda x: x.sample(frac=sample_size))

print(df.shape)

(34363, 20)
```

Appendix 2: Feature Engineering

```
# 2. Date and Time Processing
# Convert relevant columns to datetime
date_columns = ['Issue_reported at', 'issue_responded']
for col in date_columns:
    df[col] = pd.to_datetime(df[col], errors='coerce', format='%d-%m-%Y %H:%M')

df['Survey_response_Date'] = pd.to_datetime(df['Survey_response_Date'], format='%d-%b-%y', errors='coerce')

# Create features from datetime columns
df['response_time'] = (df['issue_responded'] - df['Issue_reported at']).dt.total_seconds() / 60 # in minutes

# Extract day of week in survey response date
df['Survey_response_dayofweek'] = df['Survey_response_Date'].dt.dayofweek

# Extract day of week and hour from date columns for additional features
df['issue_reported_dayofweek'] = df['Issue_reported at'].dt.dayofweek
df['issue_reported_hour'] = df['Issue_reported at'].dt.hour

# Create urgency flag
df['Is_Urgent'] = np.where(df['issue_reported_hour'].between(18, 23), 1, 0)
```

Appendix 3: Encoding

One hot Encoding

```
# using one hot encoder for low cardinality columns
low_card_cols = [ 'category']

# Applying one hot encoder for the above columns

df = pd.get_dummies(df, columns=low_card_cols, drop_first=True)

# Display columns after one-hot encoding
print("Dataframe columns after One-Hot Encoding for low-cardinality features:\n", df.columns)
```

```
Dataframe columns after One-Hot Encoding for low-cardinality features:
Index(['Sub-category', 'Customer Remarks', 'Agent_name', 'Supervisor',
      'CSAT Score', 'response_time', 'Survey_response_dayofweek',
      'issue_reported_dayofweek', 'issue_reported_hour', 'Is_Urgent',
      'category_App/website', 'category_Cancellation', 'category_Feedback',
      'category_Offers & Cashback', 'category_Onboarding related',
      'category_Order Related', 'category_Others',
      'category_Payments related', 'category_Product Queries',
      'category_Refund Related', 'category>Returns'],
      dtype='object')
```

Label Encoding:

```
# Label encoder for high cardinality columns

high_card_cols = ['Agent_name', 'Supervisor','Sub-category']

# Apply Label Encoding for each high-cardinality column individually
for col in high_card_cols:
    labelEnc = LabelEncoder()
    df[col] = labelEnc.fit_transform(df[col])

# Display encoded columns and some sample rows
print("\nAfter label Encoding for High Cardinality Columns:")
print(df[high_card_cols].head())
```

After label Encoding for High Cardinality Columns:

	Agent_name	Supervisor	Sub-category
57652	659	16	22
50164	1237	12	35
15557	490	1	15
57009	785	27	38
33965	323	32	22

Appendix 4: Scaling

```
# Define numeric columns that need scaling.
scale_columns = [ 'response_time', 'Survey_response_dayofweek', 'issue_reported_dayofweek',
                  'issue_reported_hour', 'Is_Urgent' ]

# Applying Standard Scaler
scaler = StandardScaler()
df[scale_columns] = scaler.fit_transform(df[scale_columns])
print("Data after Standardization:\n", df)
```

Appendix 5: Data Splitting

Splitting of Data

```
X = df.drop('CSAT Score', axis=1)
Y = df['CSAT Score']
```

```
from sklearn.model_selection import StratifiedShuffleSplit

# Initialize Stratified Shuffle Split
stratified_split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

for train_index, test_index in stratified_split.split(X, Y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    Y_train, Y_test = Y.iloc[train_index], Y.iloc[test_index]

print("Training data size:", X_train.shape)
print("Testing data size:", X_test.shape)
print("Y train data", Y_train.shape)
```

```
Training data size: (27490, 21)
Testing data size: (6873, 21)
Y train data (27490,)
```

Appendix 6: Sentiment Analysis

Pre-Processing

```
# Download necessary NLTK data
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('vader_lexicon')

# Text preprocessing function
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'^a-z\s', '', text) # Remove special characters
    words = word_tokenize(text)
    words = [word for word in words if word not in stopwords.words('english')] # Remove stopwords
    lemmatizer = WordNetLemmatizer()
    words = [lemmatizer.lemmatize(word) for word in words] # Lemmatize words
    return ' '.join(words)

df['Cleaned_Remarks'] = df['Customer_Remarks'].apply(preprocess_text)
```

applying VADER Sentiment Analyzer

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Initialize VADER SentimentIntensityAnalyzer
vader = SentimentIntensityAnalyzer()

# Function to compute VADER sentiment scores
def analyze_sentiment_vader(text):
    scores = vader.polarity_scores(text)
    return scores['compound'] # Return the compound score

# Apply VADER sentiment analysis
df['VADER_Sentiment_Score'] = df['Cleaned_Remarks'].apply(analyze_sentiment_vader)

# Categorize sentiment based on compound score
def categorize_sentiment(score):
    if score >= 0.05:
        return 'Positive'
    elif score <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'

df['VADER_Sentiment_Category'] = df['VADER_Sentiment_Score'].apply(categorize_sentiment)

# Function to compute TextBlob sentiment scores
def analyze_sentiment_textblob(text):
    sentiment = TextBlob(text).sentiment
    return pd.Series([sentiment.polarity, sentiment.subjectivity])

# Apply TextBlob sentiment analysis
df[['TextBlob_Polarity', 'TextBlob_Subjectivity']] = df['Cleaned_Remarks'].apply(analyze_sentiment_textblob)
```

Visualization:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Plot the countplot
ax = sns.countplot(data=df, x='VADER_Sentiment_Category', palette='coolwarm')

# Add the numbers on top of the bars
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', # Convert the height to an integer
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center',
                va='center',
                fontsize=14, # Increase the font size for better visibility
                color='black',
                xytext=(0, 12), # Adjust text offset for better spacing
                textcoords='offset points')

# Set the title and labels
plt.title('Sentiment Distribution (VADER)', fontsize=16) # Increase title font size
plt.xlabel('Sentiment Category', fontsize=12) # Increase x-axis label font size
plt.ylabel('Count', fontsize=12) # Increase y-axis label font size

# Increase the y-axis length
max_y = max([p.get_height() for p in ax.patches]) # Find the maximum value on y-axis
plt.ylim(0, max_y + 2100) # Set a higher limit to add more space above the tallest bar

# Show the plot
plt.show()
```

```
# Plot polarity vs. subjectivity using TextBlob
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x='TextBlob_Polarity',
    y='TextBlob_Subjectivity',
    hue='VADER_Sentiment_Category',
    palette='coolwarm',
    s=100
)
plt.title('Polarity vs. Subjectivity (TextBlob)')
plt.xlabel('Polarity')
plt.ylabel('Subjectivity')
plt.legend(title='Sentiment')
plt.show()
```

Appendix 7: Baseline Method:

Code for Training Data:

Baseline Method

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

# Most Frequent Class Baseline
most_frequent = DummyClassifier(strategy='most_frequent')
most_frequent.fit(X_train, Y_train)
Y_pred = most_frequent.predict(X_train)

#calculate accuracy
accuracy_bl = accuracy_score(Y_train, Y_pred)
print(f"Baseline Accuracy: {accuracy_bl:.2f}")
# Calculate F1 score
f1score_bl = f1_score(Y_train, Y_pred, average='weighted')
print(f"Baseline F1 Score: {f1score_bl:.2f}")

# Calculate confusion matrix
confusion_matrix = confusion_matrix(Y_train, Y_pred)
print(f"Baseline Confusion Matrix:\n{confusion_matrix}")

print("\nClassification Report:\n", classification_report(Y_train, Y_pred))
```

Baseline Accuracy: 0.69

Baseline F1 Score: 0.57

Baseline Confusion Matrix:

```
[[ 0  411  0  0  0]
 [ 0 19077 0  0  0]
 [ 0  3590 0  0  0]
 [ 0   818 0  0  0]
 [ 0  3594 0  0  0]]
```

Classification Report:

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	411
Excellent	0.69	1.00	0.82	19077
Good	0.00	0.00	0.00	3590
Moderate	0.00	0.00	0.00	818
Very Bad	0.00	0.00	0.00	3594
accuracy			0.69	27490
macro avg	0.14	0.20	0.16	27490
weighted avg	0.48	0.69	0.57	27490

Result of Testing data:

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

# Predict on testing data
Y_pred_test = most_frequent.predict(X_test)

# Calculate Accuracy
accuracy_bl_test = accuracy_score(Y_test, Y_pred_test)
print(f"Baseline Accuracy (Test): {accuracy_bl_test:.2f}")

# Calculate F1 Score
f1score_bl_test = f1_score(Y_test, Y_pred_test, average='weighted')
print(f"Baseline F1 Score (Test): {f1score_bl_test:.2f}")

# Calculate Confusion Matrix
conf_matrix_bl_test = confusion_matrix(Y_test, Y_pred_test)
print(f"Baseline Confusion Matrix (Test):\n{conf_matrix_bl_test}")

# Print Classification Report
print("\nBaseline Classification Report (Test):\n", classification_report(Y_test, Y_pred_test))
```

Baseline Accuracy (Test): 0.69

Baseline F1 Score (Test): 0.57

Baseline Confusion Matrix (Test):

```
[[ 0 102  0  0  0]
 [ 0 4770  0  0  0]
 [ 0 898  0  0  0]
 [ 0 205  0  0  0]
 [ 0 898  0  0  0]]
```

Baseline Classification Report (Test):

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	102
Excellent	0.69	1.00	0.82	4770
Good	0.00	0.00	0.00	898
Moderate	0.00	0.00	0.00	205
Very Bad	0.00	0.00	0.00	898
accuracy			0.69	6873
macro avg	0.14	0.20	0.16	6873
weighted avg	0.48	0.69	0.57	6873

Appendix 8: Logistic Regression:

Logistic Regression Before Hyperparameter Tuning:

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from sklearn.preprocessing import LabelEncoder

# If ytrain is not encoded, encode it to numerical values
#label_encoder = LabelEncoder()
#ytrain_encoded_lr = label_encoder.fit_transform(Y_train)

# Initialize the Logistic Regression model
log_reg = LogisticRegression(max_iter=1000, random_state=42) # max_iter increased for convergence

# Fit the model
log_reg.fit(X_train, Y_train)

# Predict on the training data
y_pred_lr = log_reg.predict(X_train)
# Calculate accuracy
accuracy_lr = accuracy_score(Y_train, y_pred_lr)
print(f"Logistic Regression Accuracy: {accuracy_lr:.2f}")

# Calculate F1 score
f1score_lr = f1_score(Y_train, y_pred_lr, average='weighted')
print(f"Logistic Regression F1 Score: {f1score_lr:.2f}")

# Calculate confusion matrix
conf_matrix = confusion_matrix(Y_train, y_pred_lr)
print(f"Logistic Regression Confusion Matrix:\n{conf_matrix}")

# Print classification report
print("\nClassification Report:\n", classification_report(Y_train, y_pred_lr))
```

```
Logistic Regression Accuracy: 0.70
Logistic Regression F1 Score: 0.59
Logistic Regression Confusion Matrix:
[[ 0 396  0  0 15]
 [ 0 18969  0  0 108]
 [ 0 3569  0  0 21]
 [ 0 800  0  0 18]
 [ 0 3251  0  0 343]]
```

```
Classification Report:
              precision    recall  f1-score   support

   Bad           0.00        0.00        0.00         411
  Excellent       0.70        0.99        0.82       19077
    Good           0.00        0.00        0.00        3590
  Moderate        0.00        0.00        0.00         818
   Very Bad       0.68        0.10        0.17        3594

 accuracy                   0.70       27490
 macro avg              0.28        0.22        0.20       27490
 weighted avg           0.58        0.70        0.59       27490
```

Logistic Regression After Hyperparameter Tuning:

```
import os
from joblib import dump
from sklearn.model_selection import GridSearchCV, StratifiedKFold

# Define the Logistic Regression model
log_reg = LogisticRegression(max_iter=1000, random_state=42)

# Define the parameter grid for hyperparameter tuning
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'saga']
}

# Define the cross-validation strategy
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize GridSearchCV
grid_search = GridSearchCV(estimator=log_reg, param_grid=param_grid, scoring='f1_macro', cv=cv, n_jobs=-1)
start = time.time()

# Fit GridSearchCV
grid_search.fit(X_train, Y_train)
end = time.time() - start
print(f"Time taken: {end} seconds")

# Best parameters from GridSearchCV
best_params = grid_search.best_params_
print(f"Best parameters: {best_params}")

# Best estimator
best_log_reg = grid_search.best_estimator_
print(best_log_reg)

#performing cross validation to the best estimator
cv_score_lr = cross_val_score(best_log_reg, X_train, Y_train, scoring = 'f1_macro', cv=cv)
print("Cross validation score for Logistic regression:", cv_score_lr)
print(f"Mean Score for Logistic regression:{cv_score_lr.mean():.2f} ")

# create a folder where all trained models will be kept
os.makedirs("models", exist_ok=True) # Ensure directory exists

dump(best_log_reg, 'models/log_reg.joblib')
print("Model saved as 'models/log_reg.joblib'")

Time taken: 221.62308263778687 seconds
Best parameters: {'C': 100, 'solver': 'liblinear'}
LogisticRegression(C=100, max_iter=1000, random_state=42, solver='liblinear')
Cross validation score for Logistic regression: [0.20800895 0.2074804 0.20988863 0.21377083 0.21463159]
Mean Score for Logistic regression:0.21
Model saved as 'models/log_reg.joblib'
```

Result on Training Data after Tuning:

Evaluating on Dataset

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report

# Predict on the training and test data
y_pred_train = best_log_reg.predict(X_train)
y_pred_test = best_log_reg.predict(X_test)

# ---- Evaluation on Training Data ----
print(f"Evaluation on Training Data:\n")

# Calculate accuracy
accuracy_lr_train = accuracy_score(Y_train, y_pred_train)
print(f"Logistic Regression Accuracy (Train): {accuracy_lr_train:.2f}")

# Calculate F1 score
f1score_lr_train = f1_score(Y_train, y_pred_train, average='weighted')
print(f"Logistic Regression F1 Score (Train): {f1score_lr_train:.2f}")

# Calculate confusion matrix
conf_matrix_train = confusion_matrix(Y_train, y_pred_train)
print(f"Logistic Regression Confusion Matrix (Train):\n{conf_matrix_train}")

# Print classification report
print("\nClassification Report (Train):\n", classification_report(Y_train, y_pred_train))

# Plot confusion matrix for training data
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_train, annot=True, fmt='d', cmap='YlGnBu')
plt.title('Confusion Matrix (Train)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

Evaluation on Training Data:

```
Logistic Regression Accuracy (Train): 0.71
Logistic Regression F1 Score (Train): 0.60
Logistic Regression Confusion Matrix (Train):
[[ 0 391  0  0 20]
 [ 0 18927  0  0 150]
 [ 0 3559  0  0 31]
 [ 0 796  0  0 22]
 [ 0 3092  0  0 502]]
```

```
Classification Report (Train):
              precision    recall  f1-score   support

    Bad                0.00      0.00      0.00         411
  Excellent            0.71      0.99      0.83        19077
    Good                0.00      0.00      0.00         3590
  Moderate            0.00      0.00      0.00          818
   Very Bad            0.69      0.14      0.23        3594

 accuracy                0.71                0.71        27490
 macro avg              0.28              0.23              0.21        27490
 weighted avg           0.58              0.71              0.60        27490
```

Result on Testing Data after Tuning:

```
# ---- Evaluation on Test Data ----
print(f"\nEvaluation on Test Data:\n")

# Calculate accuracy
accuracy_lr_test = accuracy_score(Y_test, y_pred_test)
print(f"Logistic Regression Accuracy (Test): {accuracy_lr_test:.2f}")

# Calculate F1 score
f1score_lr_test = f1_score(Y_test, y_pred_test, average='weighted')
print(f"Logistic Regression F1 Score (Test): {f1score_lr_test:.2f}")

# Define the class names in the correct order
class_names = ['Bad', 'Excellent', 'Good', 'Moderate', 'Very Bad']

# Calculate confusion matrix
conf_matrix_test = confusion_matrix(Y_test, y_pred_test)
print(f"Logistic Regression Confusion Matrix (Test):\n{conf_matrix_test}")

# Print classification report
print("\nClassification Report (Test):\n", classification_report(Y_test, y_pred_test))

# Plot confusion matrix for test data
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_test, annot=True, fmt='d', cmap='YlGnBu', xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix (Test)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

Evaluation on Test Data:

Logistic Regression Accuracy (Test): 0.69
 Logistic Regression F1 Score (Test): 0.61
 Logistic Regression Confusion Matrix (Test):

```
[[ 2  84  0  2 14]
 [ 30 4470 18 33 219]
 [ 3  841 2 10 42]
 [ 3 166 1 3 32]
 [ 9 631 2 11 245]]
```

Classification Report (Test):

	precision	recall	f1-score	support
Bad	0.04	0.02	0.03	102
Excellent	0.72	0.94	0.82	4770
Good	0.09	0.00	0.00	898
Moderate	0.05	0.01	0.02	205
Very Bad	0.44	0.27	0.34	898
accuracy			0.69	6873
macro avg	0.27	0.25	0.24	6873
weighted avg	0.57	0.69	0.61	6873

IMPLICATIONS OF SMOTE TECHNIQUE:

```
import os
import time
from joblib import dump
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold, cross_val_score
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import SMOTE
import numpy as np

# Encode the target labels if they are not numeric
label_encoder = LabelEncoder()
Y_train_encoded = label_encoder.fit_transform(Y_train)

# Apply SMOTE to balance the class distribution
smote = SMOTE(random_state=42)
X_train_smote, Y_train_smote = smote.fit_resample(X_train, Y_train_encoded)
print(f"Original dataset size: {X_train.shape}, SMOTE dataset size: {X_train_smote.shape}")

# Define Logistic Regression model with automatic class balancing
log_reg = LogisticRegression(max_iter=1000, random_state=42, class_weight="balanced")

# Reduced parameter grid
param_dist = {
    'C': [0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'saga']
}

# Reduced cross-validation splits
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

# Initialize RandomizedSearchCV
random_search = RandomizedSearchCV(
    estimator=log_reg,
    param_distributions=param_dist,
    n_iter=6, # Test a subset of combinations
    scoring='f1_weighted',
    cv=cv,
    n_jobs=-1,
    random_state=42
)

# Fit RandomizedSearchCV
start = time.time()
random_search.fit(X_train_smote, Y_train_smote)
end = time.time() - start
print(f"Time taken: {end:.2f} seconds")

# Best parameters
best_params = random_search.best_params_
print(f"Best parameters: {best_params}")

# Best estimator
best_model = random_search.best_estimator_
print(best_model)

# Perform cross-validation on the best model
cv_score_lr = cross_val_score(best_model, X_train_smote, Y_train_smote, scoring='f1_weighted', cv=cv, n_jobs=-1)
print("Cross-validation score for Logistic Regression:", cv_score_lr)
print(f"Mean Score for Logistic Regression: {cv_score_lr.mean():.2f}")

# Save the best model
os.makedirs("models", exist_ok=True) # Ensure directory exists
dump(best_model, 'models/log_reg_smote.joblib')
print("Model saved as 'models/log_reg_smote.joblib'")

Original dataset size: (27490, 21), SMOTE dataset size: (95385, 21)
Time taken: 514.12 seconds
Best parameters: {'solver': 'saga', 'C': 10}
LogisticRegression(C=10, class_weight='balanced', max_iter=1000,
                    random_state=42, solver='saga')
Cross-validation score for Logistic Regression: [0.33190407 0.33803551 0.34233424]
Mean Score for Logistic Regression: 0.34
Model saved as 'models/log_reg_smote.joblib'
```

Result of SMOTE:

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Predict on the test set
Y_pred_test = best_model.predict(X_test)

# ---- Evaluation on Test Data ----
print(f"\nEvaluation on Test Data:\n")

# Calculate accuracy
accuracy_lr_test = accuracy_score(Y_test, Y_pred_test)
print(f"Logistic Regression Accuracy (Test): {accuracy_lr_test:.2f}")

# Calculate F1 score
f1score_lr_test = f1_score(Y_test, Y_pred_test, average='weighted')
print(f"Logistic Regression F1 Score (Test): {f1score_lr_test:.2f}")

# Calculate confusion matrix
conf_matrix_test = confusion_matrix(Y_test, Y_pred_test)
print(f"Logistic Regression Confusion Matrix (Test):\n{conf_matrix_test}")

# Print classification report
print("\nClassification Report (Test):\n", classification_report(Y_test, Y_pred_test, target_names=class_names))

# Plot confusion matrix for test data
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_test, annot=True, fmt='d', cmap='YlGnBu', xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix (Test)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

Evaluation on Test Data:

Logistic Regression Accuracy (Test): 0.47
 Logistic Regression F1 Score (Test): 0.51
 Logistic Regression Confusion Matrix (Test):

```
[[ 11  46  15   8  22]
 [213 2691 932 495 439]
 [ 61 465 165 101 106]
 [ 13  96  34  26  36]
 [ 52 342 100  94 310]]
```

Classification Report (Test):

	precision	recall	f1-score	support
Bad	0.03	0.11	0.05	102
Excellent	0.74	0.56	0.64	4770
Good	0.13	0.18	0.15	898
Moderate	0.04	0.13	0.06	205
Very Bad	0.34	0.35	0.34	898
accuracy			0.47	6873
macro avg	0.26	0.27	0.25	6873
weighted avg	0.58	0.47	0.51	6873

Appendix 9: Random Forest:

Before Hyperparameter Tuning:

Random Forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from sklearn.preprocessing import LabelEncoder

# If ytrain is not encoded, encode it to numerical values
#label_encoder = LabelEncoder()
#ytrain_encoded_rf = label_encoder.fit_transform(ytrain)

# Initialize the Random Forest Classifier
random_forest = RandomForestClassifier( random_state=42,
    n_estimators=100,          # Keep 100 trees for stability
    max_depth=10,             # Limit maximum tree depth to avoid over-complexity
    min_samples_split=10,     # Require at least 10 samples to split a node
    min_samples_leaf=5,       # Require at least 5 samples per leaf
    max_features='sqrt' )     # Consider only sqrt(number of features) at each split) # n_estimators can be adjusted

# Fit the model
random_forest.fit(X_train, Y_train)

# Predict on the training data
y_pred_rf = random_forest.predict(X_train)
# Calculate accuracy
accuracy_rf = accuracy_score(Y_train, y_pred_rf)
print(f"Random Forest Classifier Accuracy: {accuracy_rf:.2f}")

# Calculate F1 score
f1score_rf = f1_score(Y_train, y_pred_rf, average='weighted')
print(f"Random Forest Classifier F1 Score: {f1score_rf:.2f}")

# Calculate confusion matrix
conf_matrix = confusion_matrix(Y_train, y_pred_rf)
print(f"Random Forest Classifier Confusion Matrix:\n{conf_matrix}")

# Print classification report
print("\nClassification Report:\n", classification_report(Y_train, y_pred_rf))
```

```
Random Forest Classifier Accuracy: 0.72
Random Forest Classifier F1 Score: 0.62
Random Forest Classifier Confusion Matrix:
[[ 0 383  0  0 28]
 [ 0 18968  0  0 109]
 [ 0 3525  0  0 65]
 [ 0 788  0  0 30]
 [ 0 2831  0  0 763]]
```

Classification Report:

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	411
Excellent	0.72	0.99	0.83	19077
Good	0.00	0.00	0.00	3590
Moderate	0.00	0.00	0.00	818
Very Bad	0.77	0.21	0.33	3594
accuracy			0.72	27490
macro avg	0.30	0.24	0.23	27490
weighted avg	0.60	0.72	0.62	27490

After Hyperparameter Tuning:

Random Forest

```
## cross validation for random forest
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.model_selection import RandomizedSearchCV

# Define the Random Forest Classifier model
rf_classifier = RandomForestClassifier(random_state=42)

# Define Parameter Grid for Hyperparameter Tuning
param_grid_rf = {
    'n_estimators': [50, 100], # Fewer trees to avoid overfitting
    'max_depth': [10, 20, None], # Restrict tree depth
    'min_samples_split': [10, 20], # More samples to split nodes
    'min_samples_leaf': [5, 10], # More samples per leaf
    'max_features': ['sqrt', 'log2'],
    'bootstrap': [True]
}

# Define the cross-validation strategy
cv_rf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize RandomizedSearchCV
random_search_rf = RandomizedSearchCV(estimator=rf_classifier,
                                      param_distributions=param_grid_rf,
                                      n_iter=20, # Limit the number of iterations
                                      scoring='f1_macro',
                                      cv=cv_rf,
                                      n_jobs=-1,
                                      random_state=42,
                                      verbose=1)

start = time.time()
random_search_rf.fit(X_train, Y_train)
end = time.time() - start
print(f"Time taken: {end:.2f} seconds")

# Best parameters
best_params_rf = random_search_rf.best_params_
print(f"Best parameters for Random Forest: {best_params_rf}")

# Best estimator
best_rf_classifier = random_search_rf.best_estimator_
print(best_rf_classifier)

# Perform Cross-Validation for the Best Model
cv_score_rf = cross_val_score(best_rf_classifier, X_train, Y_train, scoring='f1_macro', cv=cv_rf)
print("Cross-validation score for Random Forest Classifier:", cv_score_rf)
print(f"Mean score for Random Forest Classifier: {cv_score_rf.mean():.2f}")

# Save the Model
dump(best_rf_classifier, 'models/rf_optimized.joblib')
print("Model saved as 'models/rf_optimized.joblib'")

Fitting 5 folds for each of 20 candidates, totalling 100 fits
Time taken: 119.66 seconds
Best parameters for Random Forest: {'n_estimators': 50, 'min_samples_split': 10, 'min_samples_leaf': 5, 'max_features': 'log2', 'max_depth': None, 'bootstrap': True}
RandomForestClassifier(max_features='log2', min_samples_leaf=5,
                       min_samples_split=10, n_estimators=50, random_state=42)
Cross-validation score for Random Forest Classifier: [0.22837368 0.22692893 0.22522956 0.23044362 0.23489867]
Mean score for Random Forest Classifier: 0.23
Model saved as 'models/rf_optimized.joblib'
```

Result on Training Data after Tuning:

Evaluating on Training Data

```
# Import required libraries
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report

# Predict on the training data
y_pred_train = best_rf_classifier.predict(X_train)
# Predict on the test data
y_pred_test = best_rf_classifier.predict(X_test)

# Calculate accuracy (Train)
accuracy_rf_train = accuracy_score(Y_train, y_pred_train)
print(f"Random Forest Classifier Accuracy (Train): {accuracy_rf_train:.2f}")

# Calculate F1 score (Train)
f1score_rf_train = f1_score(Y_train, y_pred_train, average='weighted')
print(f"Random Forest Classifier F1 Score (Train): {f1score_rf_train:.2f}")

# Calculate confusion matrix (Train)
conf_matrix_train = confusion_matrix(Y_train, y_pred_train)
print(f"Random Forest Classifier Confusion Matrix (Train):\n{conf_matrix_train}")

# Print classification report (Train)
print("\nClassification Report (Train):\n", classification_report(Y_train, y_pred_train))

# Plot confusion matrix for training data
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_train, annot=True, fmt='d', cmap='Greens')
plt.title('Confusion Matrix (Train)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

Random Forest Classifier Accuracy (Train): 0.73

Random Forest Classifier F1 Score (Train): 0.64

Random Forest Classifier Confusion Matrix (Train):

```
[[ 0  377  0  0  34]
 [ 0 18977 0  0 100]
 [ 0  3502 14  0  74]
 [ 0   767  0  0  51]
 [ 0  2544  1  0 1049]]
```

Classification Report (Train):

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	411
Excellent	0.73	0.99	0.84	19077
Good	0.93	0.00	0.01	3590
Moderate	0.00	0.00	0.00	818
Very Bad	0.80	0.29	0.43	3594
accuracy			0.73	27490
macro avg	0.49	0.26	0.25	27490
weighted avg	0.73	0.73	0.64	27490

Result on Testing Data after Tuning:

```
# Import required libraries
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report

# Calculate accuracy (Test)
accuracy_rf_test = accuracy_score(Y_test, y_pred_test)
print(f"Random Forest Classifier Accuracy (Test): {accuracy_rf_test:.2f}")

# Calculate F1 score (Test)
f1score_rf_test = f1_score(Y_test, y_pred_test, average='weighted')
print(f"Random Forest Classifier F1 Score (Test): {f1score_rf_test:.2f}")

# Calculate confusion matrix (Test)
conf_matrix_test = confusion_matrix(Y_test, y_pred_test)
print(f"Random Forest Classifier Confusion Matrix (Test):\n{conf_matrix_test}")

# Print classification report (Test)
print("\nClassification Report (Test):\n", classification_report(Y_test, y_pred_test, target_names=class_names))

# Plot confusion matrix for test data
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_test, annot=True, fmt='d', cmap='Greens', xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix (Test)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

Random Forest Classifier Accuracy (Test): 0.72

Random Forest Classifier F1 Score (Test): 0.62

Random Forest Classifier Confusion Matrix (Test):

```
[[ 0  90  0  0 12]
 [ 0 4731 0  0 39]
 [ 0  883 0  0 15]
 [ 0  192 0  0 13]
 [ 0  687 1  0 210]]
```

Classification Report (Test):

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	102
Excellent	0.72	0.99	0.83	4770
Good	0.00	0.00	0.00	898
Moderate	0.00	0.00	0.00	205
Very Bad	0.73	0.23	0.35	898
accuracy			0.72	6873
macro avg	0.29	0.25	0.24	6873
weighted avg	0.59	0.72	0.62	6873

IMPLICATION OF SMOTE:

SMOTE & OTHER TECHNIQUES

```
# Import required Libraries
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold, cross_val_score
from imblearn.over_sampling import SMOTE
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
import seaborn as sns
from joblib import dump
import time

# Initialize LabelEncoder and encode labels consistently
label_encoder = LabelEncoder()
Y_train_encoded = label_encoder.fit_transform(Y_train) # Fit and transform training labels
Y_test_encoded = label_encoder.transform(Y_test) # Transform test labels using the same encoder

# Apply SMOTE for class imbalance
smote = SMOTE(random_state=42)
X_train_resampled, Y_train_resampled = smote.fit_resample(X_train, Y_train_encoded)

# Define the Random Forest Classifier
rf_classifier = RandomForestClassifier(random_state=42, class_weight="balanced")

# Define the parameter grid for RandomizedSearchCV
param_grid_rf = {
    'n_estimators': [50, 100],
    'max_depth': [10, 20, None],
    'min_samples_split': [10, 20],
    'min_samples_leaf': [5, 10],
    'max_features': ['sqrt']
}

# Define StratifiedKFold for cross-validation
cv_rf = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

# Initialize RandomizedSearchCV
random_search_rf = RandomizedSearchCV(
    estimator=rf_classifier,
    param_distributions=param_grid_rf,
    n_iter=10, # Reduce iterations for faster execution
    scoring='f1_weighted',
    cv=cv_rf,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

# Train the model using RandomizedSearchCV
start = time.time()
random_search_rf.fit(X_train_resampled, Y_train_resampled) # Use resampled data
end = time.time() - start
print(f"Time taken for RandomizedSearchCV: {end:.2f} seconds")

# Get the best parameters and estimator
best_params_rf = random_search_rf.best_params_
print(f"Best parameters for Random Forest: {best_params_rf}")
best_rf_classifier = random_search_rf.best_estimator_

# Perform cross-validation on the best model
cv_score_rf = cross_val_score(best_rf_classifier, X_train_resampled, Y_train_resampled, scoring='f1_weighted', cv=cv_rf, n_jobs=-1)
print("Cross-validation F1 scores for Random Forest Classifier:", cv_score_rf)
print(f"Mean F1 score for Random Forest Classifier: {cv_score_rf.mean():.2f}")

# Save the best model
dump(best_rf_classifier, 'models/rf_optimized_smote_fast.joblib')
print("Optimized Random Forest model with SMOTE saved as 'models/rf_optimized_smote_fast.joblib'")

Fitting 3 folds for each of 10 candidates, totalling 30 fits
Time taken for RandomizedSearchCV: 161.55 seconds
Best parameters for Random Forest: {'n_estimators': 50, 'min_samples_split': 10, 'min_samples_leaf': 5, 'max_features': 'sqrt', 'max_depth': None}
Cross-validation F1 scores for Random Forest Classifier: [0.75832567 0.76475292 0.76529222]
Mean F1 score for Random Forest Classifier: 0.76
Optimized Random Forest model with SMOTE saved as 'models/rf_optimized_smote_fast.joblib'
```

Result on Training data after SMOTE:

```
# ---- Evaluate on Training Data ----
y_pred_train = best_rf_classifier.predict(X_train_resampled)
accuracy_rf_train = accuracy_score(Y_train_resampled, y_pred_train)
f1score_rf_train = f1_score(Y_train_resampled, y_pred_train, average='weighted')
conf_matrix_train = confusion_matrix(Y_train_resampled, y_pred_train)
print(f"\nTraining Accuracy: {accuracy_rf_train:.2f}")
print(f"\nTraining F1 Score: {f1score_rf_train:.2f}")
print("\nTraining Confusion Matrix:\n", conf_matrix_train)
print("\nClassification Report (Test):\n", classification_report(Y_train_resampled, y_pred_train, target_names=class_names))

# Plot training confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_train, annot=True, fmt='d', cmap='Greens', xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.title('Training Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

```
Training Accuracy: 0.90
Training F1 Score: 0.91
Training Confusion Matrix:
[[18695  274    64   13    31]
 [   14 18599   96   38  330]
 [   124  3015 15204  226  508]
 [    33   616  179 18129  120]
 [   171  2015   886  339 15666]]
```

```
Classification Report (Test):
              precision    recall  f1-score   support

    Bad         0.98      0.98      0.98     19077
  Excellent    0.76      0.97      0.85     19077
    Good        0.93      0.80      0.86     19077
  Moderate     0.97      0.95      0.96     19077
  Very Bad     0.94      0.82      0.88     19077

 accuracy              0.90      95385
 macro avg           0.91      0.90      0.91      95385
 weighted avg        0.91      0.90      0.91      95385
```

Result on Testing data after SMOTE:

```
# ---- Evaluate on Test Data ----
y_pred_test = best_rf_classifier.predict(X_test)
accuracy_rf_test = accuracy_score(Y_test_encoded, y_pred_test)
f1score_rf_test = f1_score(Y_test_encoded, y_pred_test, average='weighted')
conf_matrix_test = confusion_matrix(Y_test_encoded, y_pred_test)
print(f"\nTest Accuracy: {accuracy_rf_test:.2f}")
print(f"Test F1 Score: {f1score_rf_test:.2f}")
print("Test Confusion Matrix:\n", conf_matrix_test)
# Print classification report (Test)
print("\nClassification Report (Test):\n", classification_report(Y_test_encoded, y_pred_test, target_names=class_names))
# Plot test confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_test, annot=True, fmt='d', cmap='Greens', xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.title('Test Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

Test Accuracy: 0.69

Test F1 Score: 0.62

Test Confusion Matrix:

```
[[ 1  77  4  4 16]
 [13 4469 77 29 182]
 [ 2  817 22  3  54]
 [ 4  170  9  3  19]
 [ 6  598 23  8 263]]
```

Classification Report (Test):

	precision	recall	f1-score	support
Bad	0.04	0.01	0.02	102
Excellent	0.73	0.94	0.82	4770
Good	0.16	0.02	0.04	898
Moderate	0.06	0.01	0.02	205
Very Bad	0.49	0.29	0.37	898
accuracy			0.69	6873
macro avg	0.30	0.26	0.25	6873
weighted avg	0.59	0.69	0.62	6873

Appendix 10: XGBoost Classifier:

Before Hyperparameter Tuning:

XGBOOST

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from sklearn.preprocessing import LabelEncoder
import seaborn as sns
import matplotlib.pyplot as plt

# Encode the categorical target variable Y_train
label_encoder = LabelEncoder()
Y_train_encoded = label_encoder.fit_transform(Y_train)

# ---- Initialize the XGBoost Classifier ----
xgb_classifier = XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='mlogloss')

# ---- Fit the Model ----
xgb_classifier.fit(X_train, Y_train_encoded)

# ---- Predict on the Training Data ----
y_pred_xgb = xgb_classifier.predict(X_train)

# ---- Calculate Accuracy ----
accuracy_xgb = accuracy_score(Y_train_encoded, y_pred_xgb)
print(f"XGBoost Classifier Accuracy: {accuracy_xgb:.2f}")

# ---- Calculate F1 Score ----
f1score_xgb = f1_score(Y_train_encoded, y_pred_xgb, average='weighted')
print(f"XGBoost Classifier F1 Score: {f1score_xgb:.2f}")

# ---- Calculate Confusion Matrix ----
conf_matrix_xgb = confusion_matrix(Y_train_encoded, y_pred_xgb)
print(f"XGBoost Classifier Confusion Matrix:\n{conf_matrix_xgb}")

# ---- Print Classification Report ----
print("\nClassification Report:\n", classification_report(Y_train_encoded, y_pred_xgb))
```

```
XGBoost Classifier Accuracy: 0.77
XGBoost Classifier F1 Score: 0.71
XGBoost Classifier Confusion Matrix:
[[ 97  301   0   0  13]
 [   0 18968   4   0 105]
 [   0  3144  398   0  48]
 [   0   658   3  135  22]
 [   0  2115   8   1 1470]]
```

```
Classification Report:
              precision    recall  f1-score   support

     0           1.00        0.24        0.38         411
     1           0.75        0.99        0.86       19077
     2           0.96        0.11        0.20        3590
     3           0.99        0.17        0.28         818
     4           0.89        0.41        0.56        3594

 accuracy                   0.77       27490
 macro avg              0.92        0.38        0.46       27490
 weighted avg           0.81        0.77        0.71       27490
```

After Hyperparameter Tuning:

XGBoost

```
import os
import time
from joblib import dump
from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.preprocessing import LabelEncoder

# Encoding the target variable
label_encoder = LabelEncoder()
Y_train_encoded = label_encoder.fit_transform(Y_train)
Y_test_encoded = label_encoder.transform(Y_test)

# Defining the XGBoost model
xgb_clf = XGBClassifier(
    random_state=42,
    eval_metric='mlogloss',
    use_label_encoder=False
)

# Defining the parameter grid
param_grid_xgb = {
    'n_estimators': [50, 100],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 5],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

# Cross-validation strategy
cv_xgb = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize and run GridSearchCV
grid_search_xgb = GridSearchCV(
    estimator=xgb_clf,
    param_grid=param_grid_xgb,
    scoring='f1_macro',
    cv=cv_xgb,
    n_jobs=-1,
    verbose=1
)
```

```
start_time = time.time()
grid_search_xgb.fit(X_train, Y_train_encoded)
end_time = time.time() - start_time
print(f"Time taken for grid search: {end_time:.2f} seconds")

# Retrieve and Print the Best Parameters and Model
best_params_xgb = grid_search_xgb.best_params_
print(f"Best parameters for XGBoost: {best_params_xgb}")

best_xgb_classifier = grid_search_xgb.best_estimator_
print("Best XGBoost estimator:", best_xgb_classifier)

# Performing Cross-Validation on the Best Estimator
cv_score_xgb = cross_val_score(best_xgb_classifier, X_train, Y_train_encoded,
                                scoring='f1_macro', cv=cv_xgb)
print("Cross-validation scores for XGBoost:", cv_score_xgb)
print(f"Mean F1 Macro for XGBoost: {cv_score_xgb.mean():.2f}")

# saving the Best Model
os.makedirs("models", exist_ok=True) # Ensure 'models' directory exists
dump(best_xgb_classifier, 'models/xgb_optimized.joblib')
print("Model saved as 'models/xgb_optimized.joblib'")

Fitting 5 folds for each of 32 candidates, totalling 160 fits
Time taken for grid search: 104.24 seconds
Best parameters for XGBoost: {'colsample_bytree': 1.0, 'learning_rate': 0.1, 'max_depth': 5, 'n_estimators': 100, 'subsample': 1.0}
Best XGBoost estimator: XGBClassifier(base_score=None, booster=None, callbacks=None,
    colsample_bylevel=None, colsample_bynode=None,
    colsample_bytree=1.0, device=None, early_stopping_rounds=None,
    enable_categorical=False, eval_metric='mlogloss',
    feature_types=None, gamma=None, grow_policy=None,
    importance_type=None, interaction_constraints=None,
    learning_rate=0.1, max_bin=None, max_cat_threshold=None,
    max_cat_to_onehot=None, max_delta_step=None, max_depth=5,
    max_leaves=None, min_child_weight=None, missing=nan,
    monotone_constraints=None, multi_strategy=None, n_estimators=100,
    n_jobs=None, num_parallel_tree=None, objective='multi:softprob', ...)
Cross-validation scores for XGBoost: [0.24865933 0.2417186 0.25147783 0.25217218 0.24153349]
Mean F1 Macro for XGBoost: 0.25
Model saved as 'models/xgb_optimized.joblib'
```

Result on Training Data after tuning:

Evaluating the data

```
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report

label_encoder = LabelEncoder()
Y_train_encoded = label_encoder.fit_transform(Y_train) # Y_train is categorical
Y_test_encoded = label_encoder.transform(Y_test)

# Predict on the training data (returns numeric classes)
y_pred_train_encoded = best_xgb_classifier.predict(X_train)

# Decode back to original category names
y_pred_train_xgb = label_encoder.inverse_transform(y_pred_train_encoded)

# Accuracy (Train)
accuracy_xgb_train = accuracy_score(Y_train, y_pred_train_xgb)
print(f"XGBoost Accuracy (Train): {accuracy_xgb_train:.2f}")

# F1 score (Train)
f1score_xgb_train = f1_score(Y_train, y_pred_train_xgb, average='weighted')
print(f"XGBoost F1 Score (Train): {f1score_xgb_train:.2f}")

# Confusion matrix (Train)
conf_matrix_train_xgb = confusion_matrix(Y_train, y_pred_train_xgb)
print(f"XGBoost Confusion Matrix (Train):\n{conf_matrix_train_xgb}")

# Classification report (Train)
print("\nClassification Report (Train):\n", classification_report(Y_train, y_pred_train_xgb))

# Plot confusion matrix for training data
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_train_xgb, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.title("XGBoost Confusion Matrix (Train)")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.show()
```

```
XGBoost Accuracy (Train): 0.79
XGBoost F1 Score (Train): 0.75
XGBoost Confusion Matrix (Train):
[[ 165  242    0    0    4]
 [   7 18937   25    4  104]
 [   3  2862  681    1   43]
 [   0   595    0  213   10]
 [   2  1831   13    4 1744]]
```

```
Classification Report (Train):
```

	precision	recall	f1-score	support
Bad	0.93	0.40	0.56	411
Excellent	0.77	0.99	0.87	19077
Good	0.95	0.19	0.32	3590
Moderate	0.96	0.26	0.41	818
Very Bad	0.92	0.49	0.63	3594
accuracy			0.79	27490
macro avg	0.91	0.47	0.56	27490
weighted avg	0.82	0.79	0.75	27490

Result on Testing data after Tuning:

```
# Predict on the test data (numeric)
y_pred_test_encoded = best_xgb_classifier.predict(X_test)

# Decode to original category labels
y_pred_test_xgb = label_encoder.inverse_transform(y_pred_test_encoded)

# Accuracy (Test)
accuracy_xgb_test = accuracy_score(Y_test, y_pred_test_xgb)
print(f"XGBoost Accuracy (Test): {accuracy_xgb_test:.2f}")

# F1 score (Test)
f1score_xgb_test = f1_score(Y_test, y_pred_test_xgb, average='weighted')
print(f"XGBoost F1 Score (Test): {f1score_xgb_test:.2f}")

# Confusion matrix (Test)
conf_matrix_test_xgb = confusion_matrix(Y_test, y_pred_test_xgb)
print(f"XGBoost Confusion Matrix (Test):\n{conf_matrix_test_xgb}")

# Classification report (Test)
print("\nClassification Report (Test):\n", classification_report(Y_test, y_pred_test_xgb))

# Plot confusion matrix for test data
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_test_xgb, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.title("XGBoost Confusion Matrix (Test)")
plt.ylabel("Actual")
plt.xlabel("Predicted")
plt.show()
```

```
XGBoost Accuracy (Test): 0.71
XGBoost F1 Score (Test): 0.63
XGBoost Confusion Matrix (Test):
[[ 0  85  1  0 16]
 [ 9 4612 35  9 105]
 [ 0  846 17  2 33]
 [ 1 178  4  1 21]
 [ 0  633 14  2 249]]
```

```
Classification Report (Test):
```

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	102
Excellent	0.73	0.97	0.83	4770
Good	0.24	0.02	0.04	898
Moderate	0.07	0.00	0.01	205
Very Bad	0.59	0.28	0.38	898
accuracy			0.71	6873
macro avg	0.32	0.25	0.25	6873
weighted avg	0.61	0.71	0.63	6873

IMPLICATION OF SMOTE TECHNIQUE:

SMOTE & OTHER TECHNIQUES

```
# Import required libraries
import xgboost as xgb
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
from joblib import dump
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import LabelEncoder
import time

# Initialize LabelEncoder and encode labels consistently
label_encoder = LabelEncoder()
Y_train_encoded = label_encoder.fit_transform(Y_train) # Fit and transform training labels
Y_test_encoded = label_encoder.transform(Y_test)      # Transform test labels using the same encoder

# Apply SMOTE for class imbalance
smote = SMOTE(random_state=42)
X_train_resampled, Y_train_resampled = smote.fit_resample(X_train, Y_train_encoded)

# Define the XGBoost Classifier
xgb_classifier = xgb.XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss')

# Define the parameter grid for RandomizedSearchCV
param_grid_xgb = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.2],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'gamma': [0, 0.1, 0.2],
    'min_child_weight': [1, 5, 10]
}

# Define StratifiedKFold for cross-validation
cv_xgb = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

# Initialize RandomizedSearchCV
random_search_xgb = RandomizedSearchCV(
    estimator=xgb_classifier,
    param_distributions=param_grid_xgb,
    n_iter=20, # Reduce iterations for faster execution
    scoring='f1_weighted',
    cv=cv_xgb,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

# Train the model using RandomizedSearchCV
start = time.time()
random_search_xgb.fit(X_train_resampled, Y_train_resampled)
end = time.time() - start
print(f"Time taken for RandomizedSearchCV: {end:.2f} seconds")

# Get the best parameters and estimator
best_params_xgb = random_search_xgb.best_params_
print(f"Best parameters for XGBoost: {best_params_xgb}")
best_xgb_classifier = random_search_xgb.best_estimator_

# Save the best model
dump(best_xgb_classifier, 'models/xgb_optimized_smote.joblib')
print("Optimized XGBoost model with SMOTE saved as 'models/xgb_optimized_smote.joblib'")

Fitting 3 folds for each of 20 candidates, totalling 60 fits
Time taken for RandomizedSearchCV: 205.76 seconds
Best parameters for XGBoost: {'subsample': 0.8, 'n_estimators': 200, 'min_child_weight': 1, 'max_depth': 7, 'learning_rate': 0.2, 'gamma': 0.1, 'colsample_bytree': 0.6}
Optimized XGBoost model with SMOTE saved as 'models/xgb_optimized_smote.joblib'
```

Result on Training data after SMOTE:

```
# ---- Evaluate on Training Data ----
y_pred_train = best_xgb_classifier.predict(X_train_resampled)
accuracy_xgb_train = accuracy_score(Y_train_resampled, y_pred_train)
f1score_xgb_train = f1_score(Y_train_resampled, y_pred_train, average='weighted')
conf_matrix_train = confusion_matrix(Y_train_resampled, y_pred_train)
print(f"\nTraining Accuracy: {accuracy_xgb_train:.2f}")
print(f"Training F1 Score: {f1score_xgb_train:.2f}")
print("Training Confusion Matrix:\n", conf_matrix_train)

# Plot training confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_train, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.title('Training Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

Training Accuracy: 0.89
Training F1 Score: 0.89
Training Confusion Matrix:
[[18623   253   106    70    25]
 [    7 18937    25     4   104]
 [   406 2915 14609   641   506]
 [   182   618   350 17768   159]
 [   344 1857   926   542 15408]]
```

Result on Testing Data after SMOTE:

```
# ---- Evaluate on Test Data ----
y_pred_test = best_xgb_classifier.predict(X_test)
accuracy_xgb_test = accuracy_score(Y_test_encoded, y_pred_test)
f1score_xgb_test = f1_score(Y_test_encoded, y_pred_test, average='weighted')
conf_matrix_test = confusion_matrix(Y_test_encoded, y_pred_test)
print(f"\nTest Accuracy: {accuracy_xgb_test:.2f}")
print(f"Test F1 Score: {f1score_xgb_test:.2f}")
print("Test Confusion Matrix:\n", conf_matrix_test)

# Plot test confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_test, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_)
plt.title('Test Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Classification Report for Test Data
print("\nTest Classification Report:\n", classification_report(Y_test_encoded, y_pred_test, target_names=label_encoder.classes_))

Test Accuracy: 0.71
Test F1 Score: 0.63
Test Confusion Matrix:
[[    0    85     1     0    16]
 [    9 4612    35     9   105]
 [    0   846    17     2    33]
 [    1   178     4     1    21]
 [    0   633    14     2   249]]
```